

Computational Fluid Dynamics

EN 530.767

Final Project: Flow Around Cylinder

Weixuan Li

Instructor: Prof. Rajat Mittal

Contents

1. Introduction	1
1.1 Background	1
1.2 Problem definition	1
1.3 Objective	1
2. Methods	2
2.1 Navier-Stokes equation	2
2.2 Time and space discretization	2
2.3 Algorithm procedure	2
2.4 Boundary condition	5
2.4.1 Outer boundary condition	5
2.4.2 Outflow velocity correction	6
2.4.3 Immersed boundary condition	6
2.5 Implementation of Line SOR	8
3. Results	9
3.1 Channel flow	9
3.2 Configuration of flow around cylinder	11
3.3 Flow around a cylinder at $Re_D = 150$	12
3.3.1 Pressure, velocity, and vorticity contours	12
3.3.2 Pressure drag coefficient and vortex-shedding frequency	16
3.4 Flow around a cylinder at different Reynolds number	17
3.4.1 Flow around a cylinder at $Re_D = 300$	17

3.4.2 Flow around a cylinder at $Re_D = 1000$	21
3.5 Flow around a cylinder at $Re_D = 150$ at different grid resolutions	24
3.5.1 Flow around a cylinder at $dx = 1/16$	24
3.5.2 Flow around a cylinder at $dx = 1/64$	28
3.6 Flow around a cylinder at $Re_D = 150$ with different domain sizes	31
3.6.1 Flow around a cylinder at in large domain	31
3.6.2 Flow around a cylinder at in small domain	35
4. Discussions	38
4.1 Comparison of pressure, velocity, and vorticity contours	38
4.1.1 Comparison of results at different Reynolds numbers	38
4.1.2 Comparison of results at different grid resolutions	40
4.1.3 Comparison of results with different domain sizes	41
4.2 Global results for different cases	42
5. Flow around an elliptic cylinder with attack angle 30°	43
5.1 Configuration of flow around an elliptic cylinder	43
5.2 Pressure velocity and vorticity contours	44
5.3 Lift and pressure drag coefficients	47
6. Conclusions	49
7. References	50
8. Appendix	50

1. Introduction

1.1 Background

In the field of fluid dynamics, flow around a cylinder is a classical and extensively studied topic. When a fluid such as air or water flows past a fixed cylindrical object, it forms complex flow structures behind the cylinder, including flow separation, wakes, and vortex shedding. These phenomena are crucial for engineering applications such as the design of bridges, buildings, and offshore structures, and also provide profound insights into fluid dynamics.

This project aims to explore the characteristics of flow around a cylinder through numerical simulation, analyzing the behavior of the fluid at various Reynolds numbers. This report will outline the simulation setup, numerical schemes used, and key findings based on the results.

1.2 Problem definition

Consider a circular cylinder in a cross-flow, as shown in Figure 1. The flow Reynolds number based on D is $Re_D = 150$. The space is discretized using 2nd order central difference scheme with forward Euler for the convection term and backward Euler for the diffusion term, and the time is discretized by mixed implicit-explicit scheme.

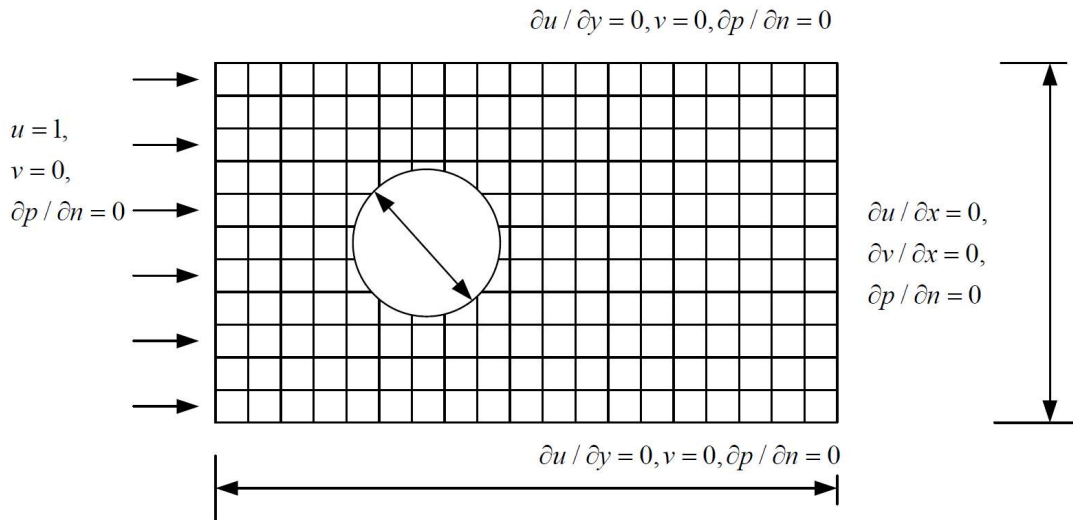


Figure 1 Configuration of flow around a cylinder problem

1.3 Objective

Simulate the flow using a Cartesian grid and the stair-Step, immersed boundary method, and a fractional-step method for time-stepping at $Re_D = 150, 300, 1000$. Plot velocity, pressure, and vorticity contours at several time instances. Quantify the vortex shedding frequency, the pressure drag coefficient on the cylinder and compare the value against existing published studies. In addition, examine and describe the effect of domain size and grid resolution on the simulation results.

2. Methods

2.1 Navier-Stokes equation

The governing equation, incompressible Navier-Stokes equation, can be written in a non-dimensional form as:

$$\begin{aligned}\frac{\partial u}{\partial t} + \nabla \cdot (Uu) &= -\nabla p + \frac{1}{Re} \nabla^2 u \\ \nabla \cdot u &= 0\end{aligned}$$

2.2 Time and space discretization

The time is discretized by a second-order fractional time step:

$$\begin{aligned}\frac{u^* - u^n}{\Delta t} &= -(\nabla \cdot (Uu))^n + \frac{1}{Re} (\nabla^2 u)^{mix(*,n)} \\ \frac{u^{n+1} - u^*}{\Delta t} &= -\nabla p^{n+1}\end{aligned}$$

The illustration of grid layout of discretized points is shown in Figure 2.

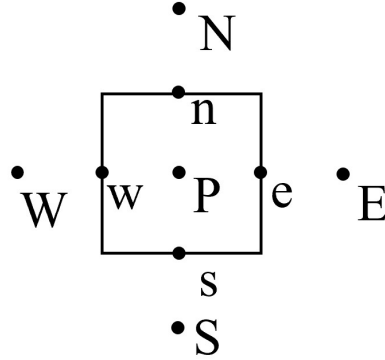


Figure 2 Illustration of grid layout of discretized points

2.3 Algorithm procedure

1. Step 1: Calculate u^*, U^*

We calculate the medium velocity u^*, U^* by the equation:

$$\frac{u^* - u^n}{\Delta t} = -(\nabla \cdot (Uu))^{mix(n,n-1)} + \frac{1}{Re} (\nabla^2 u)^{mix(*,n)}$$

where, the convection term is discretized by the second-order Adam-Bashforth method with staggered mesh. The numerical scheme can be written as:

$$\begin{aligned}
& (\nabla \cdot (Uu))^{mix(n,n-1)} \\
&= \frac{1}{2} \left[3 * \left(\frac{U_e \frac{u_E + u_P}{2} - U_w \frac{u_W + u_P}{2}}{\Delta x} + \frac{V_n \frac{u_N + u_P}{2} - V_s \frac{u_S + u_P}{2}}{\Delta y} \right)^n \right. \\
&\quad \left. - \left(\frac{U_e \frac{u_E + u_P}{2} - U_w \frac{u_W + u_P}{2}}{\Delta x} + \frac{V_n \frac{u_N + u_P}{2} - V_s \frac{u_S + u_P}{2}}{\Delta y} \right)^{n-1} \right]
\end{aligned}$$

The viscous term can be discretized by Crank–Nicolson method. The numerical scheme can be written as:

$$\begin{aligned}
& \frac{1}{Re} (\nabla^2 u)^{mix(*,n)} \\
&= \frac{1}{Re} \left[\frac{1}{2} \left(\frac{u_E - 2u_P + u_W}{\Delta x^2} + \frac{u_N - 2u_P + u_S}{\Delta y^2} \right)^* \right. \\
&\quad \left. + \frac{1}{2} \left(\frac{u_E - 2u_P + u_W}{\Delta x^2} + \frac{u_N - 2u_P + u_S}{\Delta y^2} \right)^n \right]
\end{aligned}$$

Then, the whole equation can be discretized as

$$\begin{aligned}
\left[I - \frac{\Delta t}{2Re} \nabla^2 \right] u^* &= -\frac{3}{2} \Delta t (\nabla \cdot (Uu))^n + \frac{1}{2} \Delta t (\nabla \cdot (Uu))^{n-1} + \left[I + \frac{\Delta t}{2Re} \nabla^2 \right] u^n \\
&\Rightarrow c_W u_W + c_E u_E + c_P u_P + c_N u_N + c_S u_S = RHS
\end{aligned}$$

where

$$\begin{aligned}
c_W &= -\frac{\Delta t}{2Re\Delta x^2} \\
c_E &= -\frac{\Delta t}{2Re\Delta x^2} \\
c_N &= -\frac{\Delta t}{2Re\Delta y^2} \\
c_S &= -\frac{\Delta t}{2Re\Delta y^2} \\
c_P &= 1 + \frac{\Delta t}{2Re} \left(\frac{2}{\Delta x^2} + \frac{2}{\Delta y^2} \right) \\
RHS &= -\frac{3}{2} \Delta t (\nabla \cdot (Uu))^n + \frac{1}{2} \Delta t (\nabla \cdot (Uu))^{n-1} + u^n \\
&\quad + \frac{\Delta t}{2Re} \left(\frac{u_E - 2u_P + u_W}{\Delta x^2} + \frac{u_N - 2u_P + u_S}{\Delta y^2} \right)^n
\end{aligned}$$

Using line SOR to solve this matrix equation, we can obtain u^* . The details of the implementation of line SOR will be described in section 2.5. Then, calculate U^* as the average of u^* :

$$U_e^* = \frac{u_E^* + u_P^*}{2}, V_n^* = \frac{v_N^* + v_P^*}{2}$$

2. Step 2: Solve Poisson equation to obtain pressure

To make sure that the velocity satisfy the divergence free requirement, the pressure should satisfy the Poisson equation:

$$\begin{aligned} \nabla^2 p &= \frac{\nabla \cdot U^*}{\Delta t} = \frac{1}{\Delta t} \left(\frac{U_e^* - U_w^*}{\Delta x} + \frac{V_n^* - V_s^*}{\Delta y} \right) \\ \Rightarrow \frac{p_E - 2p_P + p_W}{\Delta x^2} + \frac{p_N - 2p_P + p_S}{\Delta y^2} &= \frac{1}{\Delta t} \left(\frac{U_e^* - U_w^*}{\Delta x} + \frac{V_n^* - V_s^*}{\Delta y} \right) \\ \Rightarrow c_W p_W + c_E p_E + c_P p_P + c_N p_N + c_S p_S &= RHS \end{aligned}$$

where

$$\begin{aligned} c_W &= \frac{1}{\Delta x^2} \\ c_E &= \frac{1}{\Delta x^2} \\ c_N &= \frac{1}{\Delta y^2} \\ c_S &= \frac{1}{\Delta y^2} \\ c_P &= -\left(\frac{2}{\Delta x^2} + \frac{2}{\Delta y^2} \right) \\ RHS &= \frac{1}{\Delta t} \left(\frac{U_e^* - U_w^*}{\Delta x} + \frac{V_n^* - V_s^*}{\Delta y} \right) \end{aligned}$$

Similarly, using line SOR to solve this matrix equation, we can obtain p .

3. Step 3: Update $u^{n+1}, U^{n+1}, v^{n+1}, V^{n+1}$

$$\begin{aligned} u^{n+1} &= u^* - \Delta t (\nabla p)_{cell\ center} = u^* - \Delta t \frac{p_E - p_W}{2\Delta x} \\ U^{n+1} &= U^* - \Delta t (\nabla p)_{face\ center} = U^* - \Delta t \frac{p_E - p_P}{\Delta x} \\ v^{n+1} &= v^* - \Delta t (\nabla p)_{cell\ center} = v^* - \Delta t \frac{p_N - p_S}{2\Delta y} \end{aligned}$$

$$V^{n+1} = V^* - \Delta t (\nabla p)_{face\ center} = V^* - \Delta t \frac{p_N - p_P}{\Delta y}$$

2.4 boundary condition

2.4.1 Outer boundary condition

For the velocity boundary condition, either Dirichlet or Neumann boundary condition is applied on the wall for velocity, as shown in Figure 1. To achieve this, we set a layer of ghost points around the outermost layer, as shown in Figure 3.

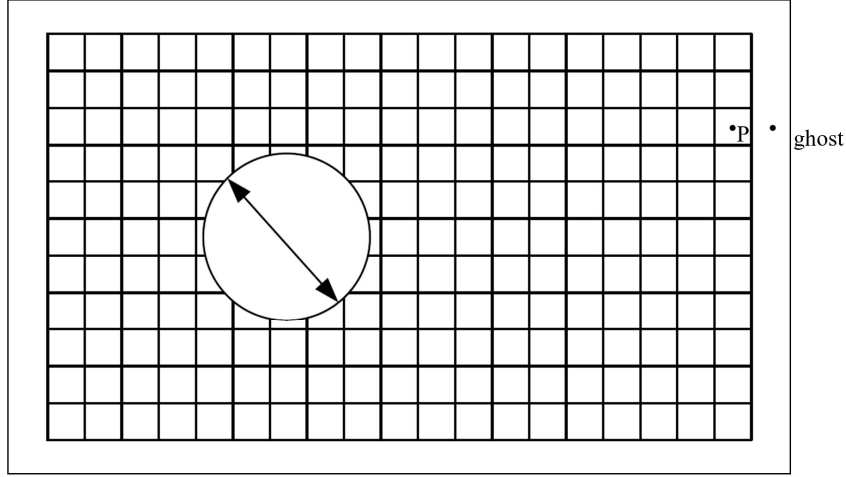


Figure 3 Illustration of layout of ghost points

Let the velocity of wall be the average of the ghost point and the point right near the wall (assume this point is point P). Then

$$\frac{u_{ghos} + u_P}{2} = u_{wall}$$

For Neumann boundary condition, the velocity gradient is 0. Therefore, the velocity of ghost point is equal to velocity of point P:

$$\frac{\partial u}{\partial x} = 0 \Rightarrow \frac{u_P - u_{ghost}}{\Delta x} = 0$$

The same as the boundary condition of pressure:

$$\frac{\partial p}{\partial n} = 0 \Rightarrow \frac{p_P - p_{ghost}}{\Delta x} = 0$$

Therefore, the four sizes of boundary conditions can be implemented as:

1. Inlet:

$$\begin{aligned} u_{ghost} &= 2u_{wall} - u_P = 2 - u_P \\ v_{ghos} &= 2v_{wall} - v_P = -v_P \end{aligned}$$

$$p_{ghost} = p_P$$

2. Upper wall:

$$\begin{aligned} u_{ghost} &= u_P \\ v_{ghost} &= 2v_{wall} - v_P = -v_P \\ p_{ghost} &= p_P \end{aligned}$$

3. Outlet:

$$\begin{aligned} u_{ghost} &= u_P \\ v_{ghost} &= v_P \\ p_{ghost} &= p_P \end{aligned}$$

4. Bottom wall:

$$\begin{aligned} u_{ghost} &= u_P \\ v_{ghost} &= -v_P \\ p_{ghost} &= p_P \end{aligned}$$

2.4.2 Outflow velocity correction

To satisfy the global mass conservation, we need to correct the net flow at the outlet according to the inlet flow:

$$u_{correct}^* = \frac{\int u_{outflow}^* dy - \int u_{inflow}^* dy}{L_y} = \frac{\sum u_{outflow}^* \Delta y - \sum u_{inflow}^* \Delta y}{L_y}$$

$$u_{outflow}^* = u_{inflow}^* + u_{correct}^*$$

2.4.3 Immersed boundary condition

For the inner boundary around the cylinder, we implement the Stair-Step, immersed boundary condition. First, we create a matrix named “ifluid” for each cell, where $ifluid = 1$ for fluid cell, and $ifluid = 0$ for solid cell. Then, we use “ifluid” to correct the stencil for velocity and pressure.

1. Dirichlet boundary condition for velocity

For velocity, the original stencil is:

$$c_W u_W + c_E u_E + c_P u_P + c_N u_N + c_S u_S = RHS$$

If the east point is solid, as shown in Figure 4, the boundary condition is

$$\frac{u_E + u_P}{2} = u_{wall} \Rightarrow u_E = 2u_{wall} - u_P$$

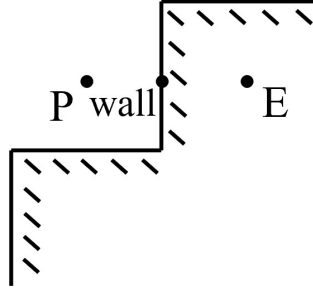


Figure 4 Configuration of Stair-Step boundary Method

Substitute the new u_E into the original equation,

$$\begin{aligned} c_W u_W + c_E (2u_{wall} - u_P) + c_P u_P + c_N u_N + c_S u_S &= RHS \\ \Rightarrow c_W u_W + (c_P - c_E) u_P + c_N u_N + c_S u_S &= RHS - 2c_E u_{wall} \end{aligned}$$

Therefore,

$$\begin{aligned} c_P^{new} &= c_P^{old} - c_E^{old} \\ c_E^{new} &= 0 \end{aligned}$$

This is similar when the west point, the north point, or the south point is solid. To integrate, the updated coefficients for the Poisson equation of velocity are:

$$\begin{aligned} c_P^{new} &= c_P^{old} - (1 - ifluid_W) c_W^{old} - (1 - ifluid_E) c_E^{old} - (1 - ifluid_N) c_N^{old} \\ &\quad - (1 - ifluid_S) c_S^{old} \\ c_E^{new} &= ifluid_E c_E^{old} \\ c_W^{new} &= ifluid_W c_W^{old} \\ c_N^{new} &= ifluid_N c_N^{old} \\ c_S^{new} &= ifluid_S c_S^{old} \end{aligned}$$

The updated RHS is:

$$RHS^{new} = RHS^{old} - 2c_E u_{wall}$$

2. Neumann boundary condition for pressure

For pressure, the original stencil is:

$$c_W p_W + c_E p_E + c_P p_P + c_N p_N + c_S p_S = RHS$$

If the east point is solid, the boundary condition is

$$\frac{\partial p}{\partial n} = 0 \Rightarrow \frac{p_P - p_E}{\Delta x} = 0 \Rightarrow p_E = p_P$$

Substitute the new p_E into the original equation,

$$c_W p_W + c_E p_P + c_P p_P + c_N p_N + c_S p_S = RHS$$

$$\Rightarrow c_W p_W + (c_P + c_E) p_P + c_N p_N + c_S p_S = RHS$$

Therefore,

$$\begin{aligned} c_P^{new} &= c_P^{old} + c_E^{old} \\ c_E^{new} &= 0 \\ RHS^{new} &= RHS^{old} \end{aligned}$$

This is similar when the west point, the north point, or the south point is solid. To integrate, the updated coefficients for the Poisson equation of pressure are:

$$\begin{aligned} c_P^{new} &= c_P^{old} + (1 - ifluid_W) c_W^{old} + (1 - ifluid_E) c_E^{old} + (1 - ifluid_N) c_N^{old} \\ &\quad + (1 - ifluid_S) c_S^{old} \\ c_E^{new} &= ifluid_E c_E^{old} \\ c_W^{new} &= ifluid_W c_W^{old} \\ c_N^{new} &= ifluid_N c_N^{old} \\ c_S^{new} &= ifluid_S c_S^{old} \end{aligned}$$

The RHS is:

$$RHS^{new} = RHS^{old}$$

2.5 Implementation of Line SOR

To solve the Poisson equation for both velocity and pressure

$$\begin{aligned} c_W^{new} u_W + c_E^{new} u_E + c_P^{new} u_P + c_N^{new} u_N + c_S^{new} u_S &= RHS^{new} \\ c_W^{new} p_W + c_E^{new} p_E + c_P^{new} p_P + c_N^{new} p_N + c_S^{new} p_S &= RHS^{new} \end{aligned}$$

To implement Line SOR, we need to move the terms containing the values of the North and South points to the right hand side. Then the equation can be written as:

$$\begin{aligned} c_W^{new} u_W + c_E^{new} u_E + c_P^{new} u_P &= RHS^{new} - c_N^{new} u_N - c_S^{new} u_S \\ c_W^{new} p_W + c_E^{new} p_E + c_P^{new} p_P &= RHS^{new} - c_N^{new} p_N - c_S^{new} p_S \end{aligned}$$

Write it in a matrix form, it can be (including the boundary):

$$\begin{bmatrix} 1 & 0 & & & & & & \\ a & b & c & & & & & \\ & a & b & c & & & & \\ & & a & b & c & & & \\ & & & \ddots & \ddots & \ddots & & \\ & & & & a & b & c & \\ & & & & & a & b & c \\ & & & & & & 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ \vdots \\ u_{n-2} \\ u_{n-1} \\ u_n \end{bmatrix} = \begin{bmatrix} d_1 \\ d_2 \\ d_3 \\ d_4 \\ \vdots \\ d_{n-2} \\ d_{n-1} \\ d_n \end{bmatrix}$$

where $b = c_p^{new}$, $a = c_w^{new}$, $c = c_E^{new}$, $d_n = RHS^{new} - c_N^{new}u_N - c_S^{new}u_S$. The matrix calculation of pressure is the same. Solve this matrix equation by TDMA and iterate until convergence, and then u and p are obtained.

3. Results

3.1 Channel flow

First, it's necessary to test the code by simulating the channel flow, where there are no-slip boundary conditions on the top and bottom walls. To make sure the flow can be fully developed, the length in x direction is set to be much longer than that in y direction, with $L_x = 10, L_y = 1$. The Reynolds number is selected as $Re_{L_y} = 150$.

The computational domain and boundary conditions are shown in Figure 5.

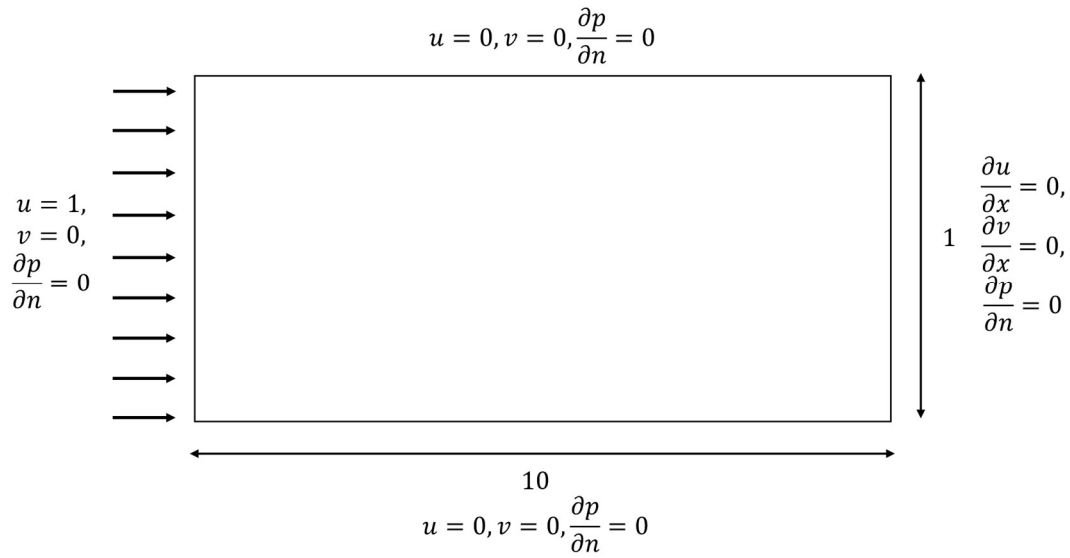


Figure 5 Computational domain and boundary conditions of channel flow

The initial conditions are all zeros for u, v, p . The grid size is $dx = dy = 1/64$. Therefore, the number of grids in horizontal and vertical directions are $N_x = 640, N_y = 64$, respectively. After the simulation converged, the flow reached the steady state. The streamlines, velocity contour, and pressure contour at steady state are shown in Figure 6-8.

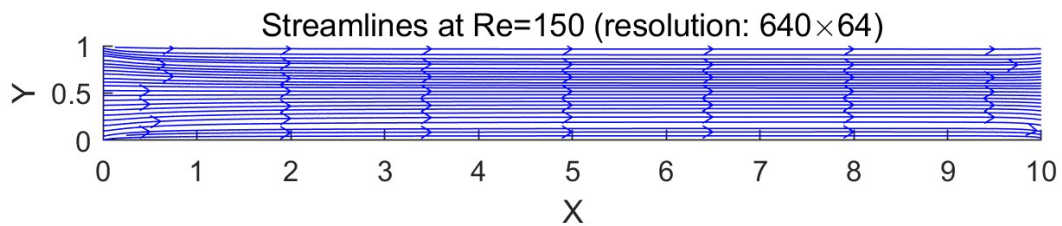


Figure 6 Streamlines of channel flow at steady state

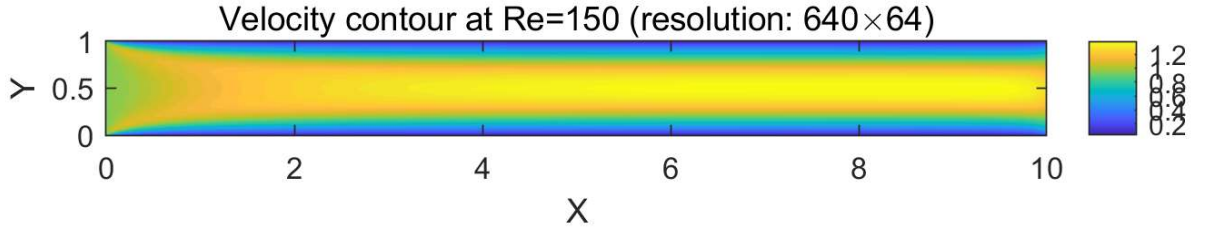


Figure 7 u velocity contour of channel flow at steady state

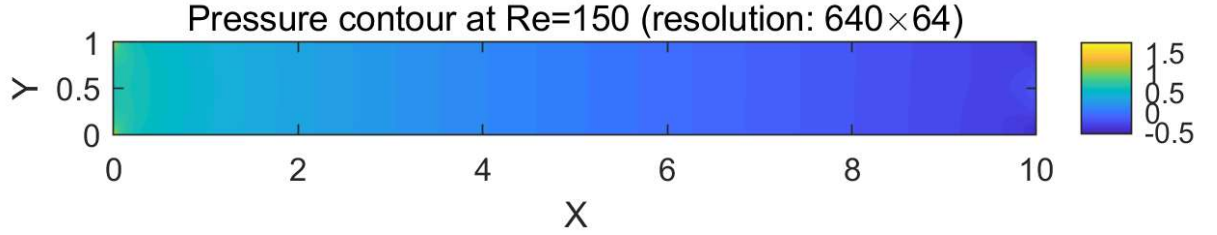


Figure 8 u velocity contour of channel flow at steady state

From Figure 6, streamlines flow from the inlet towards the outlet, with no streamlines penetrating the walls. As depicted in Figure 7, two boundary layers develop near the upper and bottom walls, which gradually thicken from the entrance to the exit. Consequently, the velocity increases gradually from the entrance to the exit. The maximum velocity is situated at the horizontal centerline of the channel, and then gradually decreases towards both sides. An evident pressure gradient exists, as illustrated in Figure 8, with pressure decreasing stepwise from the inlet to the outlet.

To further compare with theoretical channel flow, we select a cross-section in fully developed area to plot the velocity profile of u . In the fully developed region, the analytical velocity profile can be expressed as follows (assuming the horizontal centerline is at 1):

$$u = U \left[1 - \left(\frac{y-1}{H/2} \right)^2 \right]$$

where U is the maximum velocity, located at the centerline, and H is the channel height. The analytical velocity profile is plotted alongside the numerical velocity profile in fully developed region in Figure 9.

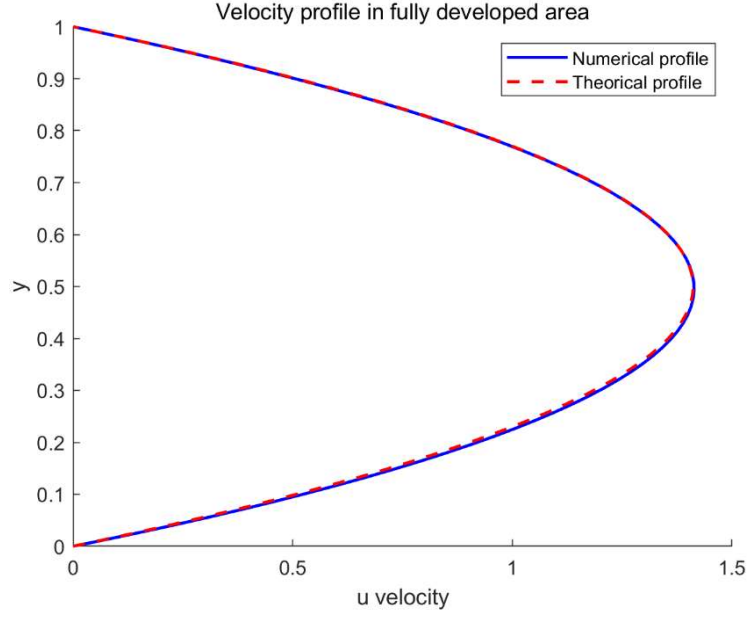


Figure 9 Cross-sectional u velocity profile in fully developed area

The close agreement between the two profiles indicates the high accuracy of this algorithm.

3.2 Configuration of flow around cylinder

Figure 10 illustrates the configuration of flow around a cylinder. Normalized by the cylinder's diameter, the computational domain dimensions are $L_x = 12, L_y = 6$. The Reynolds number is set to be $Re_D = 150$. The initial conditions for velocity components u , v , and pressure p are all zeros. The grid size is $dx = dy = 1/32$. Consequently, the number of grids in the horizontal and vertical directions are $N_x = 384, N_y = 192$, respectively. The cylinder is positioned far from the outer boundaries to minimize the adverse effects of these boundaries.

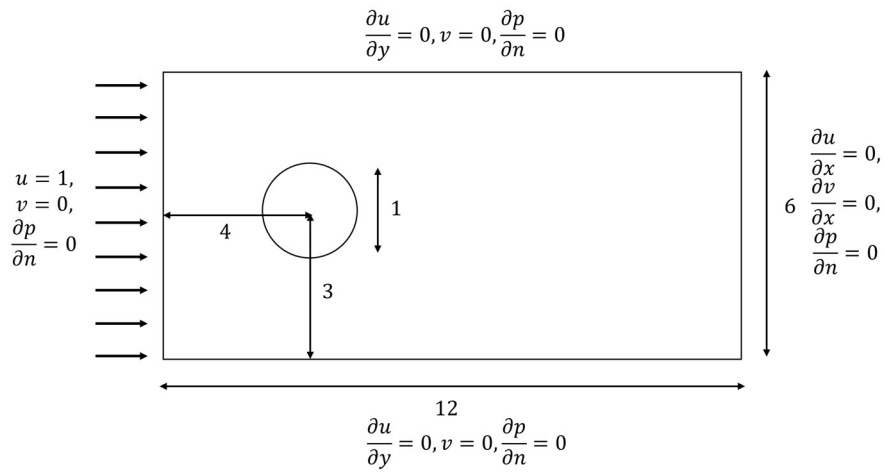
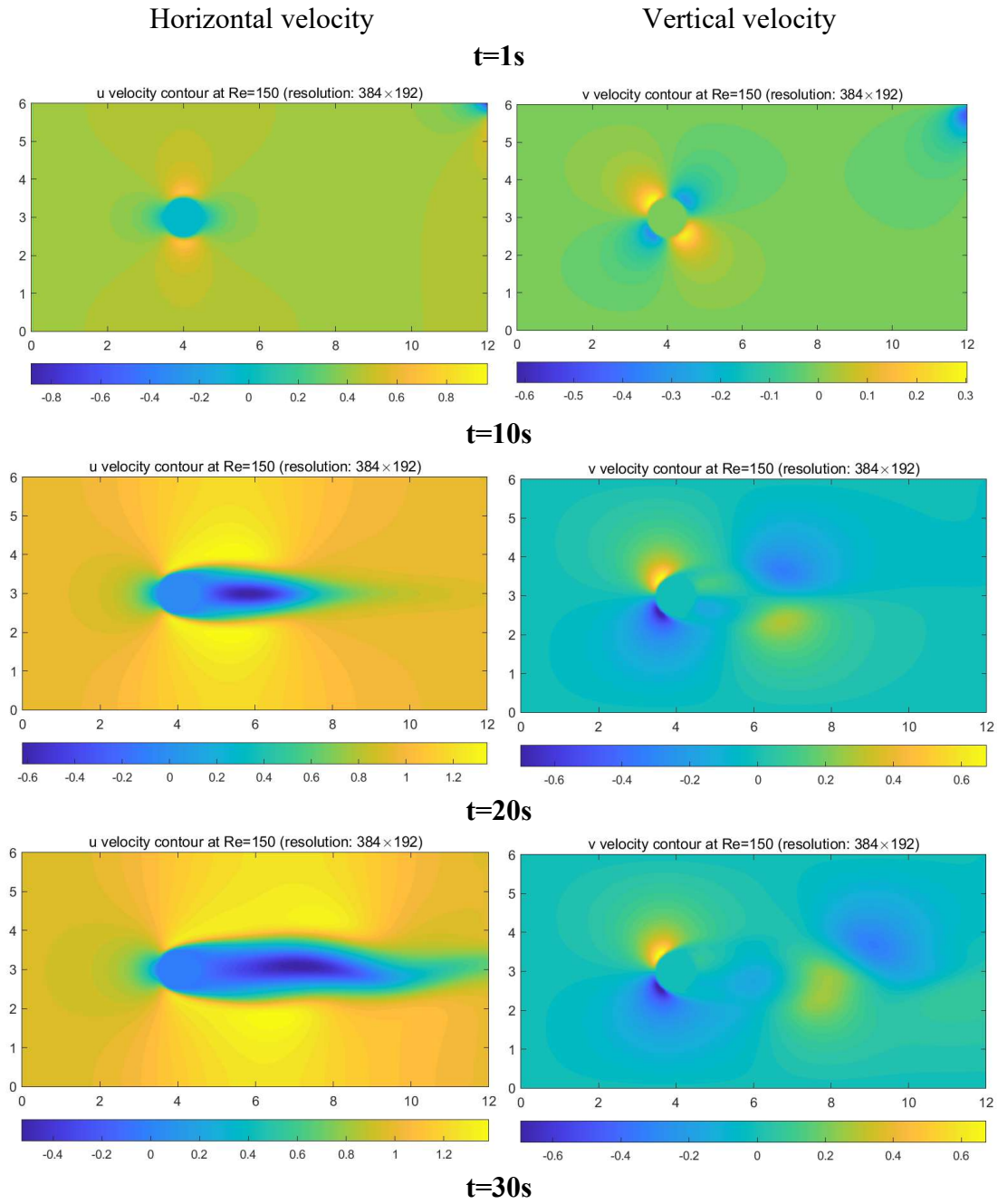


Figure 10 Computational domain and boundary conditions of flow around cylinder

3.3 Flow around a cylinder at $Re_D = 150$

3.3.1 Pressure, velocity, and vorticity contours

The pressure, velocity, vorticity contours are shown in Figure 11 and Figure 12.



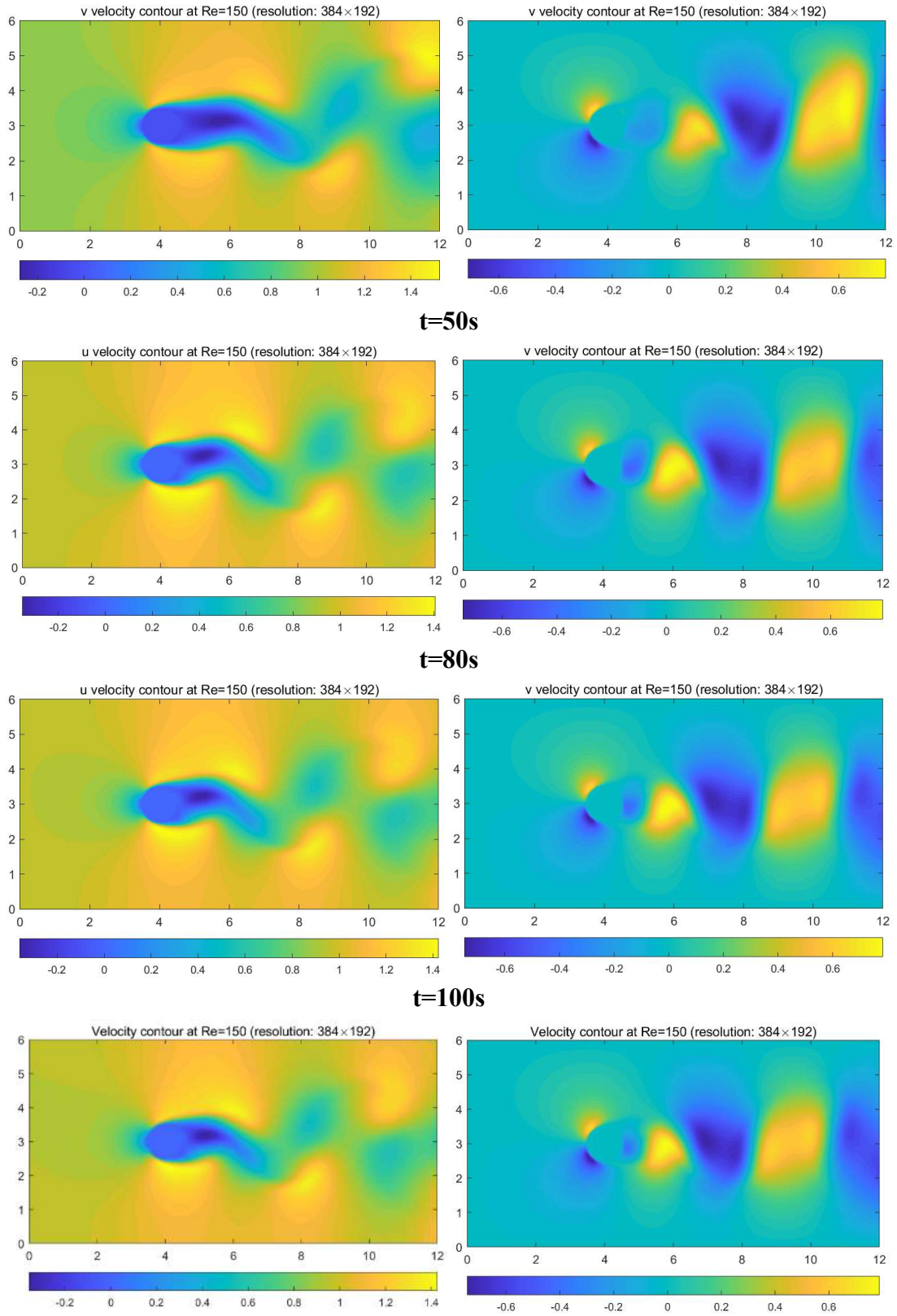
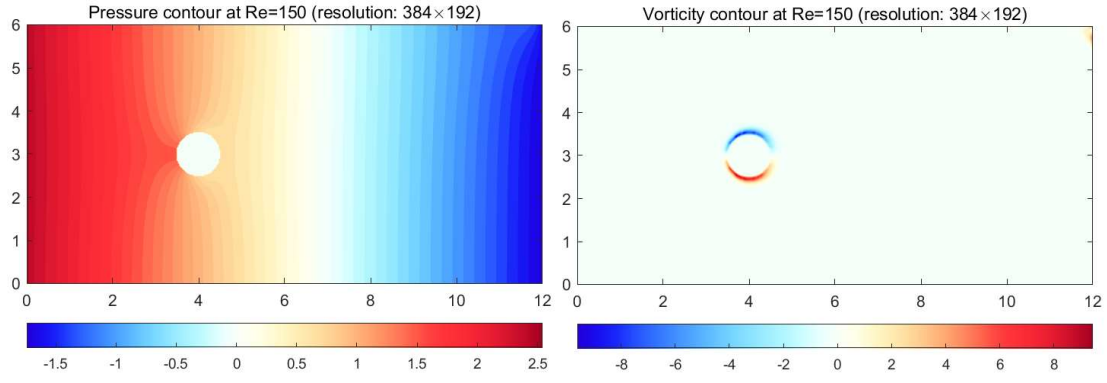


Figure 11 Horizontal and vertical velocity contours at several time intervals at $Re_D = 150$

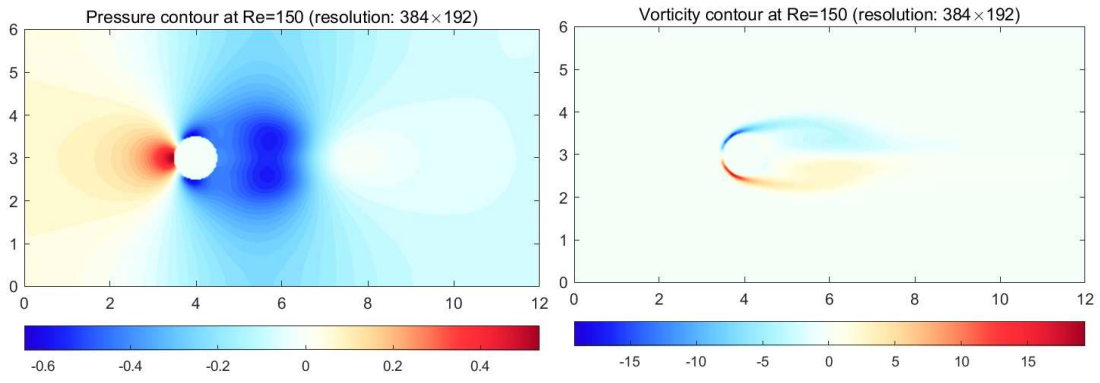
Pressure

Vorticity

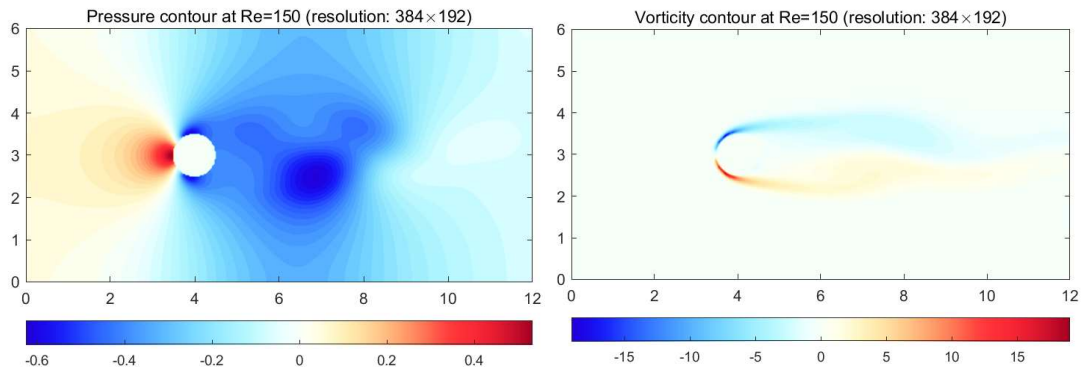
$t=1s$



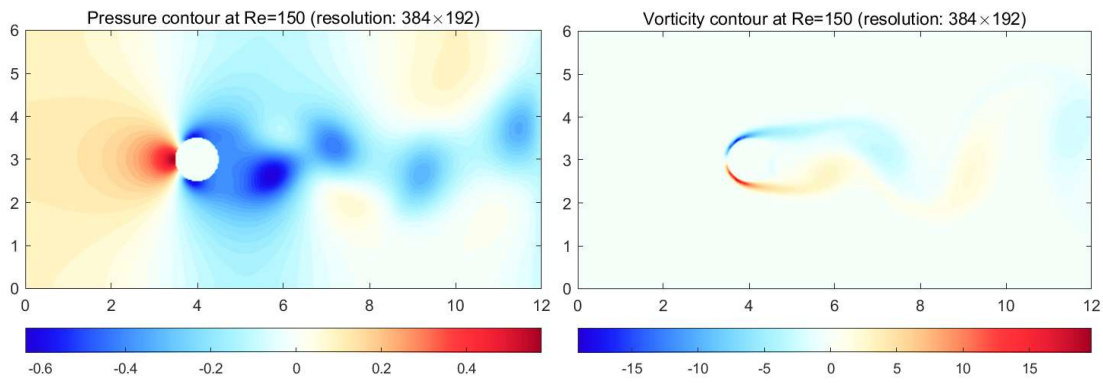
t=10s



t=20s



t=30s



t=50s

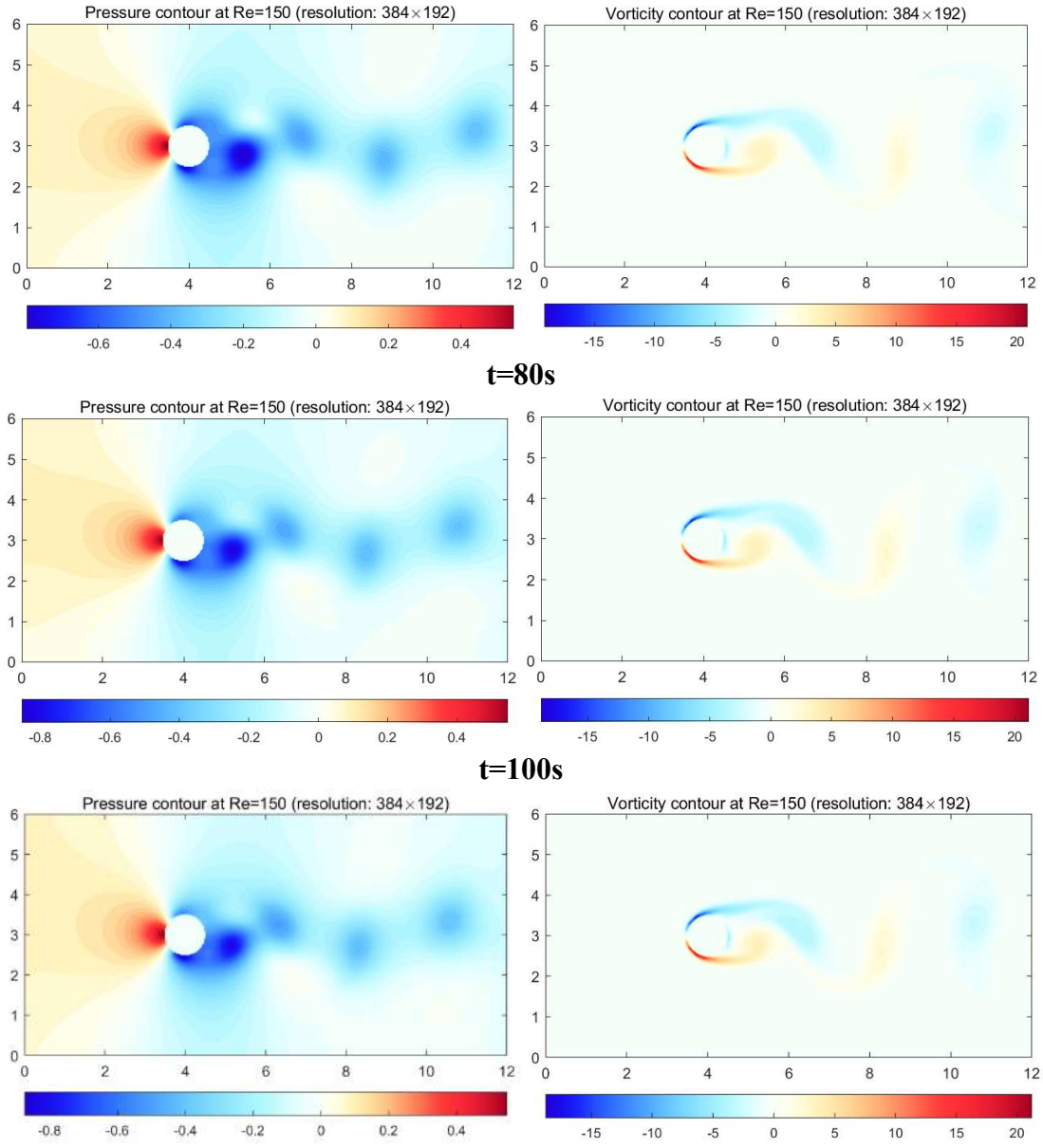


Figure 12 Pressure and vorticity contours at several time intervals at $Re_D = 150$

At the beginning, when $t = 1s$, the flow is well-distributed, with a uniform pressure gradient and no vorticity wake. At $t = 10s$, a pair of symmetric downstream vortices appear behind the cylinder. At this point, there is no vortex-shedding, and the flow is stable. At $t = 20s$, the wake starts to be disturbed, and the flow structure becomes asymmetrical. Meanwhile, the pair of vortices behind the cylinder start to shed, and the pressure field behind the cylinder also begins to fluctuate. After $t = 30s$, the cylinder's wake starts to oscillate, and periodic vortex-shedding occurs behind the cylinder. This phenomenon stabilizes around $t = 50s$, which can be noticed in Figure 13.

3.3.2 Pressure drag coefficient and vortex-shedding frequency

Figure 13 exhibits the pressure drag coefficient variation with time.

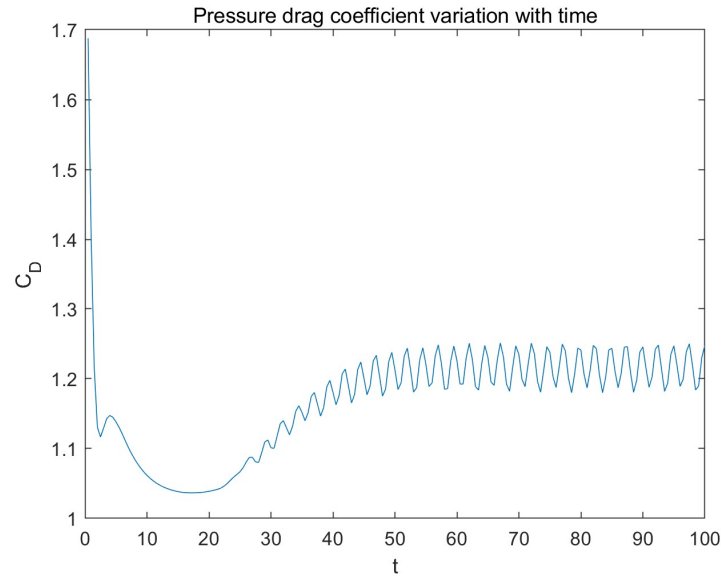


Figure 13 Pressure drag coefficient variation with time at $Re_D = 150$

The coefficient varies sharply when the flow is in developing state. After $t = 50s$, the coefficient fluctuates periodically with the mean value as $C_D = 1.22$. Perform Fourier transformations on the drag coefficient curve between $t = 50s$ and $t = 100s$, the spectrum of drag coefficient is shown in Figure 14.

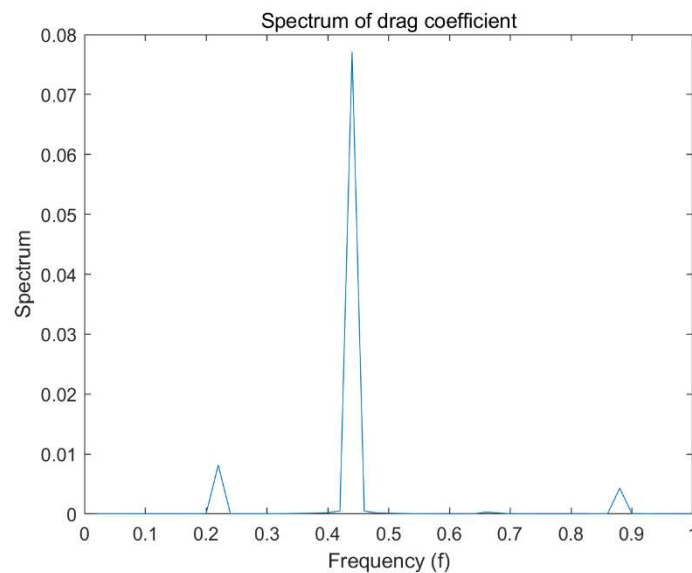


Figure 14 Spectrum of drag coefficient at $Re_D = 150$

Frequency of vortex shedding can be represented by a non-dimensional number, the Strouhal number:

$$St = \frac{fD}{U}$$

where f is the frequency of vortex-shedding, D is the diameter of the cylinder, and U is the freestream velocity.

The frequency of drag coefficient is $f_D = 0.44$. Since the drag coefficient has frequency $2f$ [1], the frequency of the vortex-shedding is $f = 0.22$. Since in this case, $U = 1$, $D = 1$, the Strouhal number is

$$St = \frac{fD}{U} = \frac{0.22 \times 1}{1} = 0.22.$$

Compared with the published results of Qu et al. [2], as shown in Table 1, the drag coefficient and the Strouhal number relatively match well.

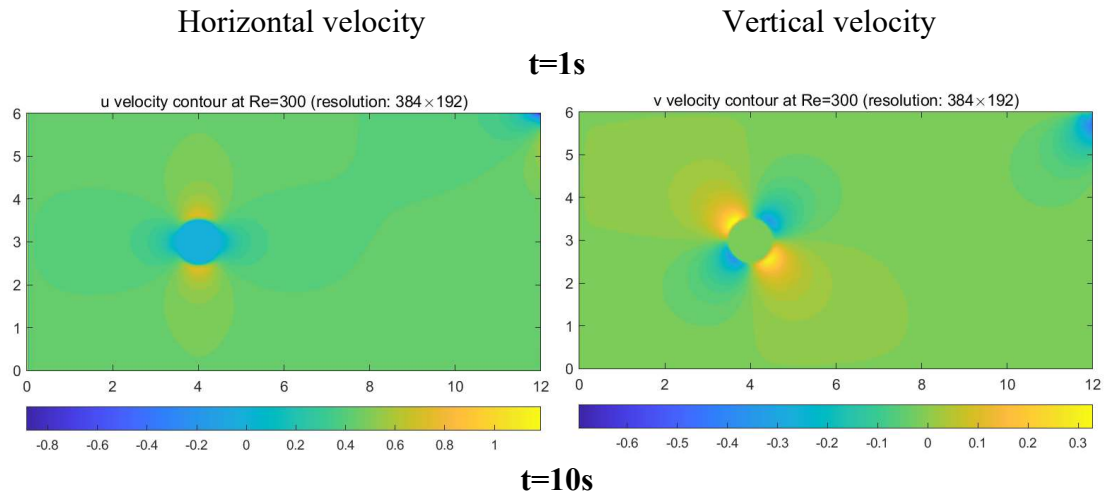
	Pressure drag coefficient C_{Dp}	Strouhal number St
Present	1.22	0.22
Qu et al. [2]	1.024	0.1841

Table 1 Comparison of pressure drag coefficient and Strouhal number with those of Qu et al. [2]

3.4 Flow around a cylinder at different Reynolds number

3.4.1 Flow around a cylinder at $Re_D = 300$

The pressure, velocity, vorticity contours at $Re_D = 300$ are shown in Figure 15 and Figure 16.



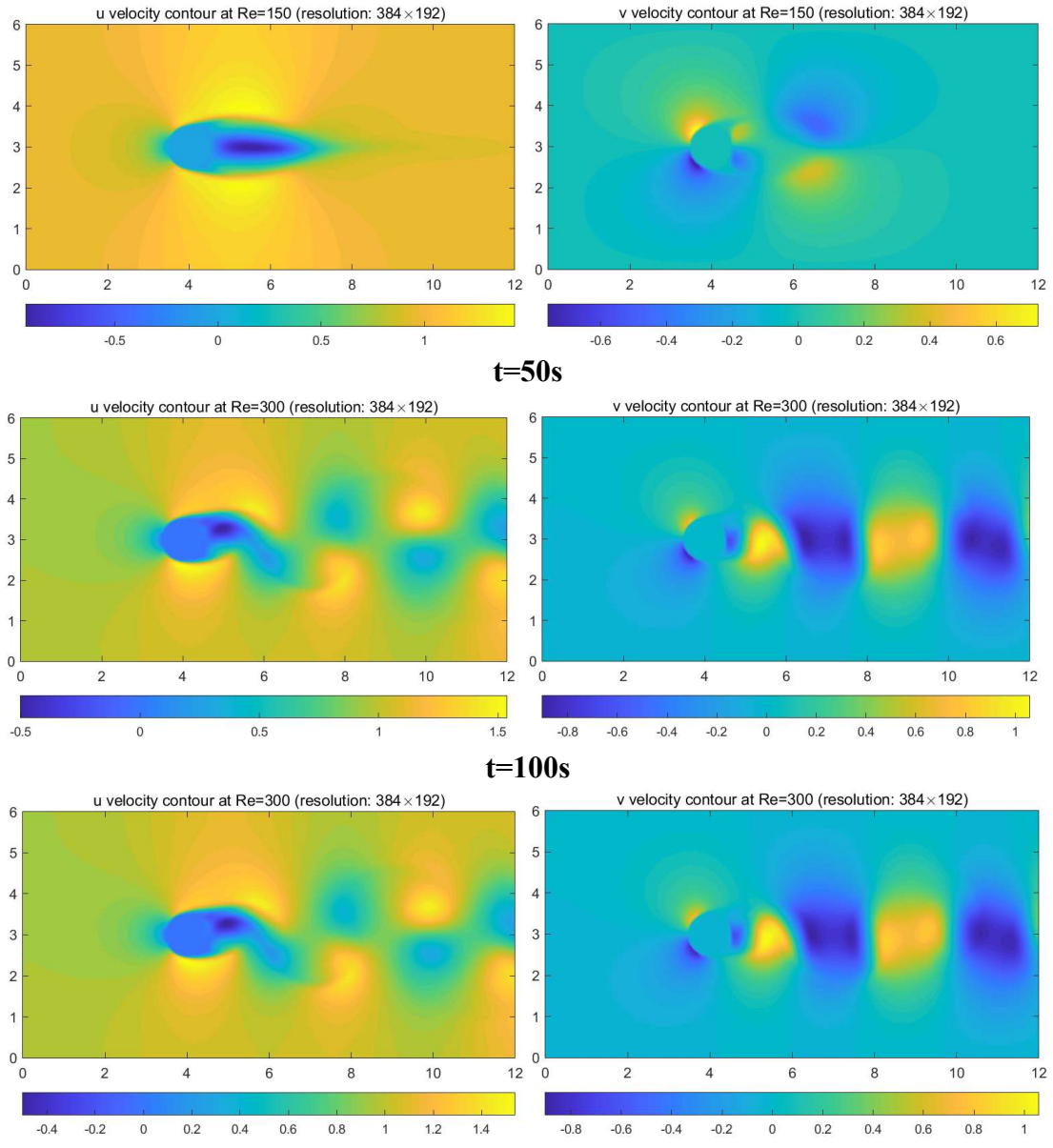
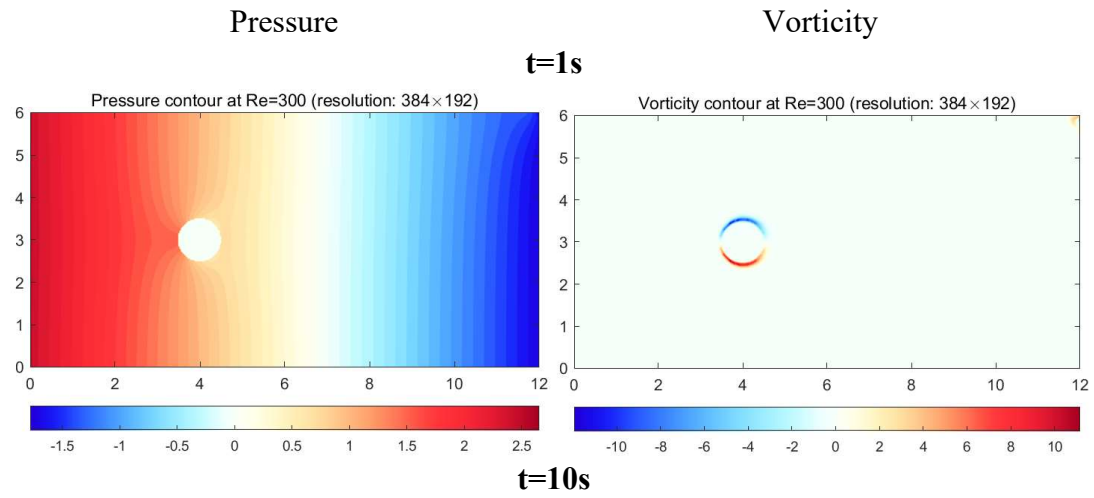


Figure 15 Horizontal and vertical velocity contours at several time intervals at $Re_D = 300$



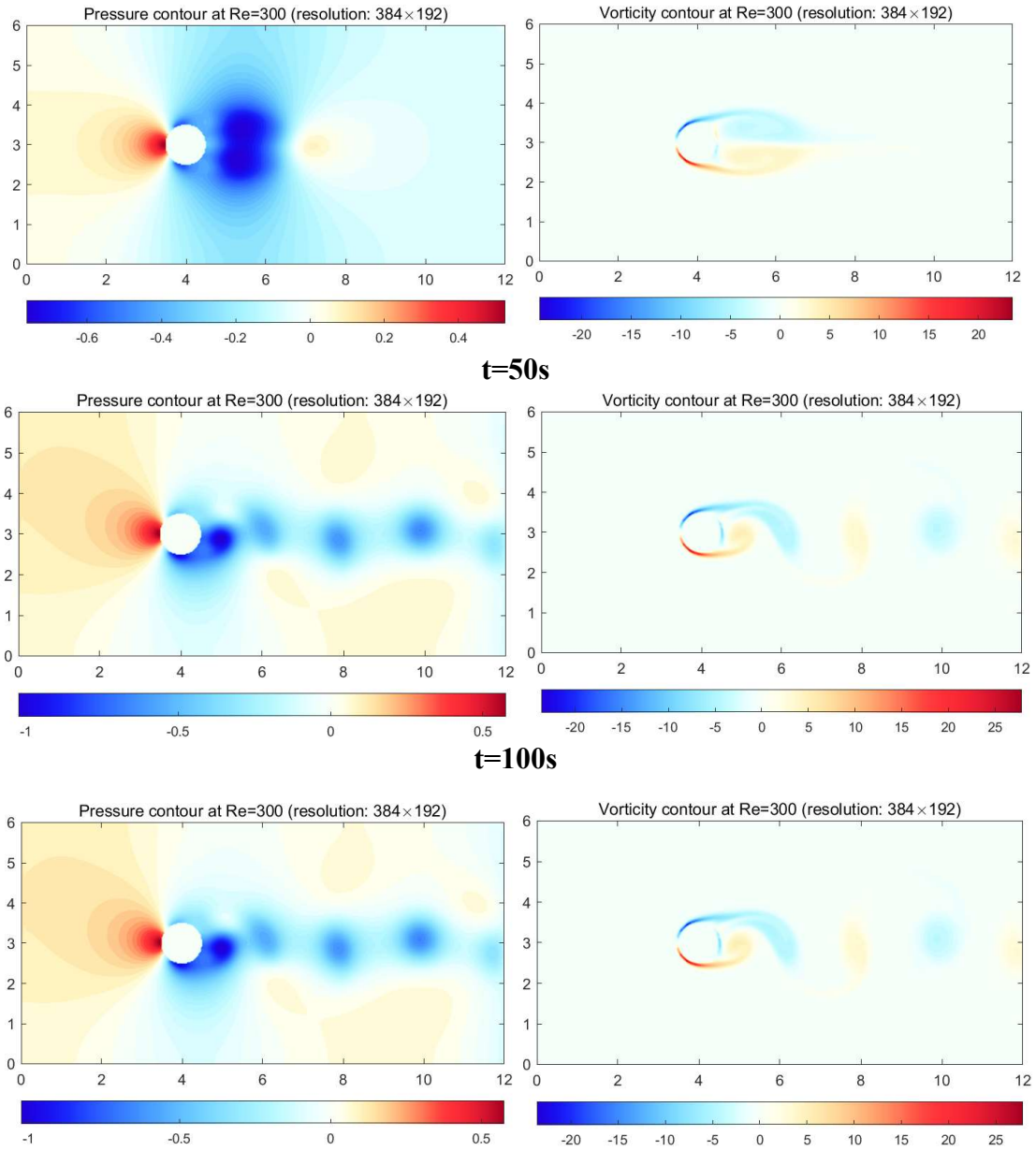


Figure 16 Pressure and vorticity contours at several time intervals at $Re_D = 300$

The developing trend is similar with that of $Re_D = 150$, with slight differences in pressure drag coefficient and vortex-shedding frequency.

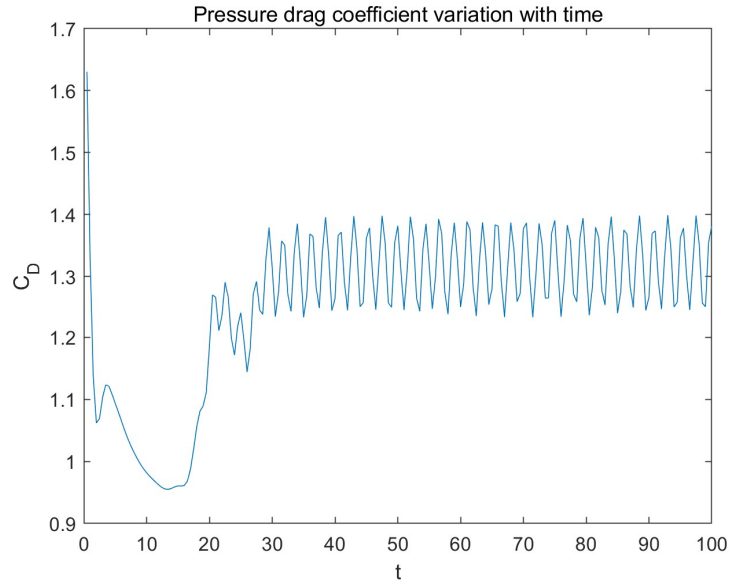


Figure 17 Pressure drag coefficient variation with time at $Re_D = 300$

Figure 17 exhibits the pressure drag coefficient variation with time at $Re_D = 300$. The coefficient also varies sharply when the flow is in developing state. The flow reaches the stable periodic state earlier, which only spends 30s, comparing with 50s for $Re_D = 150$ case. After $t = 30s$, the coefficient fluctuates periodically with the mean value as $C_D = 1.32$. Comparing with the case at $Re_D = 150$, the drag coefficient is larger. Perform Fourier transformations on the drag coefficient curve between $t = 50s$ and $t = 100s$, the spectrum of drag coefficient is shown in Figure 18.

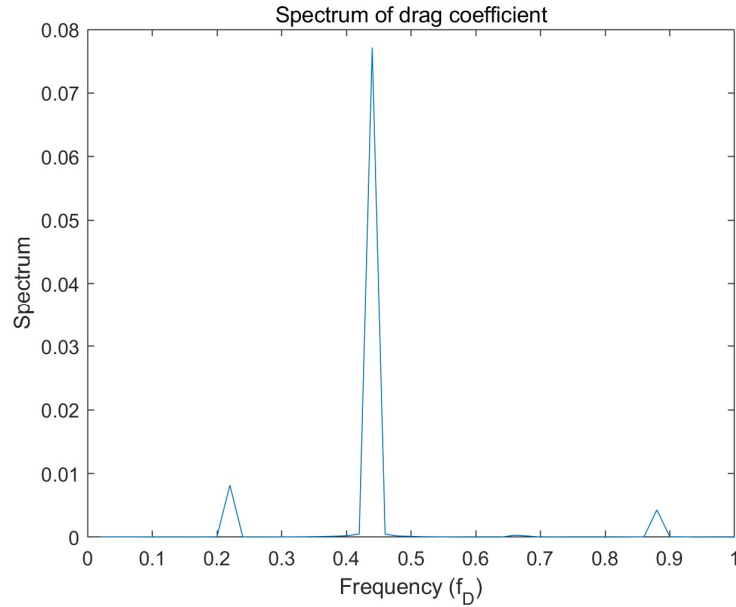


Figure 18 Spectrum of pressure drag coefficient at $Re_D = 300$

The frequency of drag coefficient is $f_D = 0.44$, the same as that of $Re_D = 150$. Therefore, the Strouhal number is $St = 0.22$. Compared with the published results of

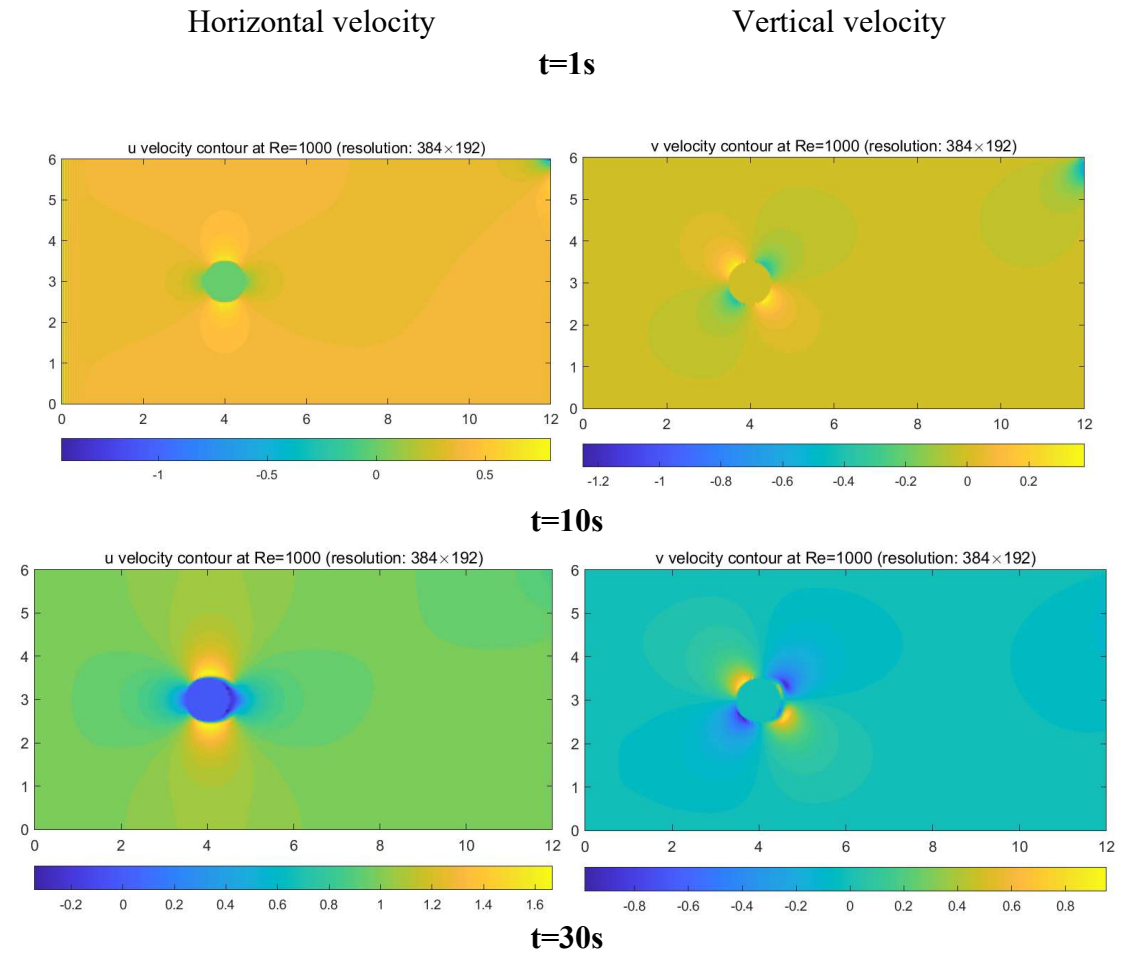
Franke et al. [3], as shown in Table 2, the drag coefficient and the Strouhal number relatively match well.

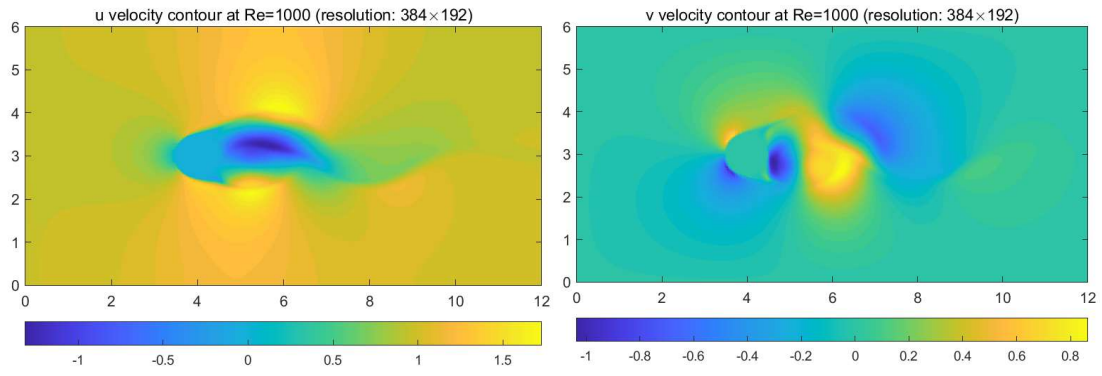
	Pressure drag coefficient C_{Dp}	Strouhal number St
$Re_D = 150$	1.22	0.22
$Re_D = 300$	1.32	0.22
Franke et al. [3] $Re_D = 300$	1.11	0.205

Table 2 Comparison of C_D and St with those of Franke et al. [3] at $Re_D = 300$

3.4.2 Flow around a cylinder at $Re_D = 1000$

The pressure, velocity, vorticity contours at $Re_D = 1000$ are shown in Figure 19 and Figure 20.





$t=100s$

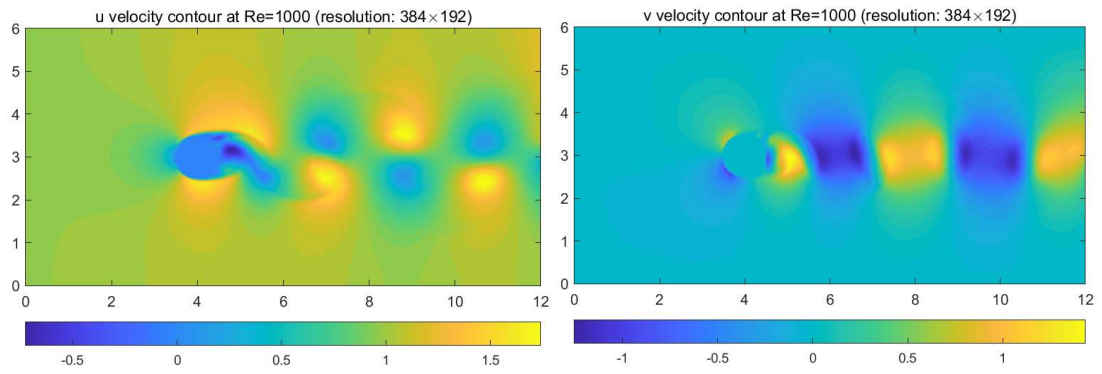
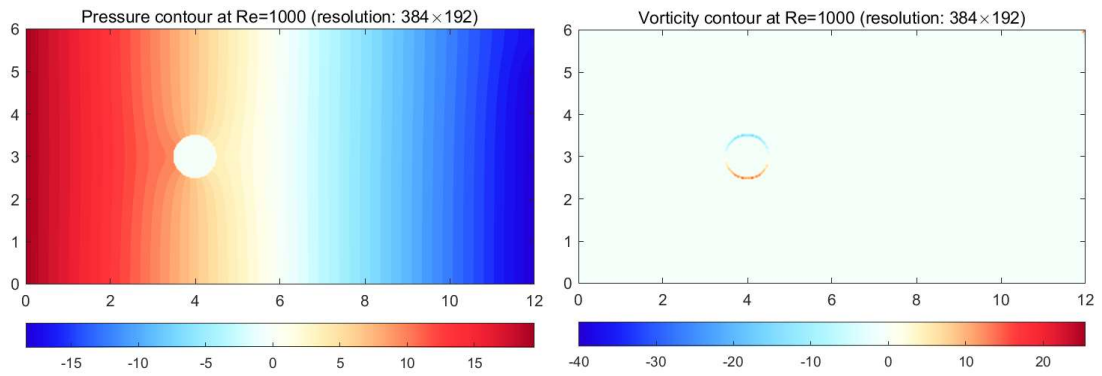


Figure 19 Horizontal and vertical velocity contours at several time intervals at $Re_D = 1000$

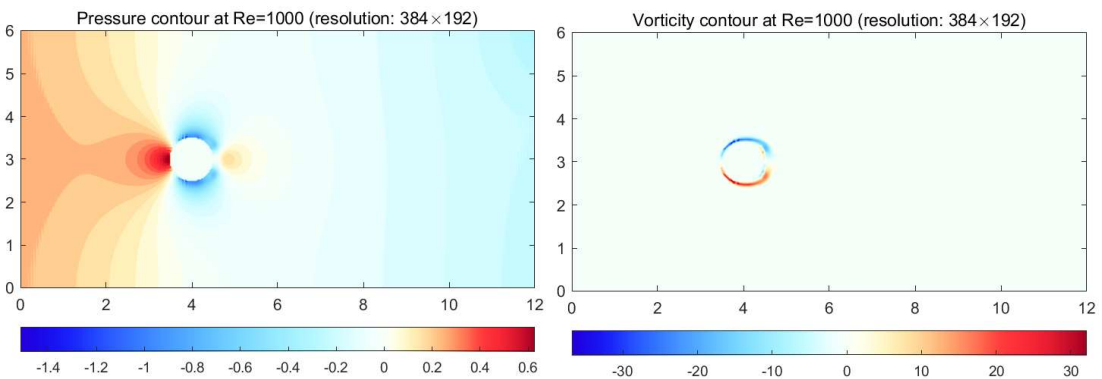
Pressure

Vorticity

$t=1s$



$t=10s$



$t=30s$

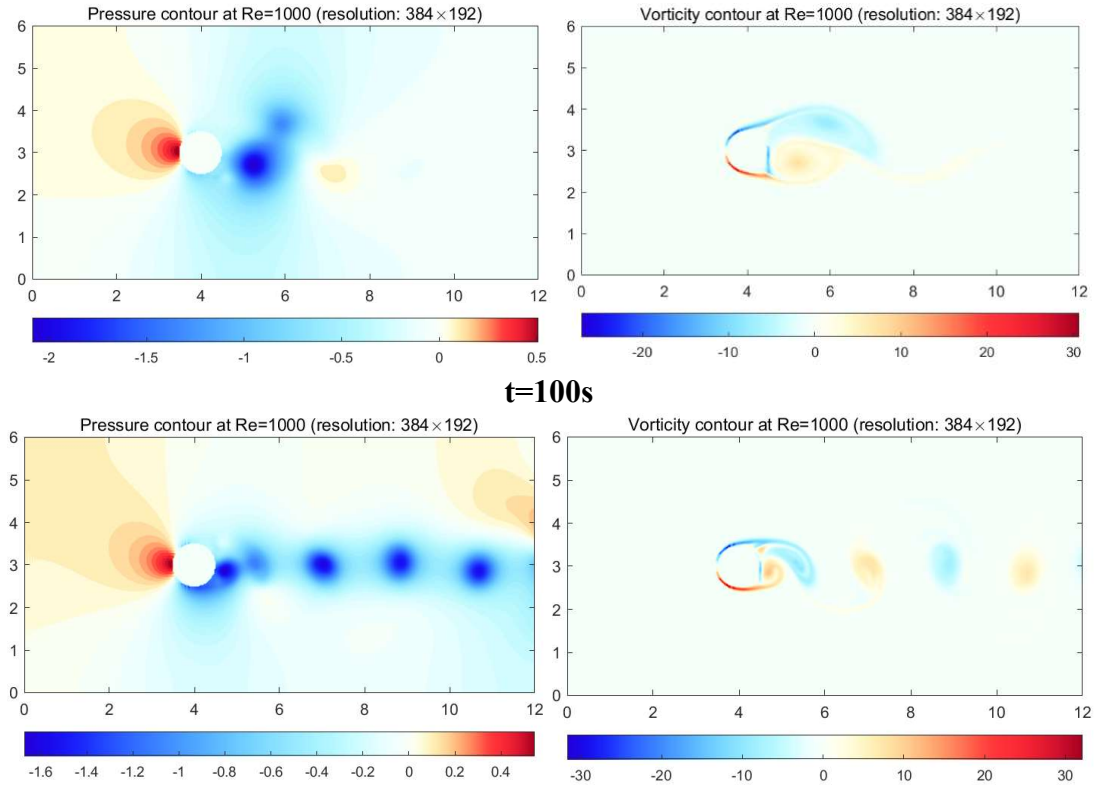


Figure 20 Pressure and vorticity contours at several time intervals at $Re_D = 1000$

Compared to the cases with $Re_D = 150$ and $Re_D = 300$, as observed from the contours, this case exhibits greater disturbance. The vortices align behind the cylinder, and compared to the other two cases, the vortices in this case are more neatly arranged. Observed from the pressure contours, the pressure extreme points are also aligned organized.

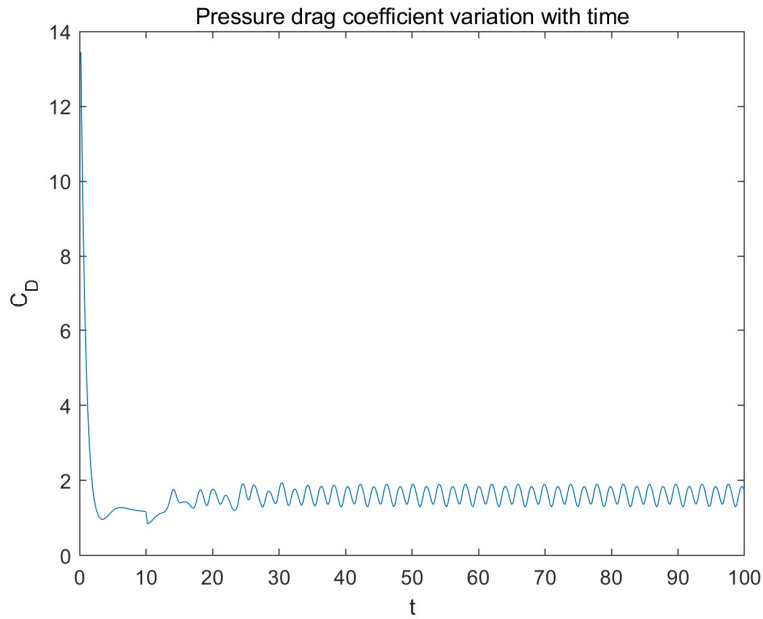


Figure 21 Pressure drag coefficient variation with time at $Re_D = 1000$

Figure 21 exhibits the pressure drag coefficient variation with time at $Re_D = 1000$. The coefficient also varies sharply when the flow is in developing state. The flow reaches the stable periodic state even earlier, which only spends 20s, comparing with 50s for $Re_D = 150$ and 30s for $Re_D = 300$. After $t = 20s$, the coefficient fluctuates periodically with the mean value as $C_D = 1.58$. Comparing with the case at $Re_D = 150$ and $Re_D = 300$, the drag coefficient is even larger. Perform Fourier transformations on the drag coefficient curve between $t = 50s$ and $t = 100s$, the spectrum of drag coefficient is shown in Figure 22.

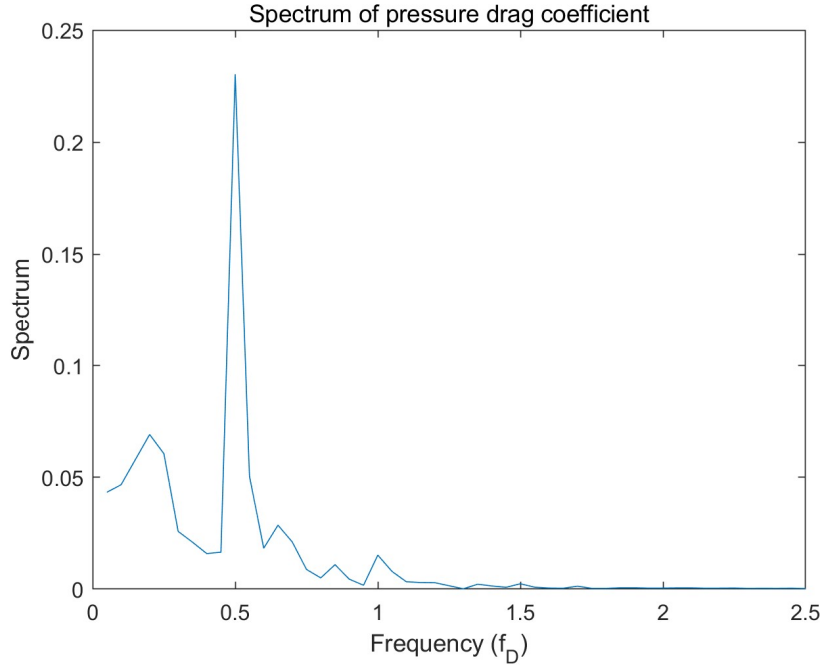


Figure 22 Spectrum of pressure drag coefficient at $Re_D = 1000$

The frequency of drag coefficient is $f_D = 0.5$, slightly larger than that of $Re_D = 150$. Therefore, the Strouhal number is $St = 0.25$. Compared with the published results of Qu et al. [2], as shown in Table 3, the drag coefficient and the Strouhal number relatively match well.

	Pressure drag coefficient C_{Dp}	Strouhal number St
$Re_D = 150$	1.22	0.22
$Re_D = 1000$	1.58	0.25
Qu et al. [2] $Re_D = 1000$	1.34	0.236

Table 3 Comparison of C_D and St with those of Qu et al. [2] at $Re_D = 1000$

3.5 Flow around a cylinder at $Re_D = 150$ at different grid resolutions

3.5.1 Flow around a cylinder at $dx = 1/16$

The pressure, velocity, vorticity contours at $dx = 1/16$ are shown in Figure 23 and

Figure 24.

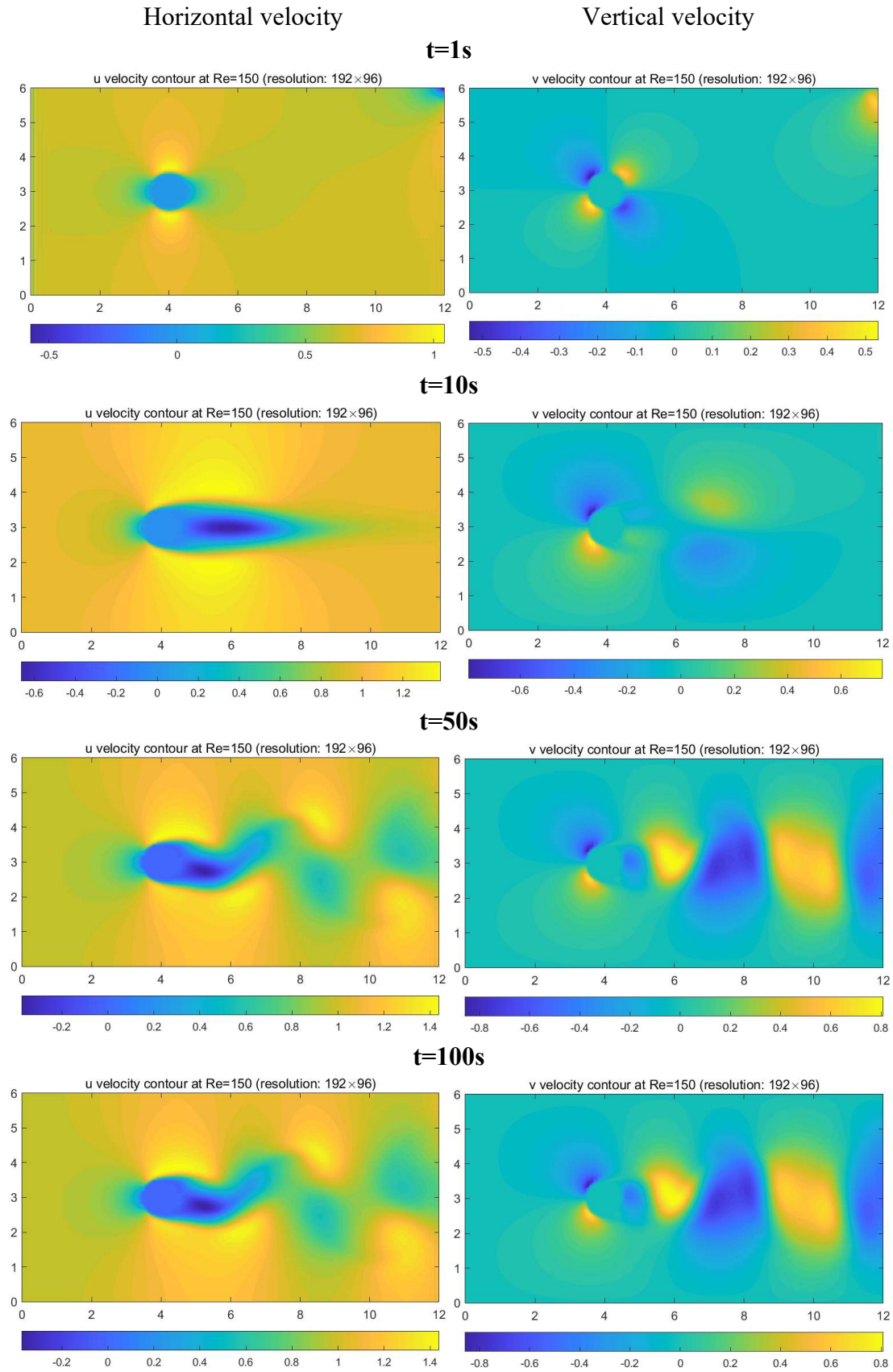


Figure 23 Horizontal and vertical velocity contours at several time intervals at $dx = 1/16$

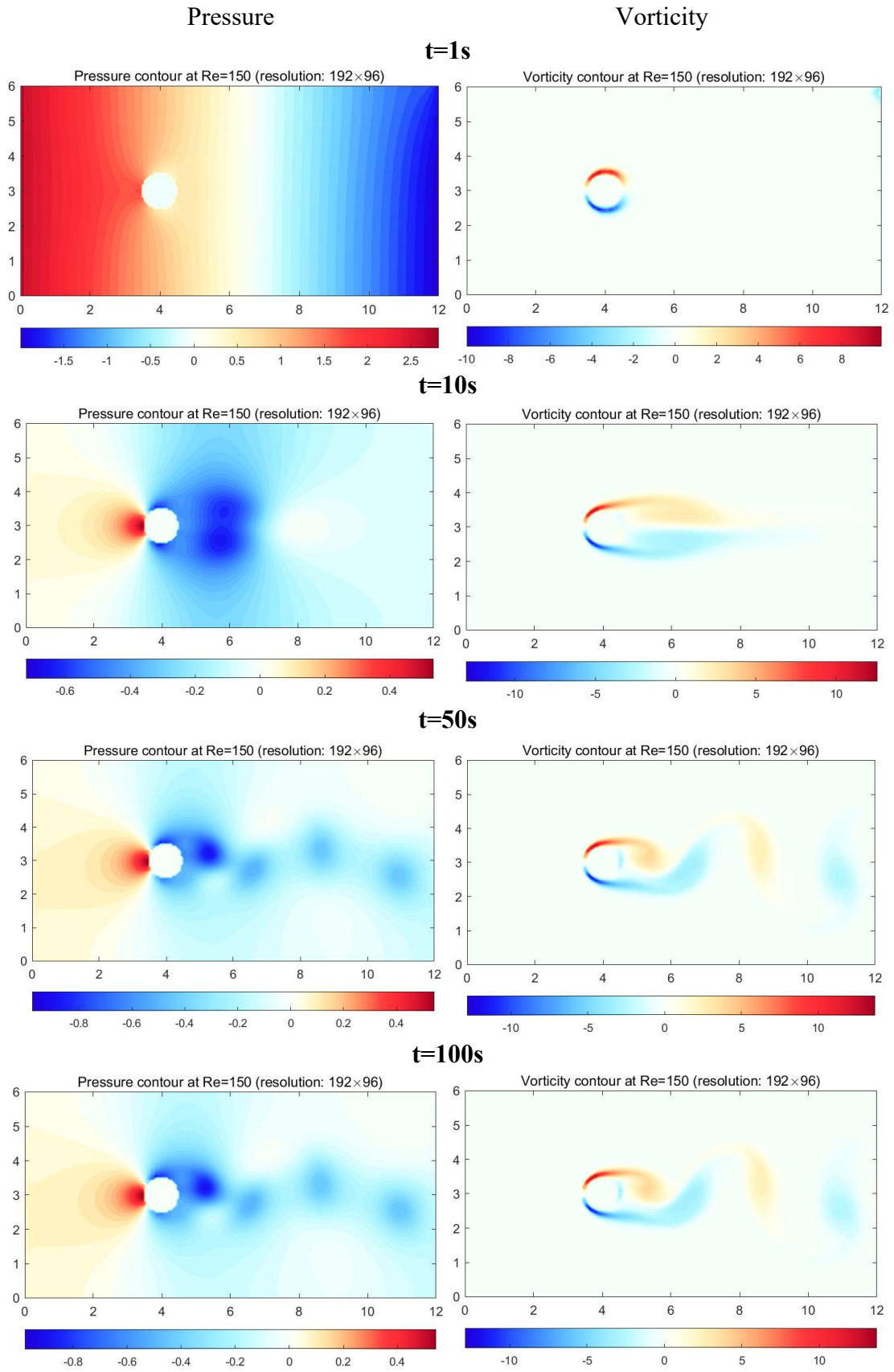


Figure 24 Pressure and vorticity contours at several time intervals at $dx = 1/16$

The developing trend is similar with that at $dx = 1/32$, with slight differences in pressure drag coefficient and vortex-shedding frequency.

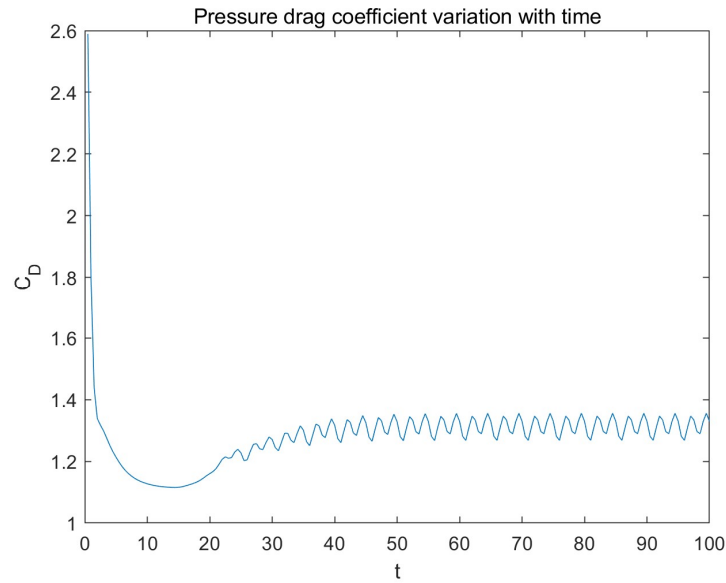


Figure 25 Pressure drag coefficient variation with time at $dx = 1/16$

Figure 25 exhibits the pressure drag coefficient variation with time at $dx = 1/16$. The coefficient also varies sharply when the flow is in developing state. The flow reaches the stable periodic state at 40s. After $t = 40s$, the coefficient fluctuates periodically with the mean value as $C_D = 1.31$. Comparing with the case at $dx = 1/32$, the drag coefficient is larger. Perform Fourier transformations on the drag coefficient curve between $t = 50s$ and $t = 100s$, the spectrum of drag coefficient is shown in Figure 26.

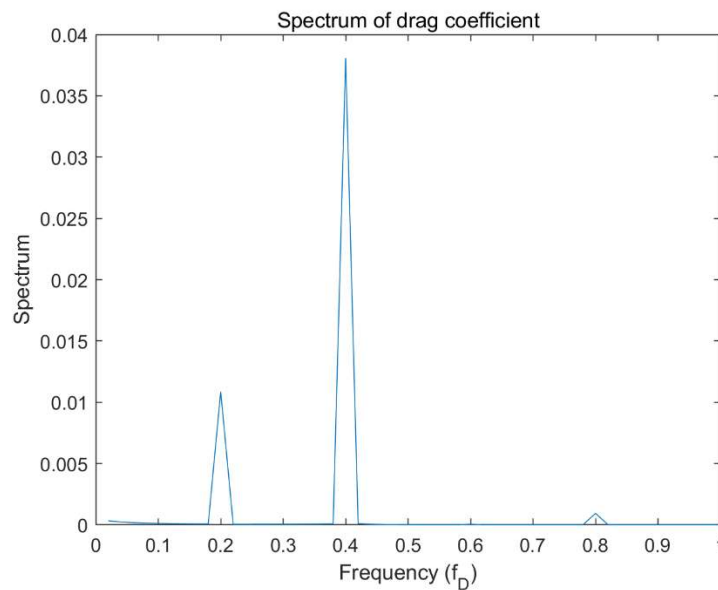


Figure 26 Spectrum of pressure drag coefficient at $dx = 1/16$

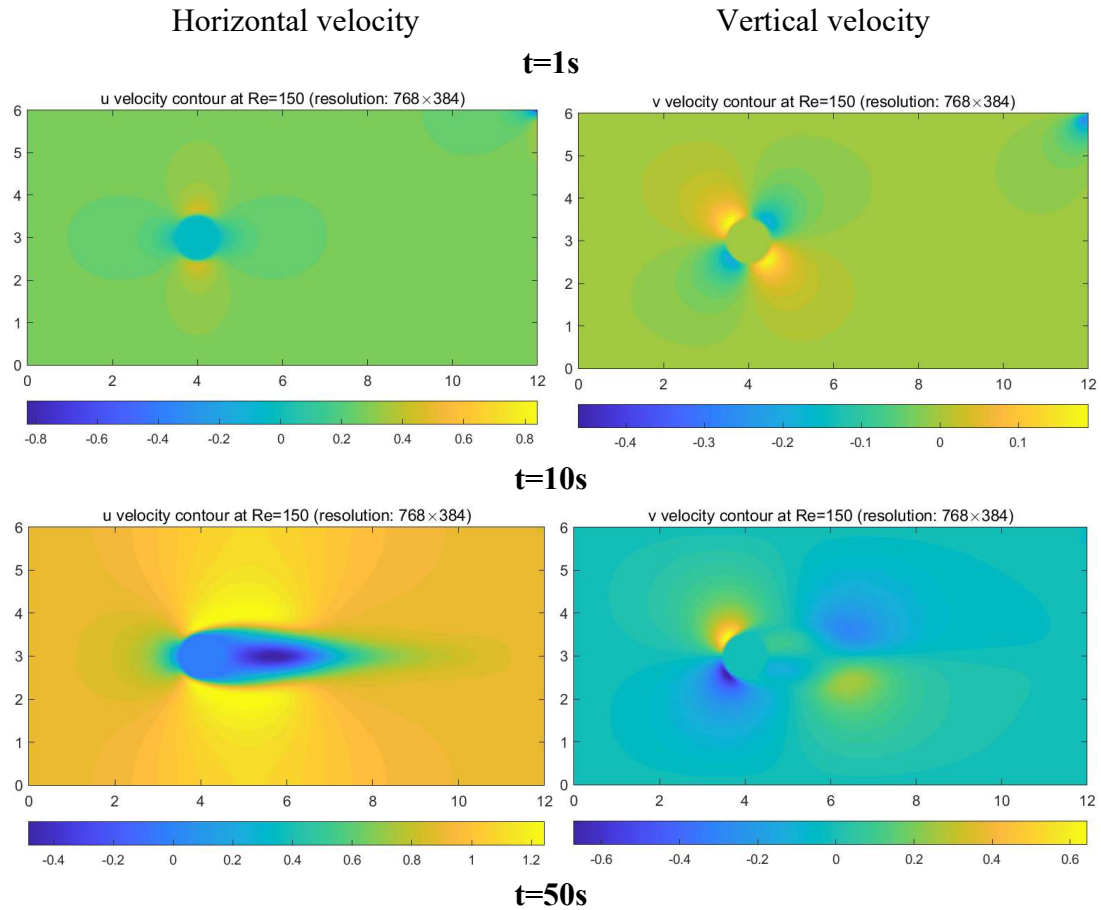
The frequency of drag coefficient is $f_D = 0.4$, slightly smaller than that of $dx = 1/32$. Therefore, the Strouhal number is $St = 0.2$. Compared with the published results of Qu et al. [2], as shown in Table 4, the drag coefficient and the Strouhal number do not match as well as that of $dx = 1/32$.

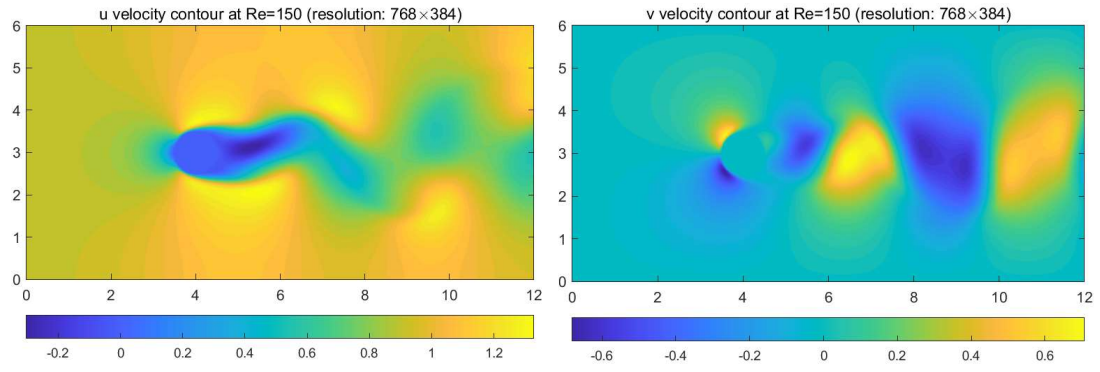
	Pressure drag coefficient C_{Dp}	Strouhal number St
$dx = 1/32$	1.22	0.22
$dx = 1/16$	1.31	0.2
Qu et al. [2]	1.024	0.1841

Table 4 Comparison of C_D and St with those of Qu et al. [2] at $dx = 1/16$ at $Re_D = 150$

3.5.2 Flow around a cylinder at $dx = 1/64$

The pressure, velocity, vorticity contours at $dx = 1/64$ are shown in Figure 27 and Figure 28.





$t=100s$

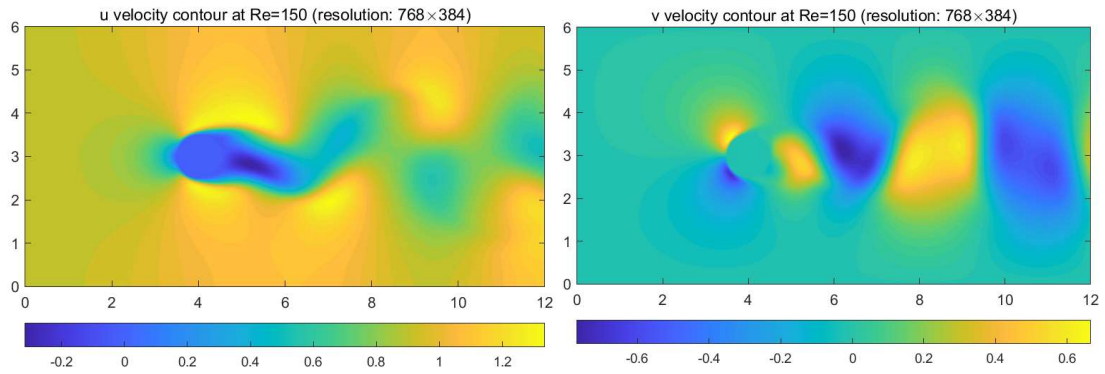
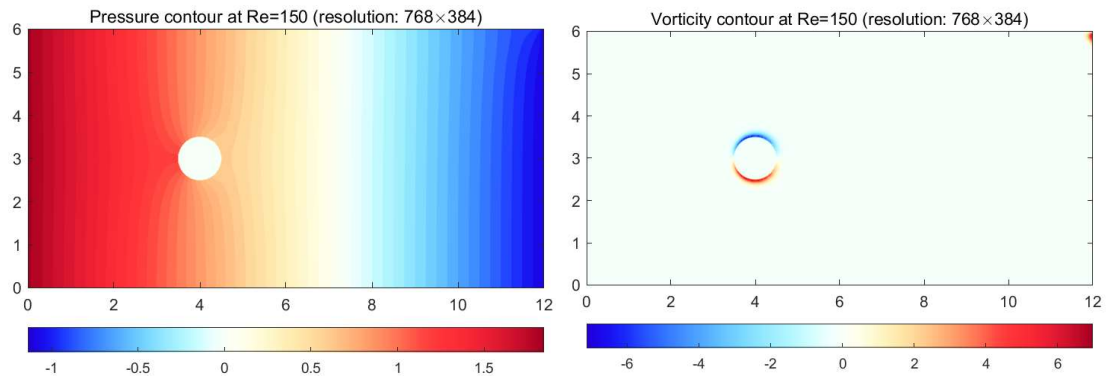


Figure 27 Horizontal and vertical velocity contours at several time intervals at $dx = 1/64$

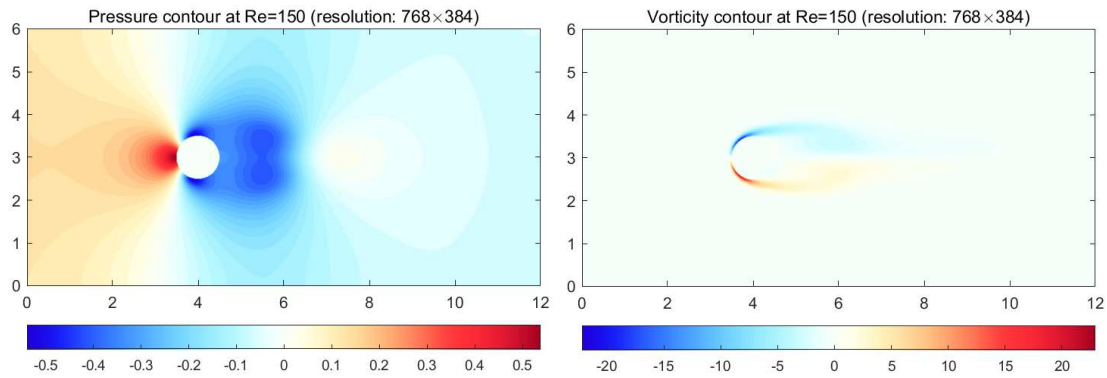
Pressure

Vorticity

$t=1s$



$t=10s$



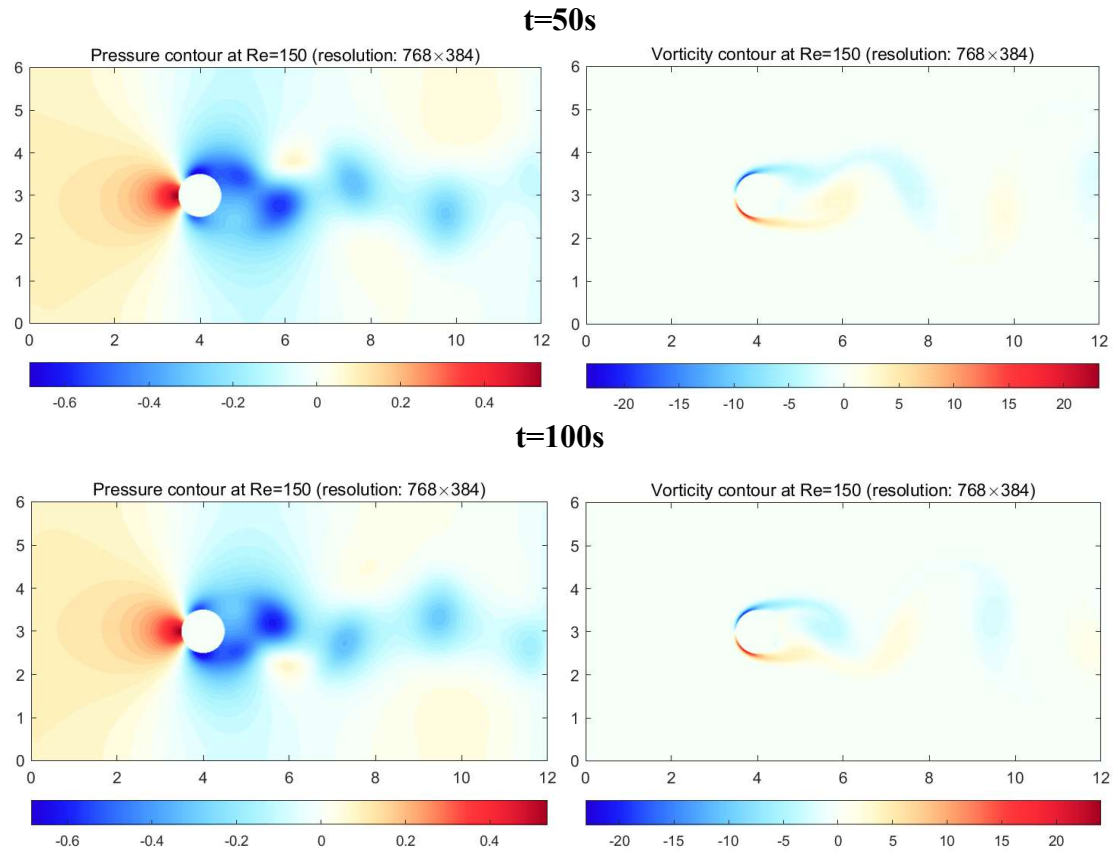


Figure 28 Pressure and vorticity contours at several time intervals at $dx = 1/64$

The developing trend is also similar with that at $dx = 1/32$, with slight differences in pressure drag coefficient and vortex-shedding frequency.

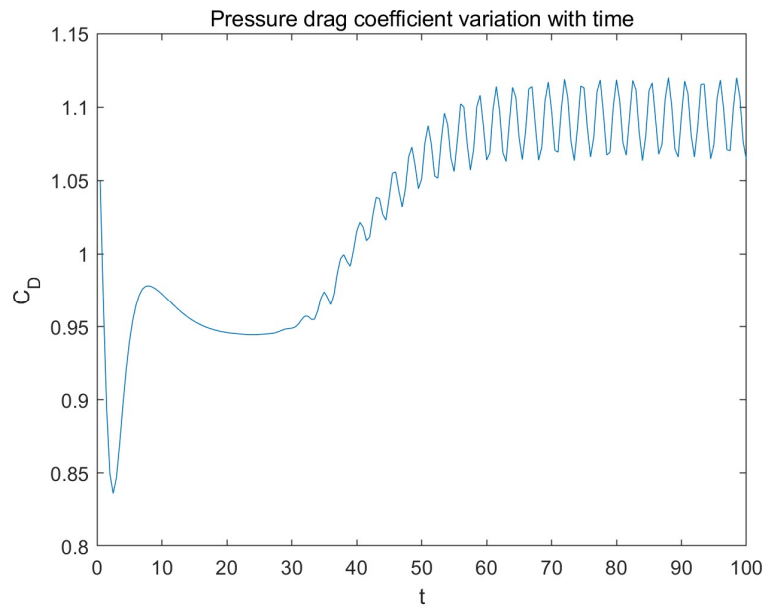


Figure 29 Pressure drag coefficient variation with time at $dx = 1/64$

Figure 29 exhibits the pressure drag coefficient variation with time at $dx = 1/64$. The coefficient also varies sharply when the flow is in developing state. The flow reaches the stable periodic state at 60s. After $t = 60s$, the coefficient fluctuates periodically with the mean value as $C_D = 1.08$. Comparing with the case at $dx = 1/32$, the drag coefficient is smaller. Perform Fourier transformations on the drag coefficient curve between $t = 60s$ and $t = 100s$, the spectrum of drag coefficient is shown in Figure 30.

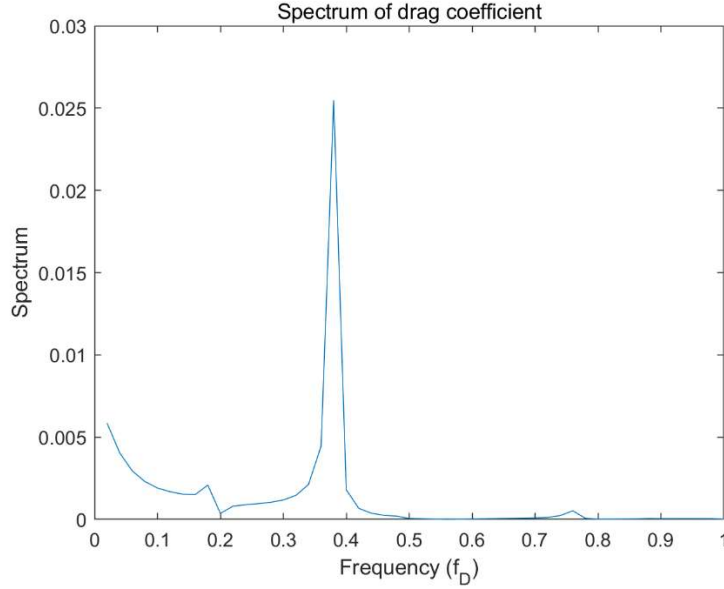


Figure 30 Spectrum of pressure drag coefficient at $dx = 1/64$

The frequency of drag coefficient is $f_D = 0.38$, slightly smaller than that of $dx = 1/32$. Therefore, the Strouhal number is $St = 0.19$. Compared with the published results of Qu et al. [2], as shown in Table 5, the drag coefficient and the Strouhal number match better.

	Pressure drag coefficient C_{Dp}	Strouhal number St
$dx = 1/32$	1.22	0.22
$dx = 1/64$	1.08	0.19
Qu et al. [2]	1.024	0.1841

Table 5 Comparison of C_D and St with those of Qu et al. [2] at $dx = 1/64$ at $Re_D = 150$

3.6 Flow around a cylinder at $Re_D = 150$ with different domain sizes

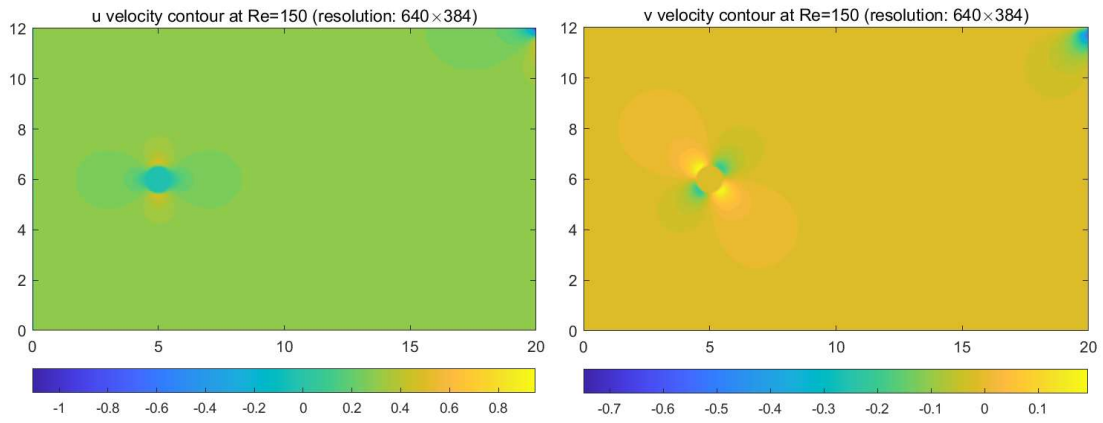
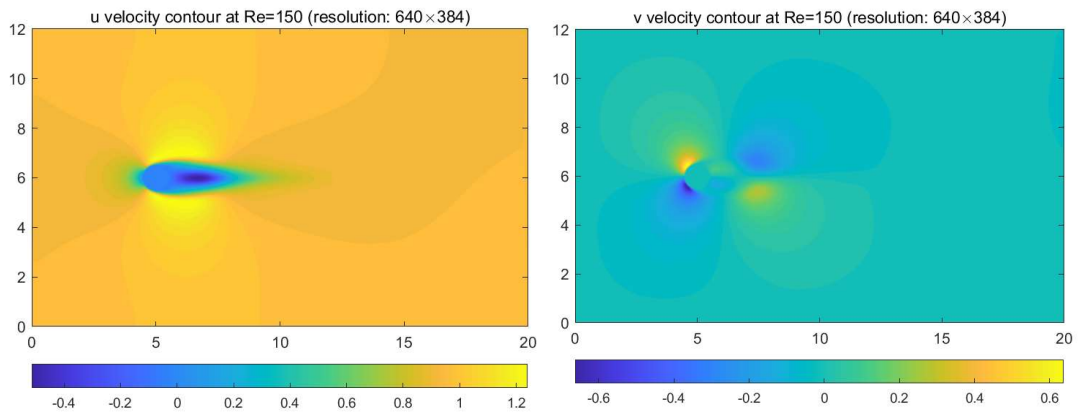
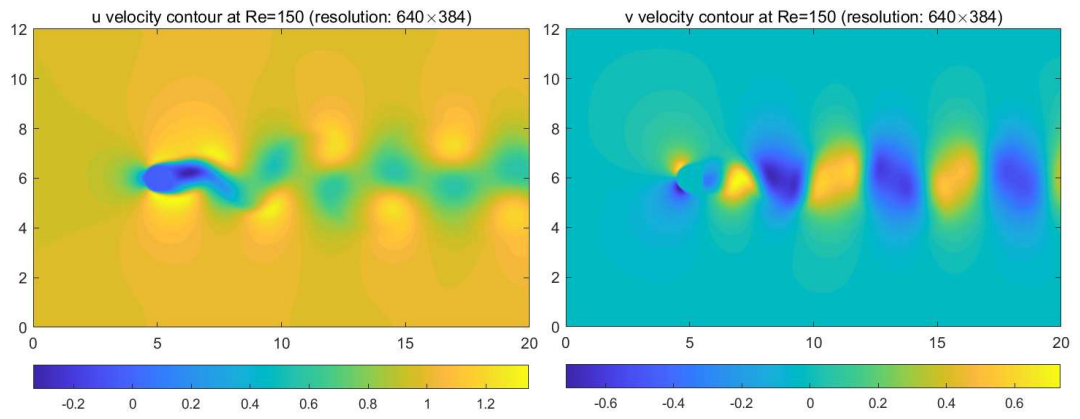
To investigate the effects of domain size, we enlarged and shrink the original domain size respectively, while keeping the grid resolution unchanged. The new larger domain size is $L_y = 12, L_x = 20; N_y = 384, N_x = 640$. The new smaller domain size is $L_y = 3, L_x = 12; N_y = 96, N_x = 384$.

3.6.1 Flow around a cylinder in large domain

The pressure, velocity, vorticity contours in large domain are shown in Figure 31 and Figure 32.

Horizontal velocity

Vertical velocity

t=1s**t=10s****t=50s****t=100s**

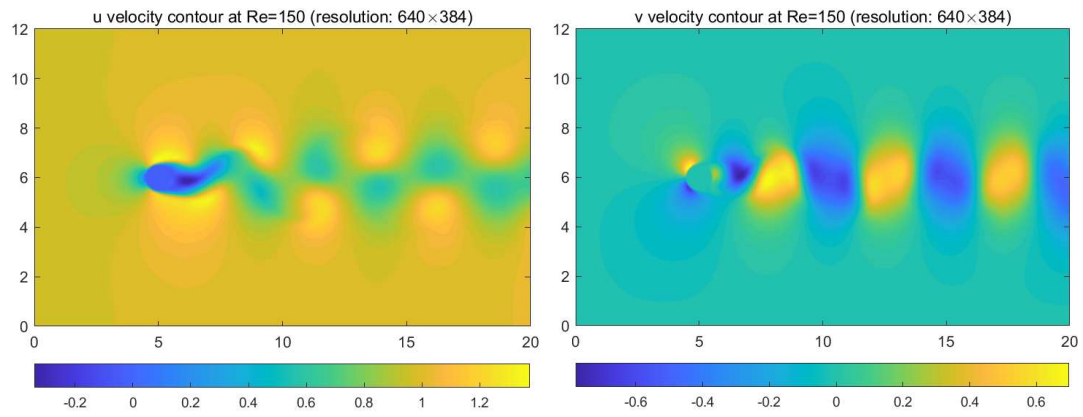
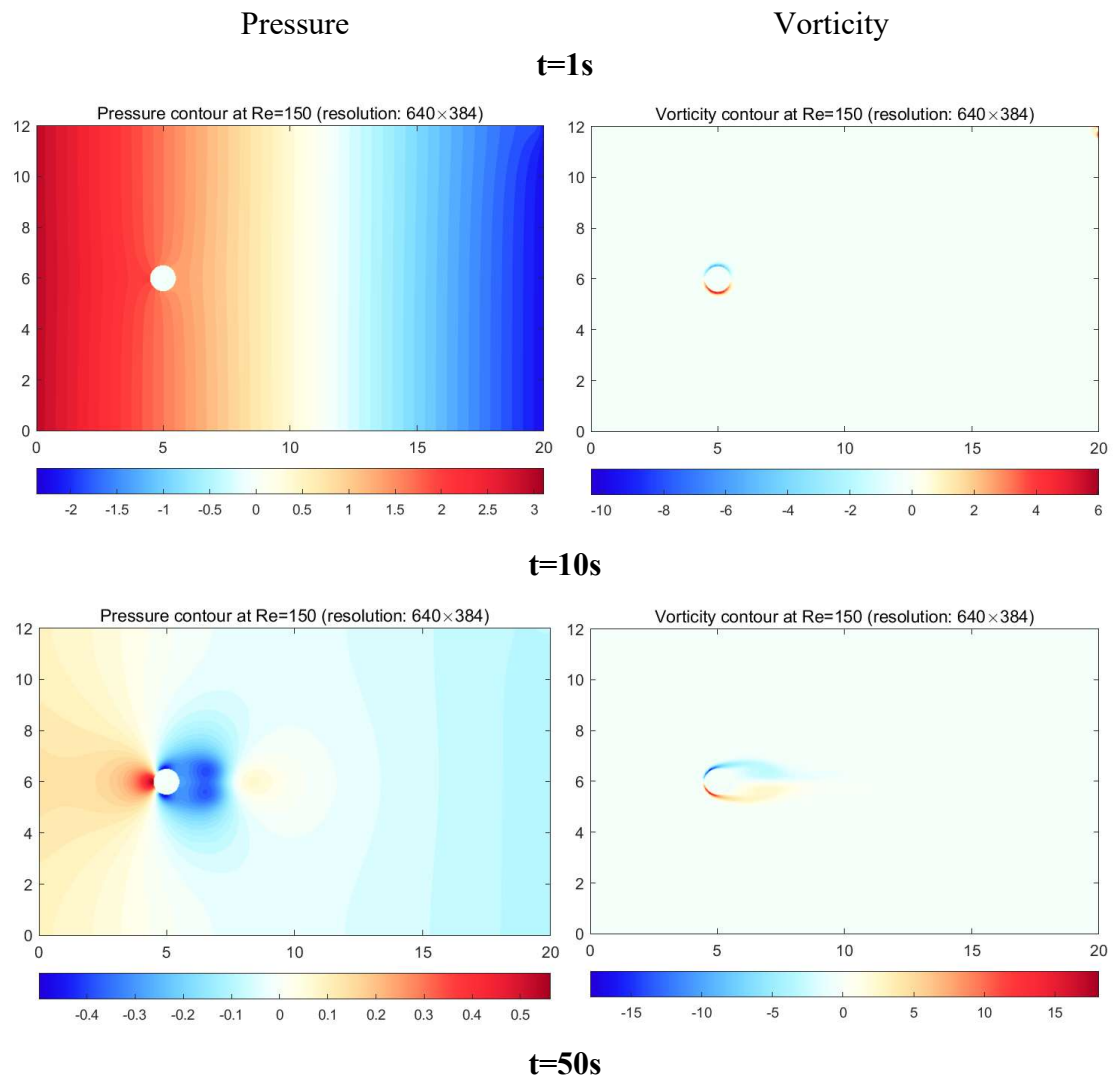


Figure 31 Horizontal and vertical velocity contours at several time intervals in large domain



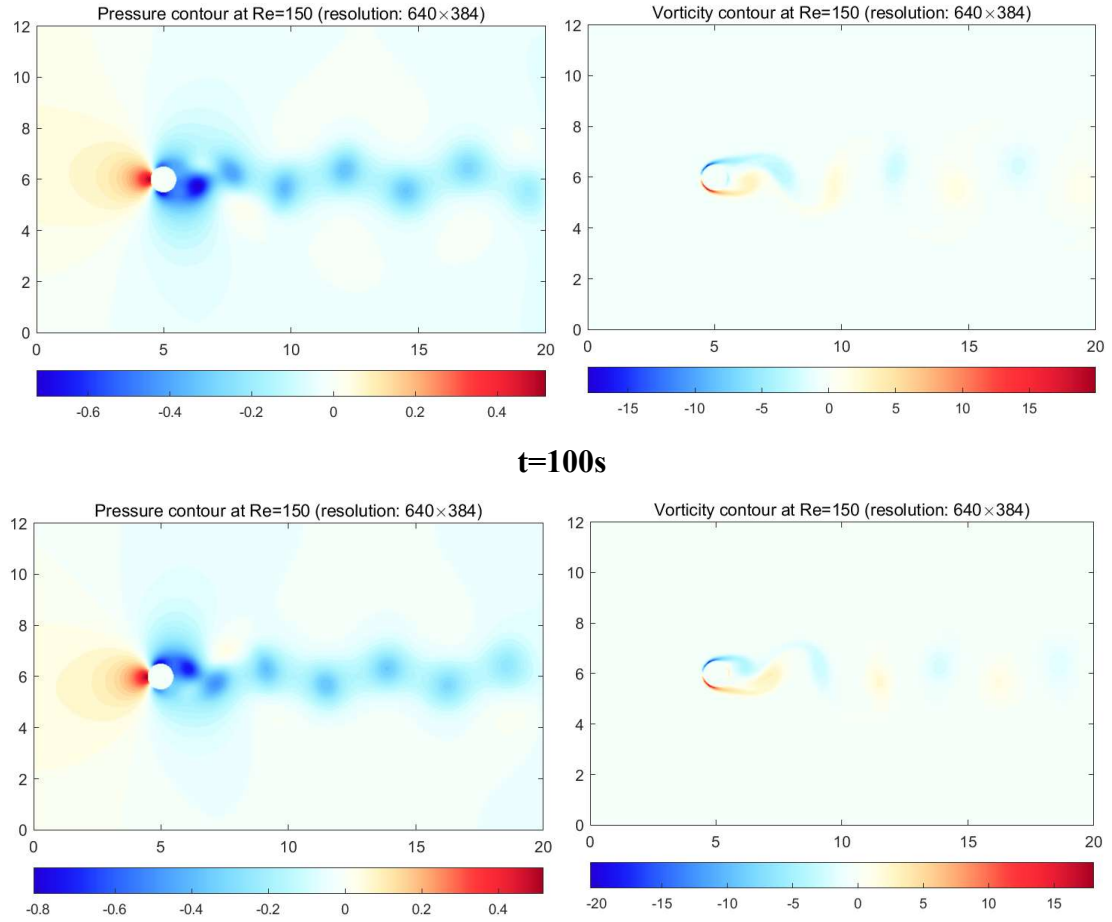


Figure 32 Pressure and vorticity contours at several time intervals in large domain

The developing trend is also similar with that from original domain, with slight differences in pressure drag coefficient and vortex-shedding frequency.

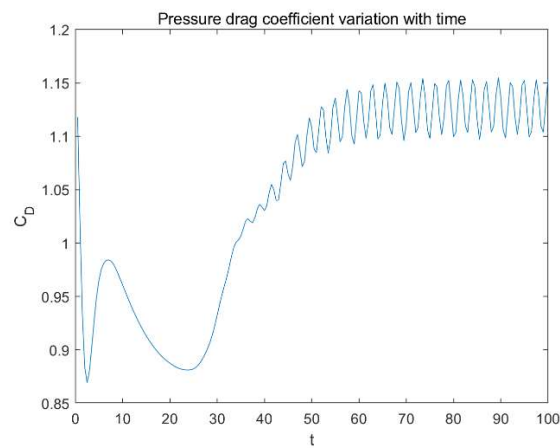


Figure 33 Pressure drag coefficient variation with time in large domain

Figure 33 exhibits the pressure drag coefficient variation with time in large domain. The coefficient also varies sharply when the flow is in developing state. The flow reaches the stable periodic state at 60s. After $t = 60s$, the coefficient fluctuates

periodically with the mean value as $C_D = 1.12$. Comparing with the case from the original domain, the drag coefficient is smaller. Perform Fourier transformations on the drag coefficient curve between $t = 60s$ and $t = 100s$, the spectrum of drag coefficient is shown in Figure 34.

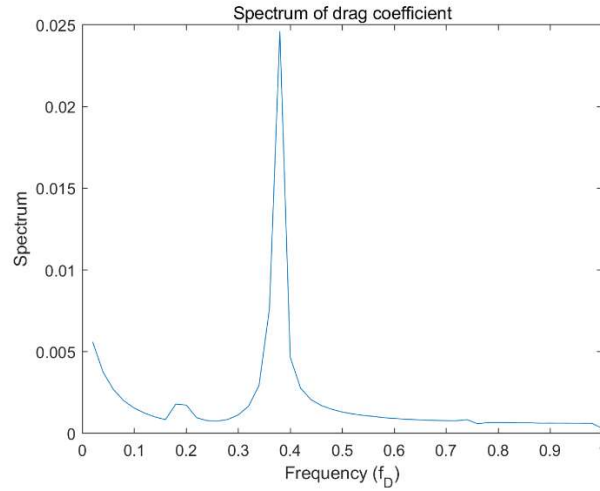


Figure 34 Spectrum of pressure drag coefficient in large domain

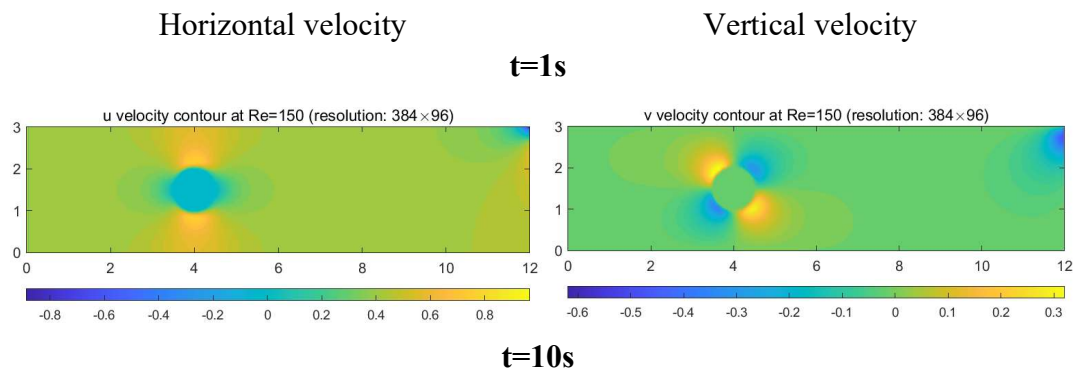
The frequency of drag coefficient is $f_D = 0.38$, slightly smaller than that from the original domain. Therefore, the Strouhal number is $St = 0.19$. Compared with the published results of Qu et al. [2], as shown in Table 6, the drag coefficient and the Strouhal number match better than that from original domain.

	Pressure drag coefficient C_{Dp}	Strouhal number St
Original domain	1.22	0.22
Large domain	1.12	0.19
Qu et al. [2]	1.024	0.1841

Table 6 Comparison of C_D and St with those of Qu et al. [2] in large domain

3.6.2 Flow around a cylinder in small domain

The pressure, velocity, vorticity contours in small domain are shown in Figure 35 and Figure 36.



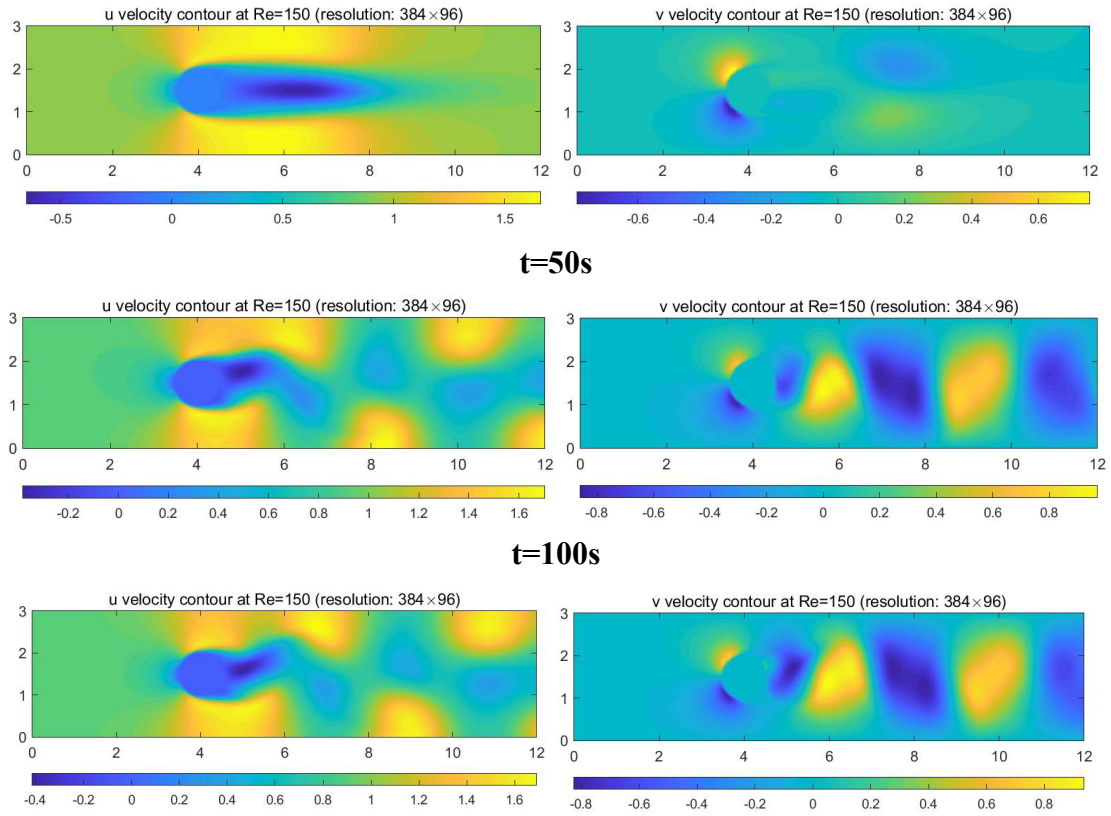
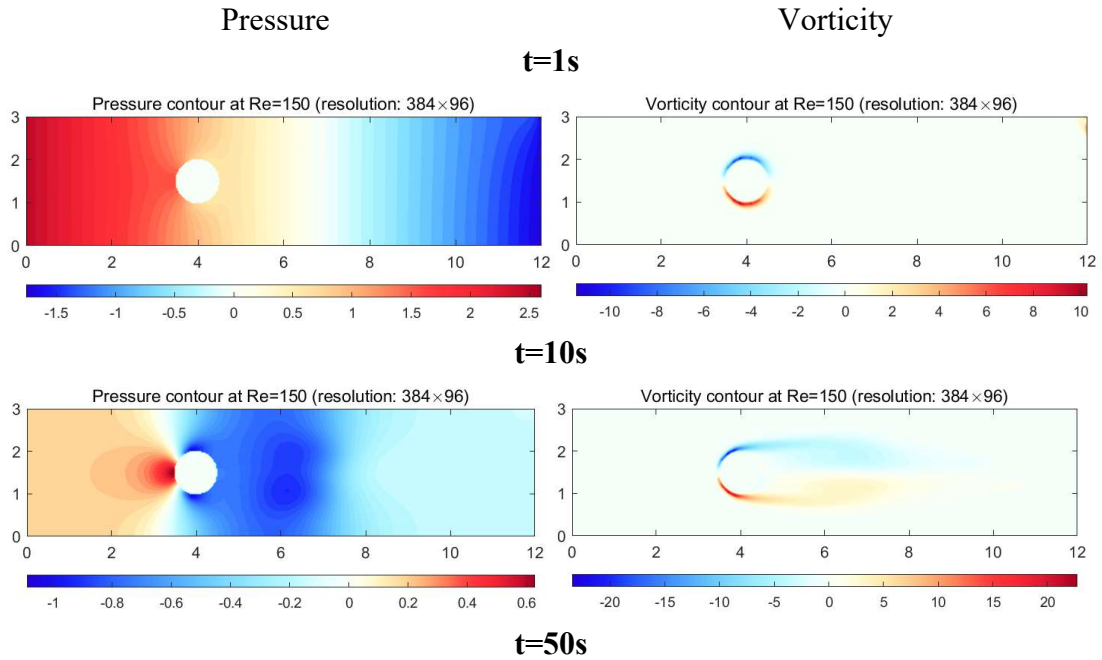


Figure 35 Horizontal and vertical velocity contours at several time intervals in small domain



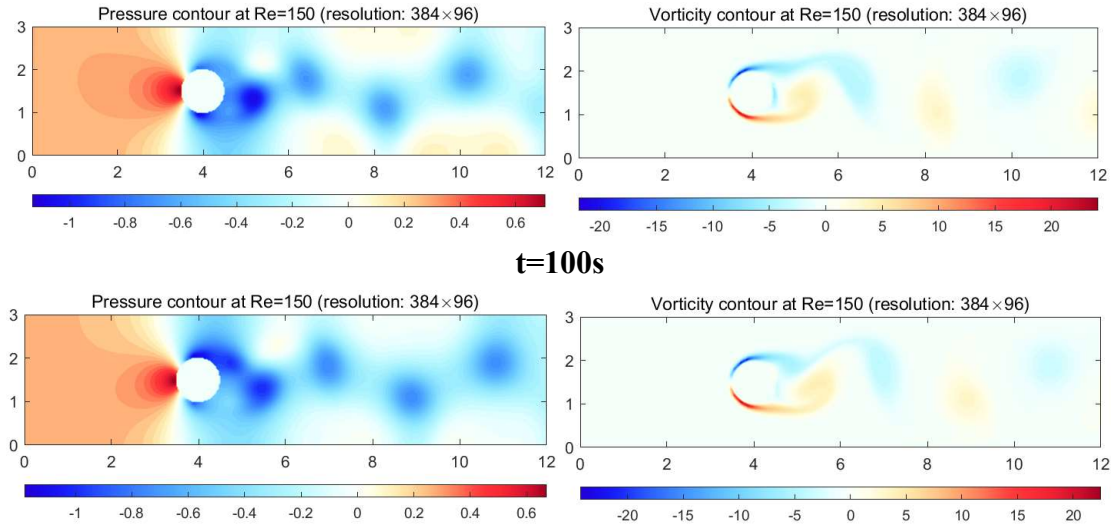


Figure 36 Pressure and vorticity contours at several time intervals in small domain

The developing trend is also similar with that from original domain, with slight differences in pressure drag coefficient and vortex-shedding frequency.

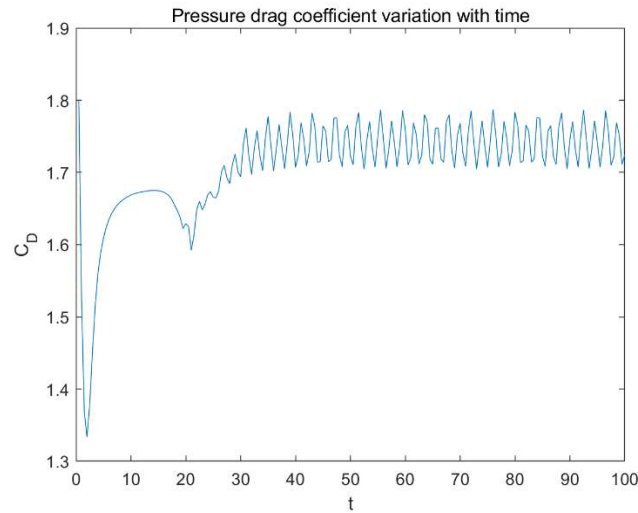


Figure 37 Pressure drag coefficient variation with time in small domain

Figure 37 exhibits the pressure drag coefficient variation with time in small domain. The coefficient also varies sharply when the flow is in developing state. The flow reaches the stable periodic state at 40s. After $t = 40s$, the coefficient fluctuates periodically with the mean value as $C_D = 1.74$. Comparing with the case from the original domain, the drag coefficient is larger. Perform Fourier transformations on the drag coefficient curve between $t = 60s$ and $t = 100s$, the spectrum of drag coefficient is shown in Figure 34.

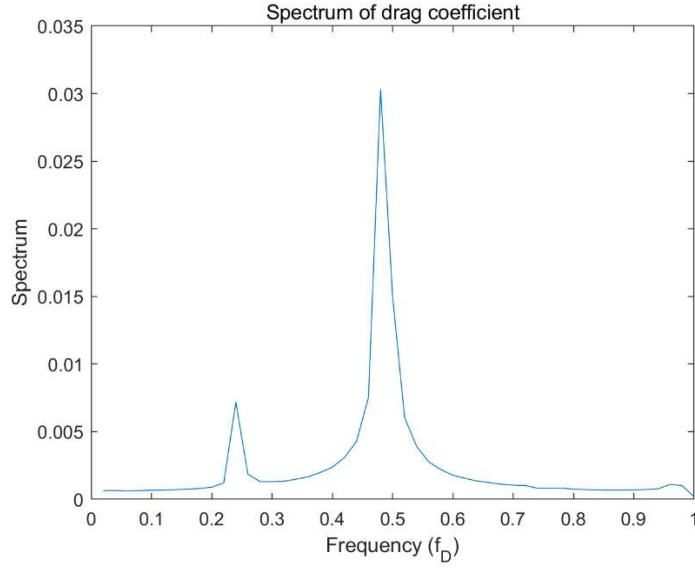


Figure 34 Spectrum of pressure drag coefficient in small domain

The frequency of drag coefficient is $f_D = 0.48$, larger than that from the original domain. Therefore, the Strouhal number is $St = 0.24$. Compared with the published results of Qu et al. [2], as shown in Table 7, the drag coefficient and the Strouhal number do not match as well as that from original domain.

	Pressure drag coefficient C_{Dp}	Strouhal number St
Original domain	1.22	0.22
Small domain	1.74	0.24
Qu et al. [2]	1.024	0.1841

Table 7 Comparison of C_D and St with those of Qu et al. [2] in small domain

Due to the small domain size, the effects of the outer boundaries on the flow around the cylinder become more pronounced, resulting in lower accuracy in the predicted results. Therefore, to minimize the extraneous effects of the outer boundaries, we need a sufficiently large domain to accurately simulate the flow.

4. Discussions

4.1 Comparison of pressure, velocity, and vorticity contours

4.1.1 Comparison of results at different Reynolds numbers

The pressure, velocity, and vorticity contours at different Reynolds numbers at $t = 50s$ and $t = 100s$ are shown in Figure 35. The first line are u velocity contours, the second line are pressure contours and the third line are vorticity contours.

150

300

1000

$t=50s$

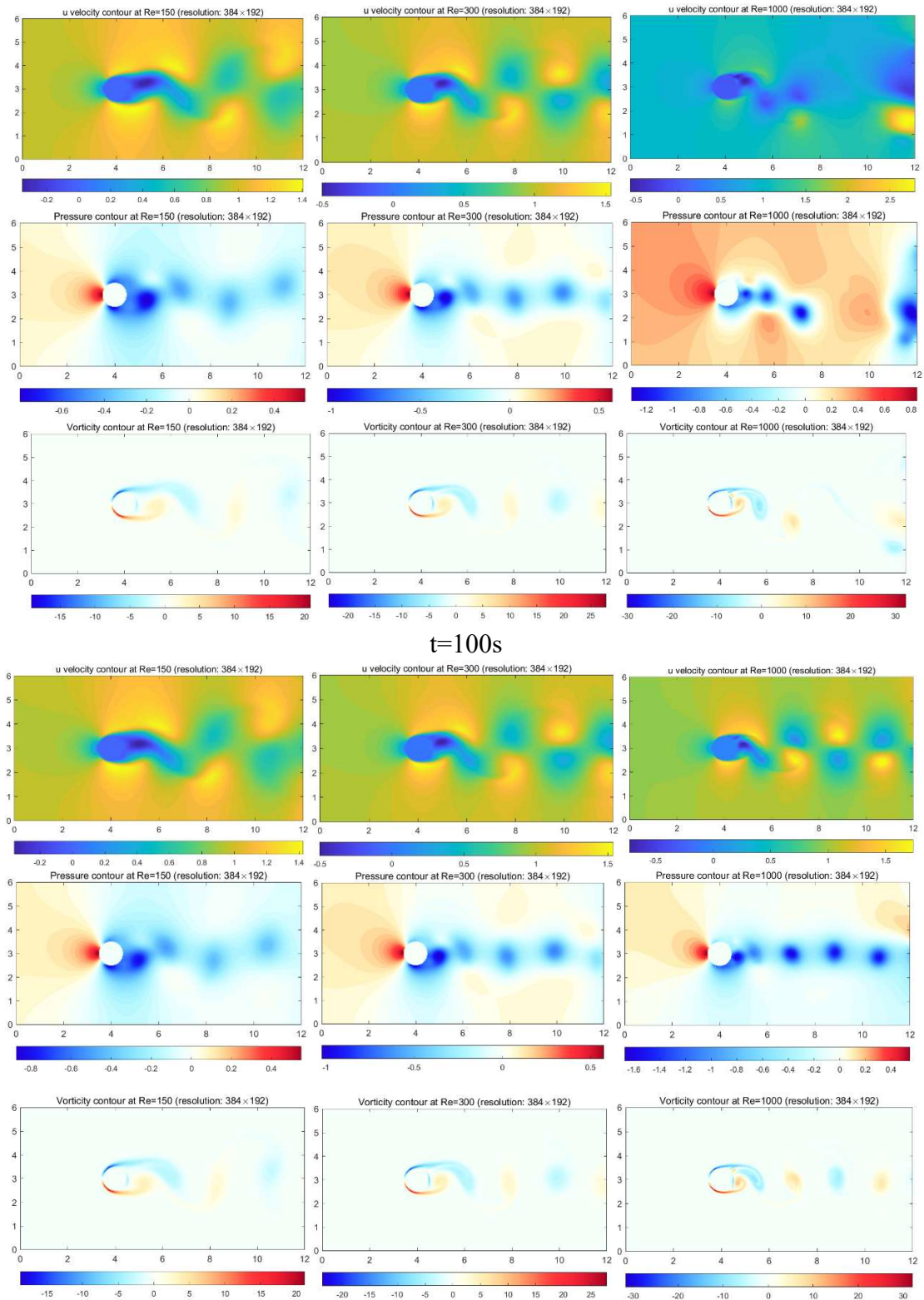


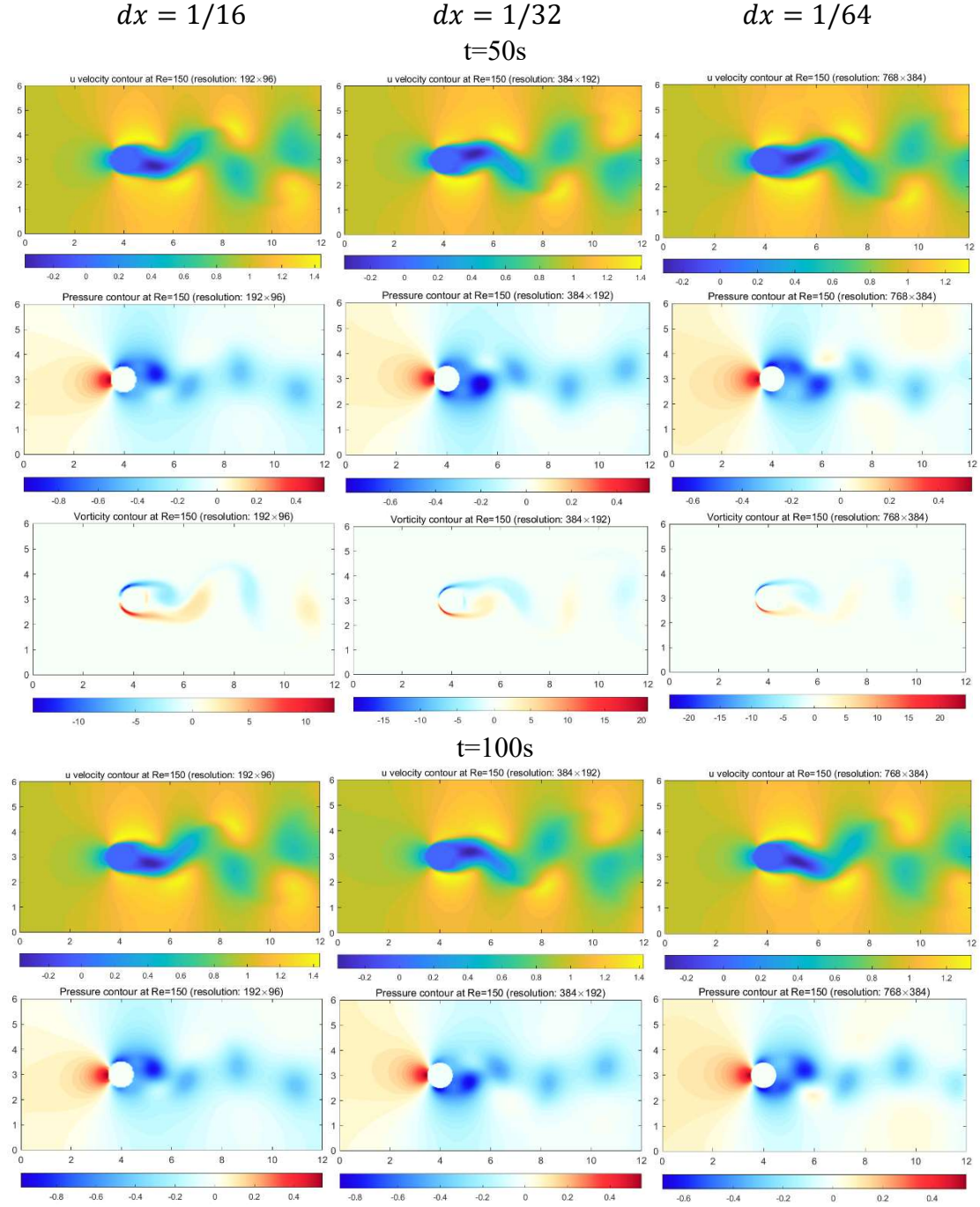
Figure 35 Comparison of contours at different Reynolds numbers

The contours at different Reynolds number are similar with each other. Compared to the cases with low Reynolds number, as observed from the contours, the large Reynolds number case exhibits greater disturbance. The vortices align behind the cylinder, and compared to the other two cases, the vortices in this case are more

neatly arranged. Observed from the pressure contours, the pressure extreme points are also align more organized when Reynolds number is large.

4.1.2 Comparison of results at different grid resolutions

The pressure, velocity, and vorticity contours at different grid resolutions at $t = 50s$ and $t = 100s$ are shown in Figure 36. The first line are u velocity contours, the second line are pressure contours and the third line are vorticity contours.



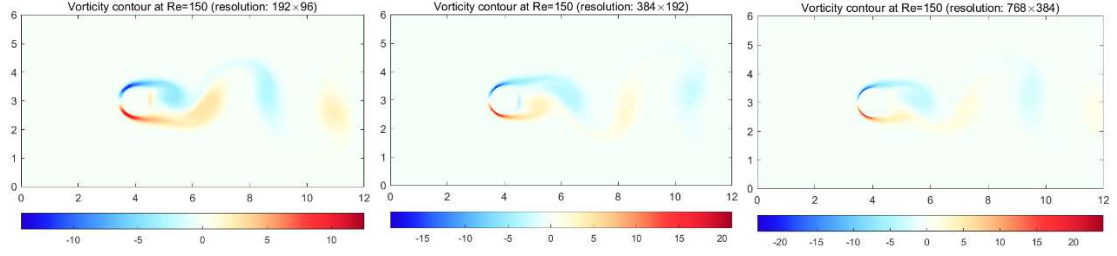
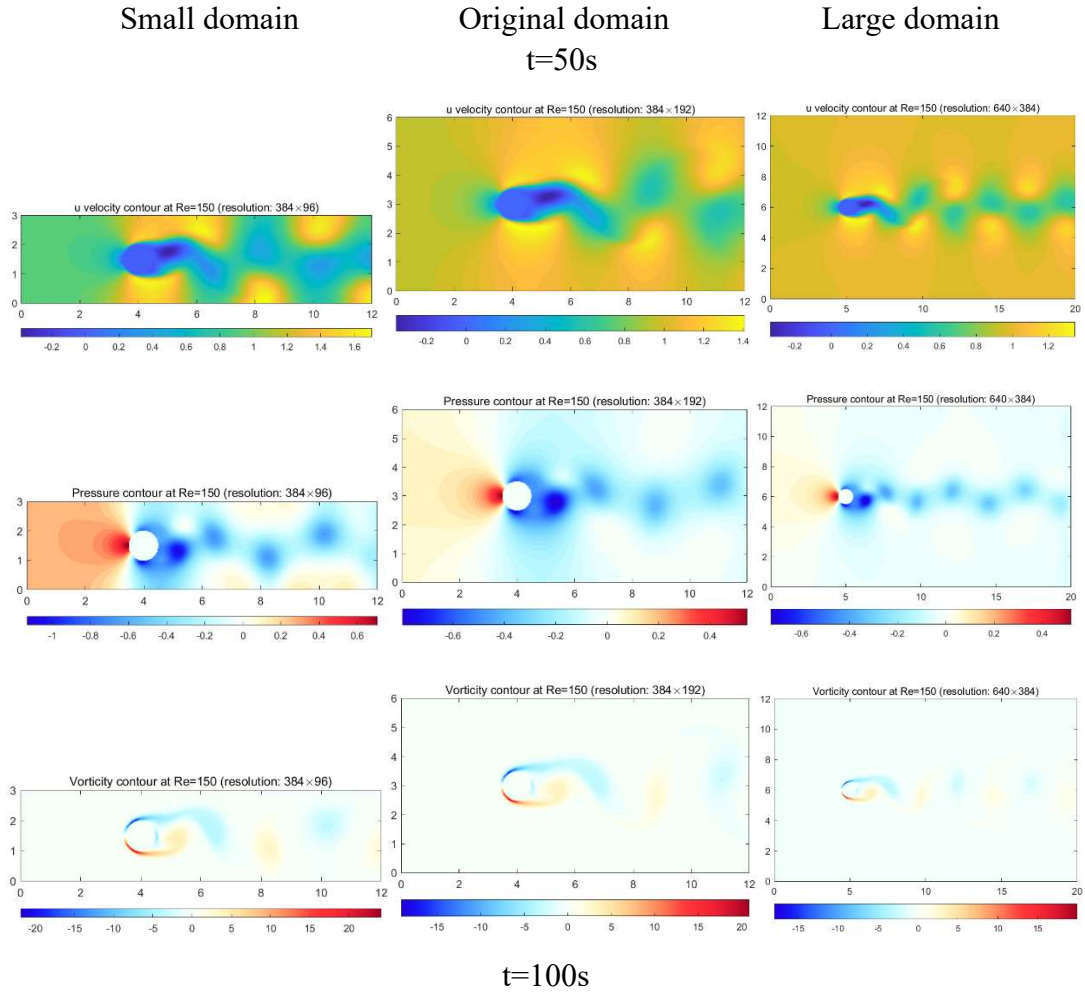


Figure 36 Comparison of contours at different grid resolutions

The contours at different grid resolutions are similar to each other. As observed from the contours, the flow structure with higher grid resolution is more organized. Additionally, the cylinder structure is captured better in the higher resolution case, where the circle is smoother and less polygonal.

4.1.3 Comparison of results with different domain sizes

The pressure, velocity, and vorticity contours at different domain sizes at $t = 50s$ and $t = 100s$ are shown in Figure 37. The first line are u velocity contours, the second line are pressure contours and the third line are vorticity contours.



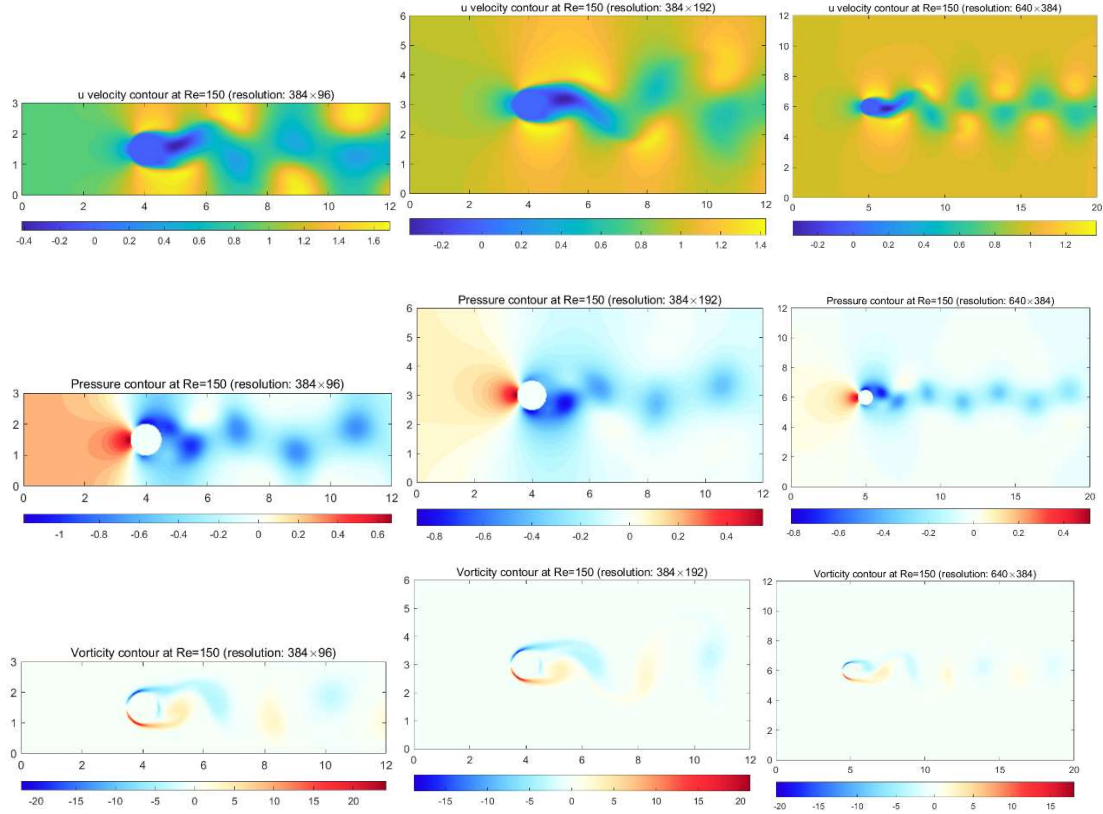


Figure 37 Comparison of contours at different domain sizes

The flow structure in a larger domain is more well-organized. The vortices and pressure extremes are aligned more neatly. When the domain is small, the wake and pressure field cannot develop well and are significantly influenced by the boundaries, which equivalently compresses the fluid. This is also why the drag coefficient calculated in a smaller domain is higher.

4.2 Global results for different cases

The global results of the pressure drag coefficient (C_{Dp}), the Strouhal number (St) and the time needed to reach the stable periodic state (t) for different Reynolds number, different grid resolutions and different domain sizes are shown in Table 8.

	Basic results $Re_D = 150$	Re_D 300	Re_D 1000	Half resolution	Double resolution	Small domain	Large domain	Qu et al. [2] $Re_D = 150$	Franke et al. [3] $Re_D = 300$	Qu et al. [2] $Re_D = 1000$
C_{Dp}	1.22	1.32	1.58	1.31	1.08	1.75	1.12	1.024	1.11	1.34
St	0.22	0.22	0.25	0.2	0.19	0.24	0.19	0.1841	0.205	0.236
$t(s)$	50	30	20	40	60	40	60	—	—	—

Table 8 Global results for different Reynolds number, different grid resolutions and different domain sizes

From Table 8, it can be seen that the pressure drag coefficient increases with increasing Reynolds number, the same as the trend of the frequency of vortex-shedding. The larger the Reynolds number, the shorter the time required to reach the stable periodic state. Comparing with the results from the published papers [2,3], it can be seen that this algorithm is more accurate at simulating lower Reynolds numbers ($Re_D = 150, Re_D = 300$) than higher Reynolds numbers ($Re_D = 1000$). In terms of grid resolution, the high-resolution grid ($dx = 1/64$) results are closer to the paper's results compared to the low-resolution grid ($dx = 1/16, dx = 1/32$), indicating that the high-resolution grid results are more accurate. Additionally, the time required to reach the stable periodic state is longer for higher resolution grid. Regarding domain size, the results from the larger domain are closer to the paper's values compared to the smaller domain, and the time required to reach the stable periodic state is also longer. This is because, in a larger domain, the simulation is less influenced by the outer boundaries and more closely resembles the flow around a cylinder in a uniform flow. In summary, this algorithm is more accurate at simulating lower Reynolds numbers than higher Reynolds numbers. With a high-resolution grid and a larger domain, the results are more accurate, and the time required to reach the stable periodic state is longer.

5. Flow around an elliptic cylinder with attack angle 30°

5.1 Configuration of flow around an elliptic cylinder

Figure 38 illustrates the configuration of flow around an elliptic cylinder. Normalized by the cylinder's major axis, the computational domain dimensions are $L_x = 12, L_y = 6$. The Reynolds number is set to be $Re_D = 300$. The initial conditions for velocity components u, v , and pressure p are all zeros. The grid size is $dx = dy = 1/32$. Consequently, the number of grids in the horizontal and vertical directions are $N_x = 384, N_y = 192$, respectively. The cylinder is positioned far from the outer boundaries to minimize the adverse effects of these boundaries. The outer boundary conditions are the same as those of circular cylinder case.

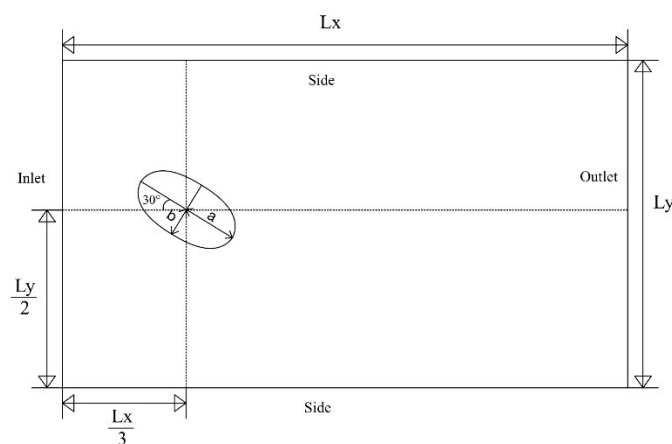
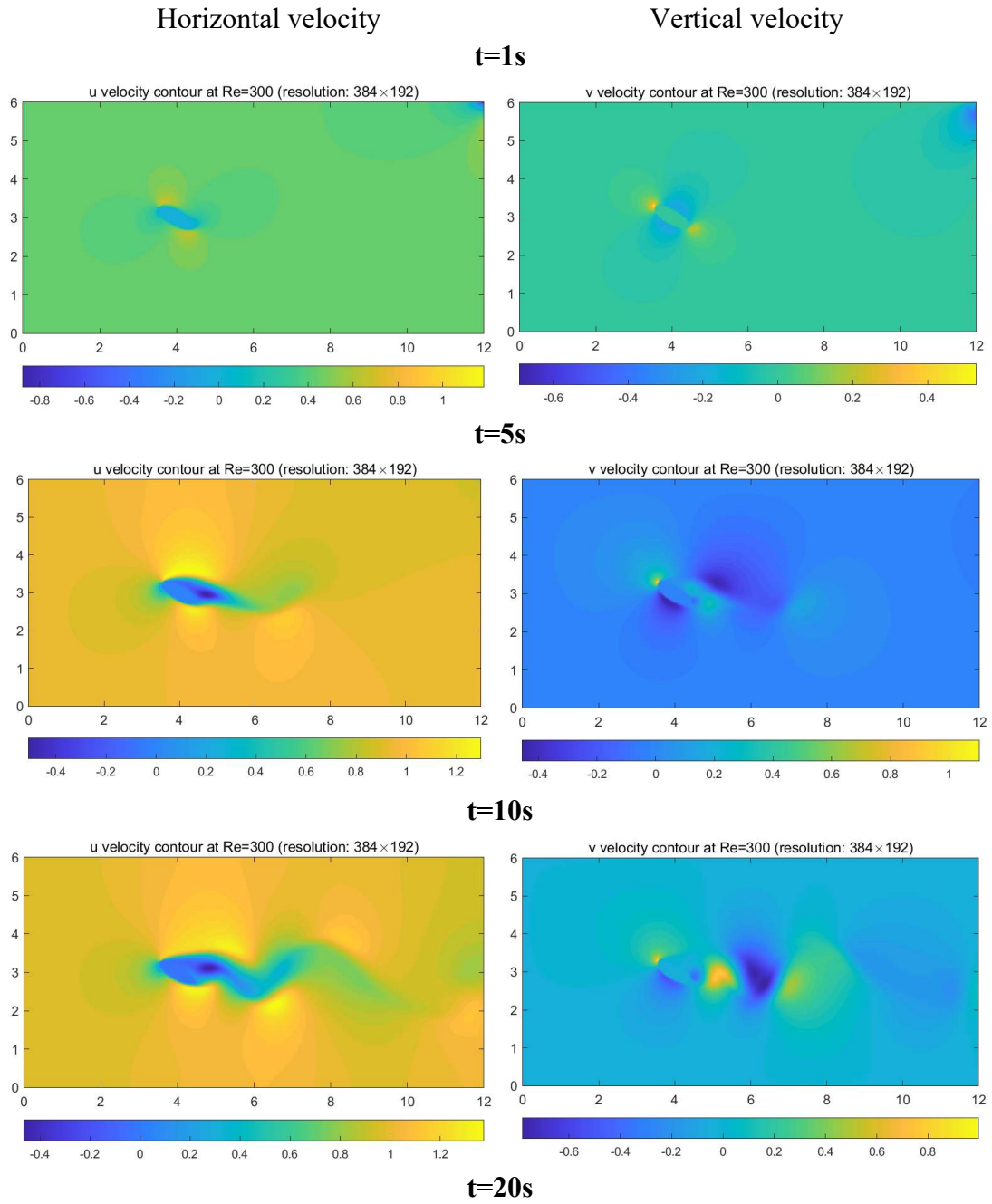
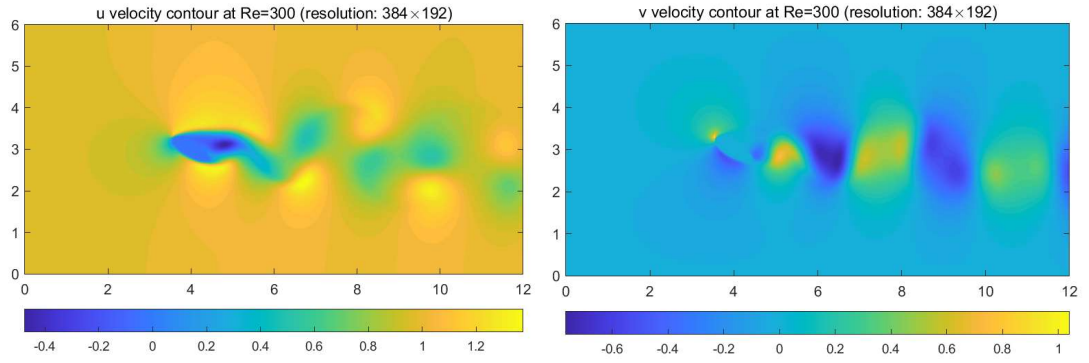


Figure 38 Configuration of flow around an elliptic cylinder

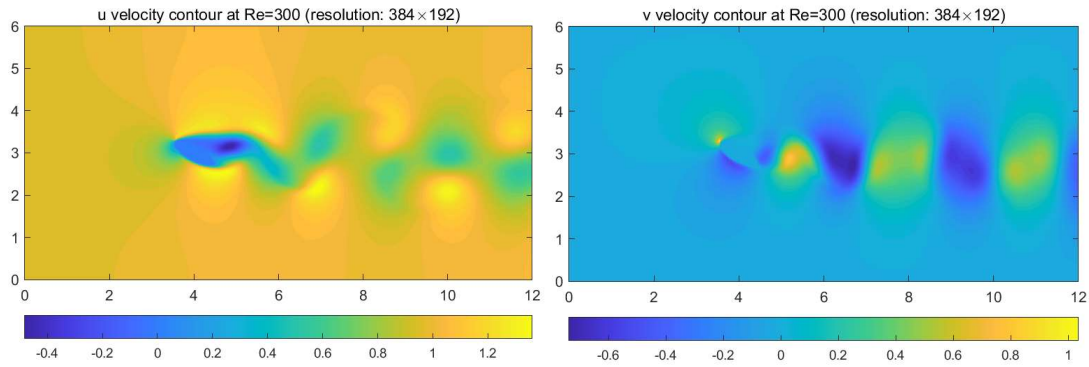
5.2 Pressure, velocity and vorticity contours

The pressure, velocity, vorticity contours of the flow around an elliptic cylinder at $Re_D = 300$ are shown in Figure 39 and Figure 40.





t=50s



t=100s

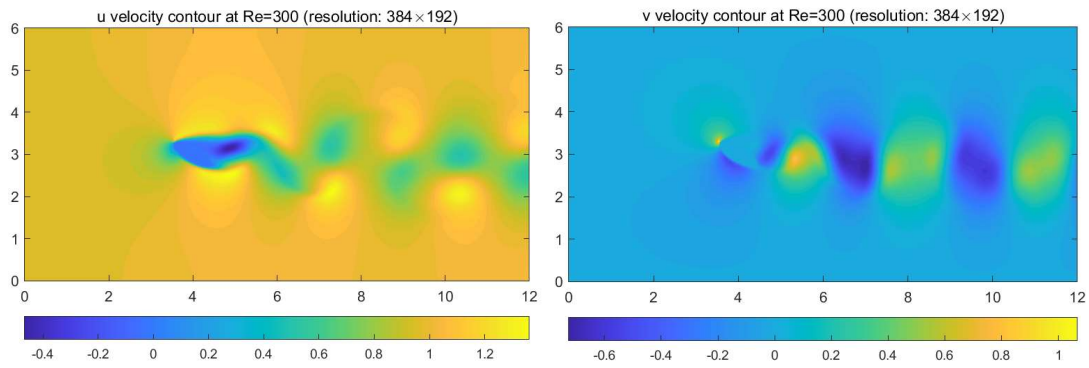
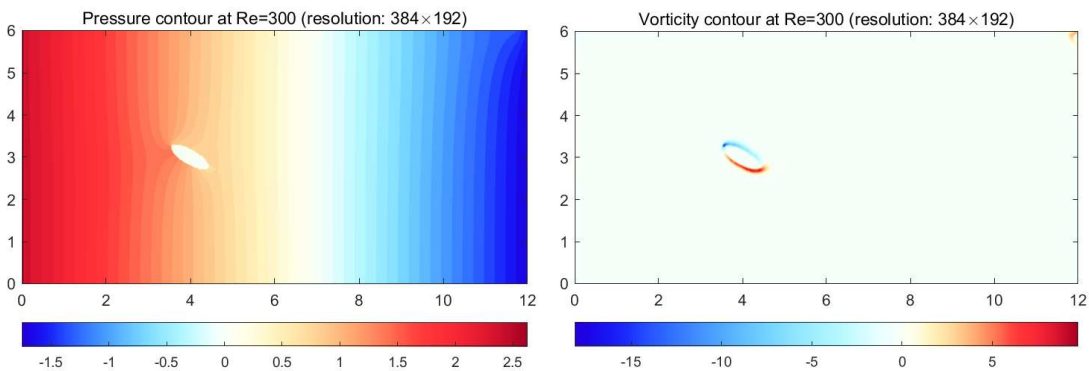


Figure 39 Horizontal and vertical velocity contours at several time intervals

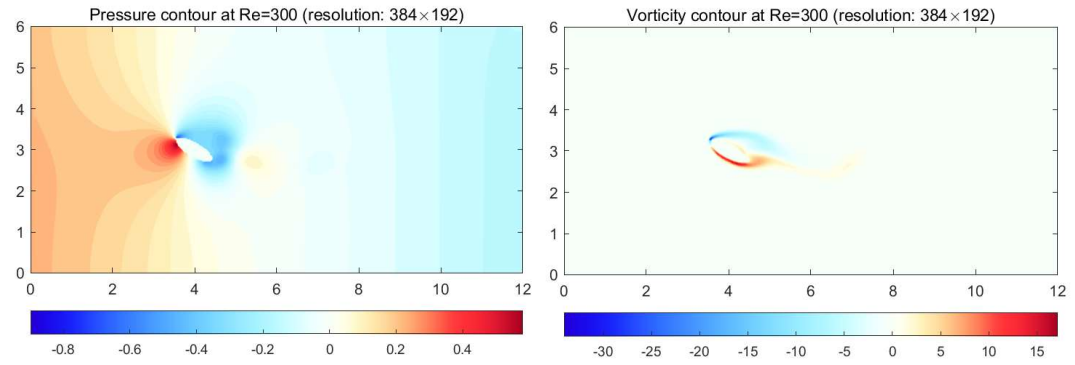
Pressure

Vorticity

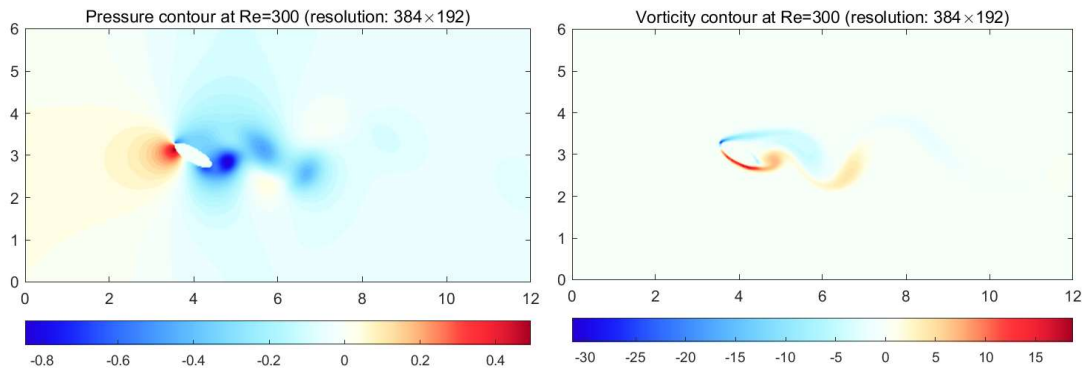
t=1s



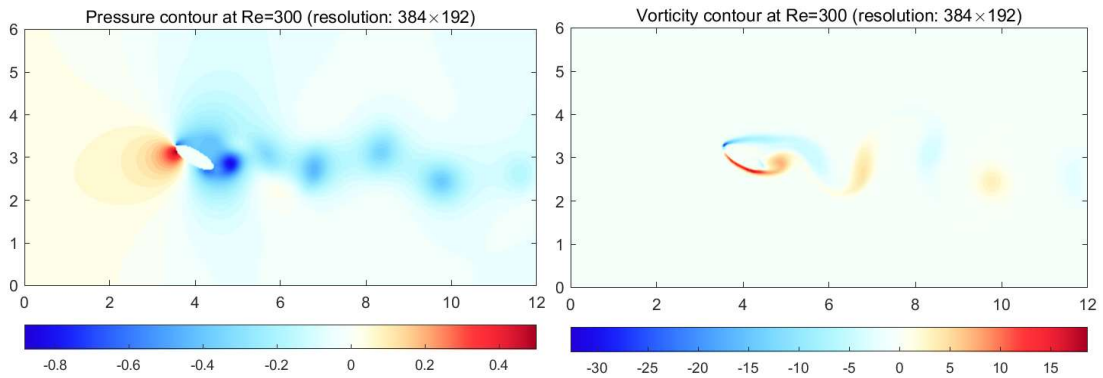
t=5s



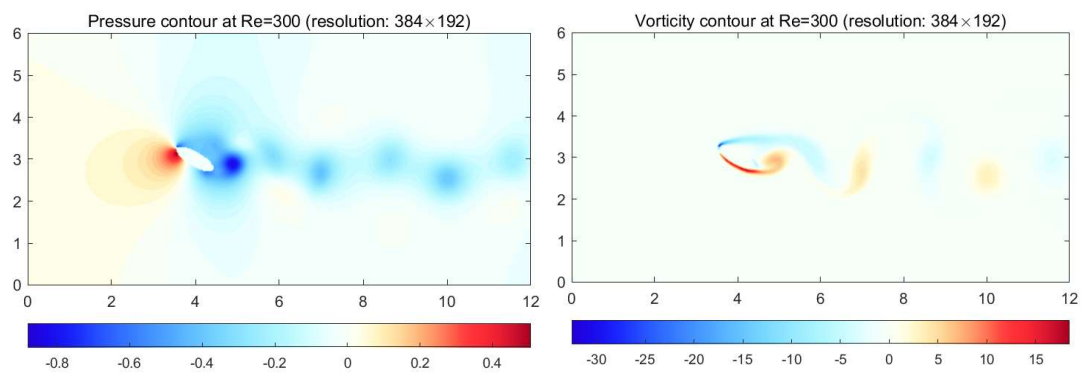
t=10s



t=20s



t=50s



t=100s

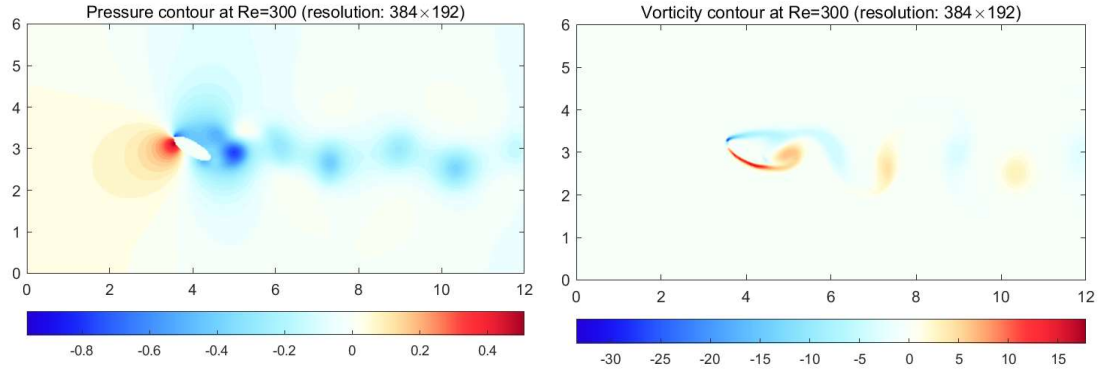


Figure 40 Pressure and vorticity contours at several time intervals

The development process is similar to that of flow around a cylinder. Initially, at $t = 1s$, the flow is well-distributed, with a uniform pressure gradient and no vorticity wake. At this point, there is no vortex-shedding, and the flow remains stable. After $t = 10s$, the vortices behind the cylinder begin to shed, and the pressure field behind the cylinder starts to fluctuate. After $t = 20s$, the vortices shed regularly, forming a vortex street. These contours demonstrate that the algorithm can successfully simulate vortex shedding phenomena for flow around an elliptic cylinder.

5.3 Lift and pressure drag coefficients

Figure 41 exhibits the pressure drag coefficient variation with time.

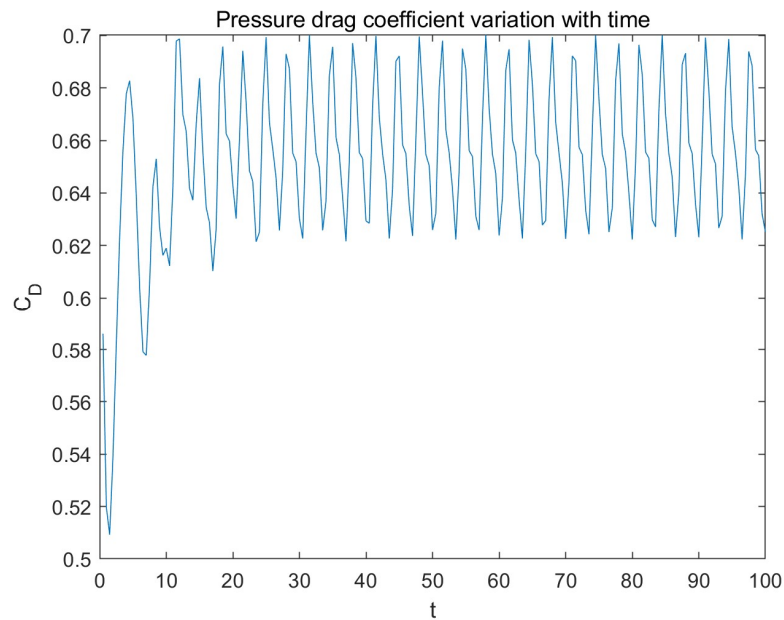


Figure 41 Pressure drag coefficient variation with time for flow around an elliptic cylinder

Compared with the circular cylinder case, in this case the drag coefficient reaches the stable periodic state early. After only $t = 20s$, the coefficient fluctuates periodically with the mean value as $C_D = 0.66$.

Figure 42 exhibits the lift coefficient variation with time.

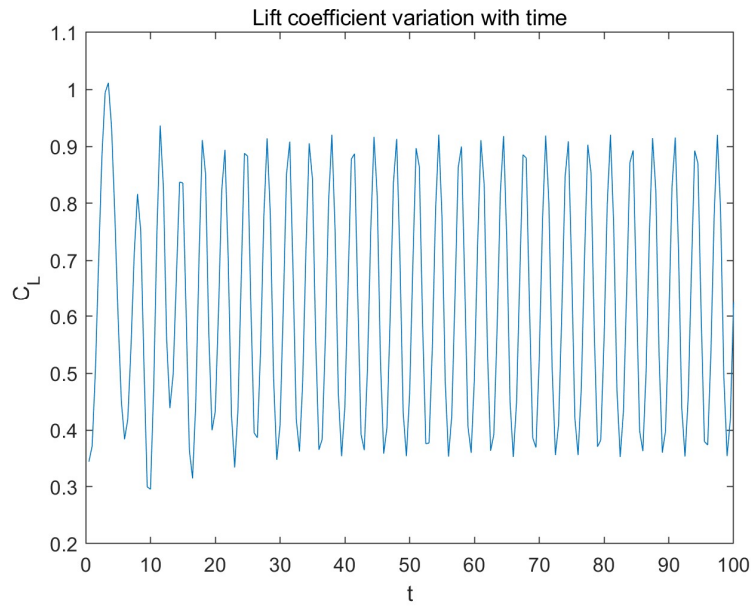


Figure 42 Lift coefficient variation with time for flow around an elliptic cylinder

The lift coefficient is more stable than the drag coefficient. After $t = 20s$, the coefficient fluctuates periodically with the mean value as $C_L = 0.62$. Perform Fourier transformations on the lift coefficient curve between $t = 50s$ and $t = 100s$, the spectrum of lift coefficient is shown in Figure 43.

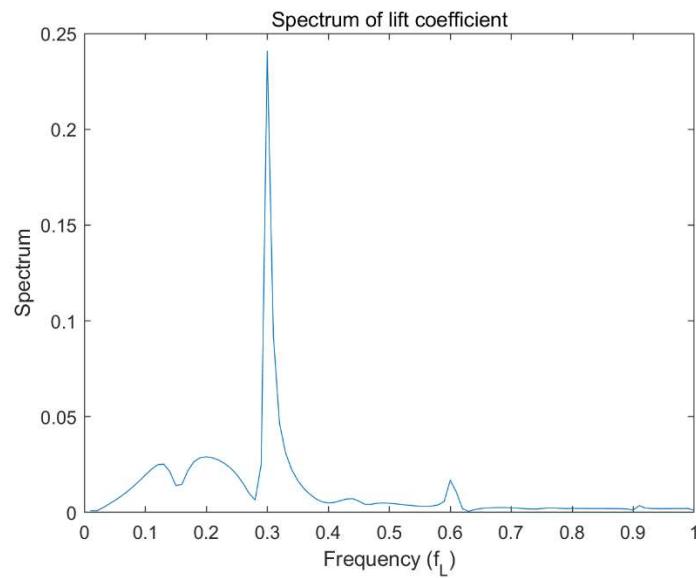


Figure 43 Spectrum of lift coefficient for flow around an elliptic cylinder

Therefore, the vortex-shedding frequency is $f = 0.3$. The Strouhal number is

$$St = \frac{fD}{U} = 0.3$$

These are the simulation results of flow around an elliptic cylinder.

6. Conclusions

This study has effectively demonstrated the capabilities of the implemented algorithm to analyze the flow characteristics around a cylinder across different Reynolds numbers, grid resolutions, and domain sizes. Through rigorous simulation using the incompressible Navier-Stokes equations, we have detailed the flow structures, including vortex shedding, wake formation, and the effects of boundary conditions.

Our findings confirm that higher Reynolds numbers correlate with increased pressure drag coefficients and more rapid attainment of a stable periodic state, while also introducing greater disturbances in flow patterns. This trend of the flow developing was observed consistently across the simulations for $Re_D = 150, 300$ and 1000 . The simulations at lower Reynolds numbers (150 and 300) displayed more accurate alignment with published studies than those at $Re_D = 1000$, highlighting challenges associated with simulating higher Reynolds numbers.

In terms of grid resolution, results from higher resolution grids proved more precise, revealing more organized flow structures and smoother contouring around the cylinder. Additionally, the simulations highlighted that larger domain sizes allow for a more accurate representation of the flow, reducing the influence of outer boundary conditions and better mimicking unbounded fluid flow conditions.

Moreover, the study has extended into the flow around an elliptic cylinder at an angle of attack, showcasing the versatility and robustness of the computational methods employed. This setup also confirmed the algorithm's ability to capture the vortex-shedding phenomenon, the lift and pressure drag coefficients for flow around the elliptic cylinder.

This study contributes to the understanding of fluid dynamics around cylindrical structures. This comprehensive investigation has reinforced the critical role of CFD in modern engineering, providing a deeper insight into fluid mechanics phenomena. The insights gained have significant implications for the design and analysis of engineering structures.

7. References

- [1] Massachusetts Institute of Technology. (n.d.). Lecture 15: Vortex-induced vibrations [PDF file]. Retrieved from <https://web.mit.edu/13.021/demos/lectures/lecture15.pdf>
- [2] Qu, L., Norberg, C., Davidson, L., Peng, S. H., & Wang, F. (2013). Quantitative numerical analysis of flow past a circular cylinder at Reynolds number between 50 and 200. *Journal of Fluids and Structures*, 39, 347-370.
- [3] Franke, R., Rodi, W., & Schönung, B. (1990). Numerical calculation of laminar vortex-shedding flow past cylinders. *Journal of Wind Engineering and Industrial Aerodynamics*, 35, 237-257.

8. Appendix

The link of the code on GitHub (written in MATLAB, including the main code and the post-processing code):

<https://github.com/LiWeixuan0190/CFD-Final-Project.git>

MATLAB source code:

1. Main code:

```
clear; clc;
% Define parameters
dt = 0.01;
Re = 150;
dx = 1/32;
dy = dx;
Lx = 12;
Ly = 6;
D = 1;
bc = 1;
Nx = Lx/dx;
Ny = Ly/dy;
omega_u = 1;
omega_p = 1.5;

% ifluid
x_center = 4;
y_center = Ly/2;
r = D/2;
x = linspace(0, Lx, Nx+2);
y = linspace(0, Ly, Ny+2);
[X, Y] = meshgrid(x, y);
dist_squared = (X-x_center).^2+(Y-y_center).^2;
ifluid = zeros(Ny+2, Nx+2);
```

```

ifluid(dist_squared >= r^2) = 1;

% Initialization
un_m1 = zeros(Ny+2,Nx+2);
Un_m1 = zeros(Ny+2,Nx+2);
vn_m1 = zeros(Ny+2,Nx+2);
Vn_m1 = zeros(Ny+2,Nx+2);
if bc == 1
    [un_m1,Un_m1,vn_m1,Vn_m1] =
BC(un_m1,Un_m1,vn_m1,Vn_m1,dy,Ly,Nx,Ny,ifluid);
else
    [un_m1,Un_m1,vn_m1,Vn_m1] =
BC_wall(un_m1,Un_m1,vn_m1,Vn_m1,dy,Ly,Nx,Ny,ifluid);
end
un = un_m1;
Un = Un_m1;
vn = vn_m1;
Vn = Vn_m1;
p_old = zeros(Ny+2,Nx+2);
p = zeros(Ny+2,Nx+2);

u_sample = zeros(Ny+2,Nx+2,200);
v_sample = zeros(Ny+2,Nx+2,200);
p_sample = zeros(Ny+2,Nx+2,200);

% Main loop
clc;
res = 1;
n = 0;
while n<10000
    t_total=tic;
    n = n+1;
    % Step 1
    NL_u = AB2(un_m1,un,Un_m1,Un,Vn_m1,Vn,Nx,Ny,dx,dy,dt);
    NL_v = AB2(vn_m1,vn,Un_m1,Un,Vn_m1,Vn,Nx,Ny,dx,dy,dt);
    dfus_u = diffusion(un,Nx,Ny,dt,Re,dx,dy,ifluid);
    dfus_v = diffusion(vn,Nx,Ny,dt,Re,dx,dy,ifluid);
    tu=tic;
    [u_star,U_star,iter_u] =
line_sor(NL_u,dfus_u,un,dt,Re,dx,dy,Nx,Ny,1,omega_u,Ly,ifluid);
    time_u_star = toc(tu);
    tv=tic;
    [v_star,V_star,iter_v] =
line_sor(NL_v,dfus_v,vn,dt,Re,dx,dy,Nx,Ny,2,omega_u,Ly,ifluid);

```

```

time_v_star = toc(tv);
if bc == 1
    [u_star,U_star,v_star,V_star] =
BC(u_star,U_star,v_star,V_star,dy,Ly,Nx,Ny,ifluid);
else
    [u_star,U_star,v_star,V_star] =
BC_wall(u_star,U_star,v_star,V_star,dy,Ly,Nx,Ny,ifluid);
end

% Step 2
tp=tic;
[p,iter_p] = Possion(U_star,V_star,dx,dy,dt,Nx,Ny,p_old,omega_p,ifluid);
time_p = toc(tp);

% Step 3
[un_p1,Un_p1] =
fractional_step(u_star,U_star,p,dt,dx,dy,Nx,Ny,1,ifluid);
[vn_p1,Vn_p1] =
fractional_step(v_star,V_star,p,dt,dx,dy,Nx,Ny,2,ifluid);
if bc == 1
    [un_p1,Un_p1,vn_p1,Vn_p1] =
BC(un_p1,Un_p1,vn_p1,Vn_p1,dy,Ly,Nx,Ny,ifluid);
else
    [un_p1,Un_p1,vn_p1,Vn_p1] =
BC_wall(un_p1,Un_p1,vn_p1,Vn_p1,dy,Ly,Nx,Ny,ifluid);
end
umax = max(un_p1, [], 'all');
vmax = max(vn_p1, [], 'all');
cfl_u = umax*dt/dx^2;
cfl_v = vmax*dt/dy^2;
iunstable = 0;
if cfl_u > 1 || cfl_v > 1
    iunstable = 1;
end

% Step 4
tres=tic;
[max_residual,sum_residual] = residual(un_p1,un,vn_p1,vn,p,p_old);
res = sum_residual;
time_res = toc(tres);

% Update u,v
un_m1 = un;
vn_m1 = vn;

```

```

Un_m1 = Un;
Vn_m1 = Vn;

un = un_p1;
vn = vn_p1;
Un = Un_p1;
Vn = Vn_p1;
p_old = p;
time_total = toc(t_total);

if rem(n, 1) == 0
    fprintf('n=%d, T=%f, res=%f, iter_u=%d, iter_v=%d, iter_p=%d,
iunstable=%d, cfl_u=%f, cfl_v=%f, max_residual=%f, sum_residual=%f\n', ...
        n, n*dt, res, iter_u, iter_v, iter_p, iunstable, cfl_u, cfl_v,
max_residual, sum_residual);
    fprintf('T_u=%.2f%%, T_v=%.2f%%, T_p=%.2f%%, T_res=%.2f%%\n', ...
        time_u_star/time_total * 100, time_v_star/time_total * 100,
time_p/time_total * 100, time_res/time_total * 100);
end

rsampling = 50;
if rem(n, rsampling) == 0
    u_sample(:, :, n/rsampling) = un;
    v_sample(:, :, n/rsampling) = vn;
    p_sample(:, :, n/rsampling) = p;
end

if mod(n, 2000) == 0
    filename = sprintf('Data_%d.mat', n/2000);
    save(filename);
end

end

%% Step 1 u_star
function [u_star, U_star, k] =
line_sor(NL, dfus, un, dt, Re, dx, dy, Nx, Ny, uv, omega, Ly, ifluid)
    src = NL + dfus;
    R = dt / (2 * Re);
    rtot = 1;
    uk = un;
    k = 0;
    cw = ones(Ny + 2, Nx + 2) * (-R / dx^2);
    ce = ones(Ny + 2, Nx + 2) * (-R / dx^2);

```



```

cn = ones(Ny+2,Nx+2)*(-R/dy^2);
cs = ones(Ny+2,Nx+2)*(-R/dy^2);
cp = ones(Ny+2,Nx+2)*(1+2*R/dx^2+2*R/dy^2);
[cp,ce,cw,cn,cs,src] = coeff_u(cp,ce,cw,cn,cs,src,ifluid,Nx,Ny);
while rtot > 1e-6
    k = k+1;
    temp = uk;
    for i = 2:Ny+1
        B = src(i,2:Nx+1)-cs(i,2:Nx+1).*temp(i-1,2:Nx+1)-
cn(i,2:Nx+1).*temp(i+1,2:Nx+1);
        temp(i,2:Nx+1) =
tdmac(cw(i,3:Nx+1),cp(i,2:Nx+1),ce(i,2:Nx),B,Nx);
    end
    temp(2:Ny+1,2:Nx+1)=(1-
omega).*uk(2:Ny+1,2:Nx+1)+omega.*temp(2:Ny+1,2:Nx+1);
    if uv == 1
        [temp,~,~,~] = BC(temp,temp,temp,temp,dy,Ly,Nx,Ny,ifluid);
    else
        [~,~,temp,~] = BC(temp,temp,temp,temp,dy,Ly,Nx,Ny,ifluid);
    end
    uk_1 = temp;
    udiff = uk_1(2:end-1,2:end-1) - uk(2:end-1,2:end-1);
    rtot = rms(udiff,'all');
    uk = uk_1;
end
u_star = uk_1;
U_star = zeros(Ny+2,Nx+2);
if uv == 1
    U_star(2:Ny+1,2:Nx+2) =
(u_star(2:Ny+1,2:Nx+2)+u_star(2:Ny+1,1:Nx+1))/2;
else
    U_star(2:Ny+2,2:Nx+1) =
(u_star(2:Ny+2,2:Nx+1)+u_star(1:Ny+1,2:Nx+1))/2;
end

end

function NL = AB2(un_m1,un,Un_m1,Un,Vn_m1,Vn,Nx,Ny,dx,dy,dt)
NL = zeros(Ny+2,Nx+2);
for i = 2:Ny+1
    NL(i,2:Nx+1) = -1/2*dt*( ...
        3*((Un(i,3:Nx+2).*(un(i,3:Nx+2)+un(i,2:Nx+1)))/2 -
Un(i,2:Nx+1).*(un(i,2:Nx+1)+un(i,1:Nx))/2)/dx ...

```

```

        + (Vn(i+1,2:Nx+1).*(un(i+1,2:Nx+1)+un(i,2:Nx+1))/2 -
Vn(i,2:Nx+1).*(un(i,2:Nx+1)+un(i-1,2:Nx+1))/2)/dy) ...
        - ((Un_m1(i,3:Nx+2).*(un_m1(i,3:Nx+2)+un_m1(i,2:Nx+1))/2 -
Un_m1(i,2:Nx+1).*(un_m1(i,2:Nx+1)+un_m1(i,1:Nx))/2)/dx ...
        + (Vn_m1(i+1,2:Nx+1).*(un_m1(i+1,2:Nx+1)+un_m1(i,2:Nx+1))/2 -
Vn_m1(i,2:Nx+1).*(un_m1(i,2:Nx+1)+un_m1(i-1,2:Nx+1))/2)/dy) ...
    );

    end
end

function dfus = diffusion(un,Nx,Ny,dt,Re,dx,dy,ifluid)
    dfus = zeros(Ny+2,Nx+2);
    src = zeros(Ny+2,Nx+2);
    R = dt/2/Re;
    cw = ones(Ny+2,Nx+2)*R/dx^2;
    ce = ones(Ny+2,Nx+2)*R/dx^2;
    cn = ones(Ny+2,Nx+2)*R/dy^2;
    cs = ones(Ny+2,Nx+2)*R/dy^2;
    cp = ones(Ny+2,Nx+2)*(1+R*(-2/dx^2-2/dy^2));
    [cp,ce,cw,cn,cs,src]=coeff_u(cp,ce,cw,cn,cs,src,ifluid,Nx,Ny);
    dfus(2:Ny+1,2:Nx+1) =
cp(2:Ny+1,2:Nx+1).*un(2:Ny+1,2:Nx+1)+ce(2:Ny+1,2:Nx+1).*un(2:Ny+1,3:Nx+2) .
..

+cw(2:Ny+1,2:Nx+1).*un(2:Ny+1,1:Nx)+cn(2:Ny+1,2:Nx+1).*un(3:Ny+2,2:Nx+1) ...
+cs(2:Ny+1,2:Nx+1).*un(1:Ny,2:Nx+1);

end

%% Step 2 p
function [p,k] = Possion(U_star,V_star,dx,dy,dt,Nx,Ny,p_old,omega,ifluid)
    pk = p_old;
    rtot = 1;
    U_star(2:Ny-1,end)=U_star(2:Ny-1,end)+mean(U_star(2:Ny-1,2)-
U_star(2:Ny-1,end));
    src = zeros(Ny+2,Nx+2);
    src(2:Ny+1,2:Nx+1) = ifluid(2:Ny+1,2:Nx+1).* ...
        (((U_star(2:Ny+1,3:Nx+2)-
U_star(2:Ny+1,2:Nx+1))/dx+(V_star(3:Ny+2,2:Nx+1)-
V_star(2:Ny+1,2:Nx+1))/dy)/dt);
    k = 0;
    cw = ones(Ny+2,Nx+2)*(1/dx^2);
    ce = ones(Ny+2,Nx+2)*(1/dx^2);
    cn = ones(Ny+2,Nx+2)*(1/dy^2);
    cs = ones(Ny+2,Nx+2)*(1/dy^2);

```

```

cp = ones(Ny+2,Nx+2)*(-2/dx^2-2/dy^2);
[cp,ce,cw,cn,cs,src] = coeff_p(cp,ce,cw,cn,cs,src,ifluid,Nx,Ny);
while rtot > 1e-6
    k = k+1;
    temp = pk;
    for i = 2:Ny+1
        B = src(i,2:Nx+1)-cs(i,2:Nx+1).*temp(i-1,2:Nx+1)-
cn(i,2:Nx+1).*temp(i+1,2:Nx+1);
        temp(i,2:Nx+1) =
tdmac(cw(i,3:Nx+1),cp(i,2:Nx+1),ce(i,2:Nx),B,Nx);
    end
    temp(2:Ny+1,2:Nx+1)=(1-
omega).*pk(2:Ny+1,2:Nx+1)+omega.*temp(2:Ny+1,2:Nx+1);

    temp(1,2:Nx+1) = temp(2,2:Nx+1);
    temp(end,2:Nx+1) = temp(end-1,2:Nx+1);
    temp(2:Ny+1,1) = temp(2:Ny+1,2);
    temp(2:Ny+1,end) = temp(2:Ny+1,end-1);
    pk_1 = temp;

    pdiff = pk_1(2:end-1,2:end-1) - pk(2:end-1,2:end-1);
    rtot = rms(pdiff,'all');
    pk = pk_1;
    if rem(k, 100) == 0
        fprintf('k=%d, residual=%f\n', ...
            k, rtot);
    end
end
p = pk_1;

end

%% step 3 un_1
function [un_1,Un_1] =
fractional_step(u_star,U_star,p,dt,dx,dy,Nx,Ny,uv,ifluid)
    un_1 = zeros(Ny+2,Nx+2);
    Un_1 = zeros(Ny+2,Nx+2);
    pcc = zeros(Ny+2,Nx+2);
    pfc = zeros(Ny+2,Nx+2);
    src = zeros(Ny+2,Nx+2);
    if uv == 1
        cw = -ones(Ny+2,Nx+2);
        ce = ones(Ny+2,Nx+2);
        cn = zeros(Ny+2,Nx+2);

```

```

cs = zeros(Ny+2,Nx+2);
cp = zeros(Ny+2,Nx+2);
[cp,ce,cw,cn,cs,src] = coeff_p(cp,ce,cw,cn,cs,src,ifluid,Nx,Ny);
pcc(2:Ny+1,2:Nx+1) =
(ce(2:Ny+1,2:Nx+1).*p(2:Ny+1,3:Nx+2)+cp(2:Ny+1,2:Nx+1).*p(2:Ny+1,2:Nx+1)+cw
(2:Ny+1,2:Nx+1).*p(2:Ny+1,1:Nx))/2/dx;

cw = -ones(Ny+2,Nx+2);
ce = zeros(Ny+2,Nx+2);
cn = zeros(Ny+2,Nx+2);
cs = zeros(Ny+2,Nx+2);
cp = ones(Ny+2,Nx+2);
[cp,ce,cw,cn,cs,src] = coeff_p(cp,ce,cw,cn,cs,src,ifluid,Nx,Ny);
pfc(2:Ny+1,2:Nx+2) =
(cp(2:Ny+1,2:Nx+2).*p(2:Ny+1,2:Nx+2)+cw(2:Ny+1,2:Nx+2).*p(2:Ny+1,1:Nx+1))/d
x;

un_1(2:Ny+1,2:Nx+1) =
ifluid(2:Ny+1,2:Nx+1).*(u_star(2:Ny+1,2:Nx+1)-dt*pcc(2:Ny+1,2:Nx+1));
Un_1(2:Ny+1,2:Nx+2) = U_star(2:Ny+1,2:Nx+2)-dt*pfc(2:Ny+1,2:Nx+2);
else
cw = zeros(Ny+2,Nx+2);
ce = zeros(Ny+2,Nx+2);
cn = ones(Ny+2,Nx+2);
cs = -ones(Ny+2,Nx+2);
cp = zeros(Ny+2,Nx+2);
[cp,ce,cw,cn,cs,src] = coeff_p(cp,ce,cw,cn,cs,src,ifluid,Nx,Ny);
pcc(2:Ny+1,2:Nx+1) =
(cn(2:Ny+1,2:Nx+1).*p(3:Ny+2,2:Nx+1)+cp(2:Ny+1,2:Nx+1).*p(2:Ny+1,2:Nx+1)+cs
(2:Ny+1,2:Nx+1).*p(1:Ny,2:Nx+1))/2/dy;

cw = zeros(Ny+2,Nx+2);
ce = zeros(Ny+2,Nx+2);
cn = zeros(Ny+2,Nx+2);
cs = -ones(Ny+2,Nx+2);
cp = ones(Ny+2,Nx+2);
[cp,ce,cw,cn,cs,src] = coeff_p(cp,ce,cw,cn,cs,src,ifluid,Nx,Ny);
pfc(2:Ny+2,2:Nx+1) =
(cp(2:Ny+2,2:Nx+1).*p(2:Ny+2,2:Nx+1)+cs(2:Ny+2,2:Nx+1).*p(1:Ny+1,2:Nx+1))/d
y;

un_1(2:Ny+1,2:Nx+1) =
ifluid(2:Ny+1,2:Nx+1).*(u_star(2:Ny+1,2:Nx+1)-dt*pcc(2:Ny+1,2:Nx+1));
Un_1(2:Ny+2,2:Nx+1) = U_star(2:Ny+2,2:Nx+1)-dt*pfc(2:Ny+2,2:Nx+1);

```

```

        end
    end

%% Step 4 Residual
function [max_residual,sum_residual] = residual(un_p1,un,vn_p1,vn,p,p_old)
    dff_u = un_p1(2:end-1,2:end-1)-un(2:end-1,2:end-1);
    dff_v = vn_p1(2:end-1,2:end-1)-vn(2:end-1,2:end-1);
    dff_p = p(2:end-1,2:end-1)-p_old(2:end-1,2:end-1);
    max_residual = max(abs(dff_u) + abs(dff_v) + abs(dff_p), [], 'all');
    sum_residual = rms(dff_u + dff_v+dff_p,'all');
end

%% Boundary condition
function [un,Un,vn,Vn] = BC(un_old,Un_old,vn_old,Vn_old,dy,Ly,Nx,Ny,ifluid)
    % left
    Un_old(:,2) = 1;
    un_old(:,1) = 2-un_old(:,2);
    vn_old(:,1) = -vn_old(:,2);
    % right
    Un_old(:,end) = Un_old(:,end-1);
    un_old(:,end) = un_old(:,end-1);
    vn_old(:,end) = vn_old(:,end-1);
    % upper
    un_old(end,:) = un_old(end-1,:);
    vn_old(end,:) = -vn_old(end-1,:);
    Vn_old(end,:) = 0;
    % bottom
    un_old(1,:) = un_old(2,:);
    vn_old(1,:) = -vn_old(2,:);
    Vn_old(2,:) = 0;
    % inner
    Un_old(2:Ny+1,2:Nx+2) =
ifluid(2:Ny+1,2:Nx+2).*Un_old(2:Ny+1,2:Nx+2).*ifluid(2:Ny+1,1:Nx+1);
    Vn_old(2:Ny+2,2:Nx+1) =
ifluid(2:Ny+2,2:Nx+1).*Vn_old(2:Ny+2,2:Nx+1).*ifluid(1:Ny+1,2:Nx+1);
    % update
    un = un_old;
    vn = vn_old;
    Un = Un_old;
    Vn = Vn_old;
end

function [un,Un,vn,Vn] =
BC_wall(un_old,Un_old,vn_old,Vn_old,dy,Ly,Nx,Ny,ifluid)

```

```

% left
Un_old(:,2) = 1;
un_old(:,1) = 2-un_old(:,2);
vn_old(:,1) = -vn_old(:,2);
% right
Un_old(:,end) = un_old(:,end-1);
un_old(:,end) = un_old(:,end-1);
vn_old(:,end) = vn_old(:,end-1);
% upper
un_old(end,:) = -un_old(end-1,:);
vn_old(end,:) = -vn_old(end-1,:);
Vn_old(end,:) = 0;
% bottom
un_old(1,:) = -un_old(2,:);
vn_old(1,:) = -vn_old(2,:);
Vn_old(2,:) = 0;
% inner
Un_old(2:Ny+1,2:Nx+2) =
ifluid(2:Ny+1,2:Nx+2).*Un_old(2:Ny+1,2:Nx+2).*ifluid(2:Ny+1,1:Nx+1);
Vn_old(2:Ny+2,2:Nx+1) =
ifluid(2:Ny+2,2:Nx+1).*Vn_old(2:Ny+2,2:Nx+1).*ifluid(1:Ny+1,2:Nx+1);
% update
un = un_old;
vn = vn_old;
Un = Un_old;
Vn = Vn_old;
end

function [cp,ce,cw,cn,cs,src] = coeff_u(cp,ce,cw,cn,cs,src,ifluid,Nx,Ny)
cp(2:Ny+1,2:Nx+1) = ifluid(2:Ny+1,2:Nx+1).*(cp(2:Ny+1,2:Nx+1)- ...
(1-ifluid(2:Ny+1,1:Nx)).*cw(2:Ny+1,2:Nx+1)- ...
(1-ifluid(2:Ny+1,3:Nx+2)).*ce(2:Ny+1,2:Nx+1)- ...
(1-ifluid(3:Ny+2,2:Nx+1)).*cn(2:Ny+1,2:Nx+1)- ...
(1-ifluid(1:Ny,2:Nx+1)).*cs(2:Ny+1,2:Nx+1))+ ...
(1-ifluid(2:Ny+1,2:Nx+1)));
cw(2:Ny+1,2:Nx+1) =
ifluid(2:Ny+1,2:Nx+1).*(ifluid(2:Ny+1,1:Nx).*cw(2:Ny+1,2:Nx+1));
ce(2:Ny+1,2:Nx+1) =
ifluid(2:Ny+1,2:Nx+1).*(ifluid(2:Ny+1,3:Nx+2).*ce(2:Ny+1,2:Nx+1));
cw(2:Ny+1,2:Nx+1) =
ifluid(2:Ny+1,2:Nx+1).*(ifluid(3:Ny+2,2:Nx+1).*cn(2:Ny+1,2:Nx+1));
cs(2:Ny+1,2:Nx+1) =
ifluid(2:Ny+1,2:Nx+1).*(ifluid(1:Ny,2:Nx+1).*cs(2:Ny+1,2:Nx+1));
src(2:Ny+1,2:Nx+1) = ifluid(2:Ny+1,2:Nx+1).*src(2:Ny+1,2:Nx+1);

```

end

```
function [cp,ce,cw,cn,cs,src] = coeff_p(cp,ce,cw,cn,cs,src,ifluid,Nx,Ny)
    cp(2:Ny+1,2:Nx+1) = ifluid(2:Ny+1,2:Nx+1).*(cp(2:Ny+1,2:Nx+1)+ ...
        (1-ifluid(2:Ny+1,1:Nx)).*cw(2:Ny+1,2:Nx+1)+ ...
        (1-ifluid(2:Ny+1,3:Nx+2)).*ce(2:Ny+1,2:Nx+1)+ ...
        (1-ifluid(3:Ny+2,2:Nx+1)).*cn(2:Ny+1,2:Nx+1)+ ...
        (1-ifluid(1:Ny,2:Nx+1)).*cs(2:Ny+1,2:Nx+1)) + ...
        (1-ifluid(2:Ny+1,2:Nx+1));
    cw(2:Ny+1,2:Nx+1) =
    ifluid(2:Ny+1,2:Nx+1).*(ifluid(2:Ny+1,1:Nx).*cw(2:Ny+1,2:Nx+1));
    ce(2:Ny+1,2:Nx+1) =
    ifluid(2:Ny+1,2:Nx+1).*(ifluid(2:Ny+1,3:Nx+2).*ce(2:Ny+1,2:Nx+1));
    cn(2:Ny+1,2:Nx+1) =
    ifluid(2:Ny+1,2:Nx+1).*(ifluid(3:Ny+2,2:Nx+1).*cn(2:Ny+1,2:Nx+1));
    cs(2:Ny+1,2:Nx+1) =
    ifluid(2:Ny+1,2:Nx+1).*(ifluid(1:Ny,2:Nx+1).*cs(2:Ny+1,2:Nx+1));
    src(2:Ny+1,2:Nx+1) = ifluid(2:Ny+1,2:Nx+1).*src(2:Ny+1,2:Nx+1);
end
```

```
function [x] = tdmac(a,b,c,d,n)
    for i=2:n
        b(i) = b(i) - c(i-1)/b(i-1)*a(i-1);
        d(i) = d(i) - d(i-1)/b(i-1)*a(i-1);
    end
    x(n) = d(n)/b(n);
    for i=n-1:-1:1
        x(i) = (d(i) - c(i)*x(i+1))/b(i);
    end
end
```

2. Post-processing code:

```
%% Contours
i = 100/0.5;
directory = 'E:\Learning Materials\3. JHU\SP24\Computational Fluid
Dynamics\final project\cylinder';

% Pressure contour
[X, Y] = meshgrid(linspace(0, Lx, Nx), linspace(0, Ly, Ny));
figure
contourf(X, Y, p_sample(2:end-1,2:end-1,i),50,'LineStyle','none')
colorbar('southoutside');
```

```

load('colorData.mat','BuDRd_18')
colormap(othercolor('BuDRd_18'));
shading flat;
setPivot(0)
title(['Pressure contour at ', sprintf('Re=%d ',Re), '(resolution: ',
sprintf('%.d',Nx), '\times', sprintf('%d',Ny))]
axis equal
xlim([0 Lx])
ylim([0 Ly])
filename = sprintf('p_re%d_%ds.png', Re, i*0.5);
filepath = fullfile(directory, filename);
saveas(gcf, filepath);

% u Velocity contour
[X, Y] = meshgrid(linspace(0, Lx, Nx), linspace(0, Ly, Ny));
figure('Position', [100, 100, 600, 400]);
scale_factor = 2;
velocity_magnitude = sqrt(un(2:Ny+1,2:Nx+1).^2 +vn(2:Ny+1,2:Nx+1).^2);
contourf(X, Y, u_sample(2:Ny+1,2:Nx+1,i),50, 'LineColor', 'none');
colorbar('southoutside');
shading flat;
axis equal
title(['u velocity contour at ', sprintf('Re=%d ',Re), '(resolution: ',
sprintf('%.d',Nx), '\times', sprintf('%d',Ny))]
xlim([0 Lx])
ylim([0 Ly])
filename = sprintf('un_re%d_%ds.png', Re, i*0.5);
filepath = fullfile(directory, filename);
saveas(gcf, filepath);

% v Velocity contour
[X, Y] = meshgrid(linspace(0, Lx, Nx), linspace(0, Ly, Ny));
figure('Position', [100, 100, 600, 400]);
scale_factor = 2;
velocity_magnitude = sqrt(un(2:Ny+1,2:Nx+1).^2 +vn(2:Ny+1,2:Nx+1).^2);
contourf(X, Y, v_sample(2:Ny+1,2:Nx+1,i),50, 'LineColor', 'none');
colorbar('southoutside');
shading flat;
axis equal
title(['v velocity contour at ', sprintf('Re=%d ',Re), '(resolution: ',
sprintf('%.d',Nx), '\times', sprintf('%d',Ny))]
xlim([0 Lx])
ylim([0 Ly])
filename = sprintf('vn_re%d_%ds.png', Re, i*0.5);

```



```

filepath = fullfile(directory, filename);
saveas(gcf, filepath);

% Vorticity contour
[X, Y] = meshgrid(linspace(0, Lx, Nx), linspace(0, Ly, Ny));
u = u_sample(2:Ny+1, 2:Nx+1, i);
v = v_sample(2:Ny+1, 2:Nx+1, i);
[w1,w2] = curl(X,Y,u,v);
figure
contourf(X, Y, w1, 80, 'LineStyle', 'none');
load('colorData.mat','BuDRd_18')
colormap(othercolor('BuDRd_18'));
shading flat;
colorbar('southoutside');
setPivot(0)
axis equal
title(['Vorticity contour at ', sprintf('Re=%d ',Re), '(resolution: ',
sprintf('%.d',Nx), '\times', sprintf('%d'),Ny)])
filename = sprintf('vor_re%d_%ds.png', Re, i*0.5);
filepath = fullfile(directory, filename);
saveas(gcf, filepath);

close all;

%% t vs Drag Coefficient
[bdsurf,bdn,bds,bde,bdw]=bodycontact(1-ifluid);
for i = 1:n/rsampling
    pre=p_sample(:, :, i);
    dragw=bdw.*pre;
    drage=bde.*pre;
    fdrag=sum(sum(dragw))*dy-sum(sum(drage))*dy;
    cd(i)=(-2*fdrag);
    t(i)=0.5*i;
end
figure;
plot(t,cd)
xlabel('t');
ylabel('C_D');
title('Pressure drag coefficient variation with time')
cdpm=mean(mean(cd));

%% t vs Lift Coefficient
[bdsurf,bdn,bds,bde,bdw]=bodycontact(1-ifluid);
for i = 1:n/rsampling

```

```

    pre=p_sample(:, :, i);
    drags=bds.*pre;
    dragn=bdn.*pre;
    fdrag=sum(sum(drags))*dy-sum(sum(dragn))*dy;
    cl(i)=(-2*fdrag);
    tl(i)=0.5*i;
end
figure;
plot(tl, cl)
xlabel('t');
ylabel('C_L');
title('Lift coefficient variation with time')
cdlm = mean(mean(cl));

%% Frequency
[bdsurf, bdn, bds, bde, bdw]=bodycontact(1-ifluid);
p_post = zeros(Ny+2, Nx+2, 100);
tp = zeros(100, 1);
cdp = zeros(100, 1);
p_post(:, :, 1:100) = p_sample(:, :, 101:1:200); % Extract 50s~100s
for i = 1:100
    pre=p_post(:, :, i);
    dragw=bdw.*pre;
    drage=bde.*pre;
    fdrag=sum(sum(dragw))*dy-sum(sum(drage))*dy;
    cdp(i)=(-2*fdrag);
    tp(i)=0.5*(i+100);
end
figure;
plot(tp, cdp)
xlabel('tp');
ylabel('C_{Dp}');
title('Pressure drag coefficient variation with time')

L=length(cdp);
Fs = 2;
f = Fs*(0:(L/2))/L;
Y=fft(cdp);
P2 = abs(Y/L);
P1 = P2(1:L/2+1);
P1(2:end-1) = 2*P1(2:end-1);
figure
plot(f(2:end), P1(2:end))
title('Spectrum of pressure drag coefficient')

```

```

xlabel("Frequency (f_D)")
ylabel("Spectrum")

%% Functions
function setPivot(varargin)
if nargin==0
    ax=gca;pivot=0;
else
    if isa(varargin{1},'matlab.graphics.axis.Axes')
        ax=varargin{1};
        if nargin>1
            pivot=varargin{2};
        else
            pivot=0;
        end
    else
        ax=gca;pivot=varargin{1};
    end
end
try
    CLimit=get(ax,'CLim');
catch
end
try
    CLimit=get(ax,'ColorLimits');
catch
end
CMap=colormap(ax);
CLen=[pivot-CLimit(1),CLimit(2)-pivot];
if all(CLen>0)
    [CV,CInd]=sort(CLen);
    CRatio=round(CV(1)/CV(2).*300)./300;
    CRatioCell=split(rats(CRatio),' ');
    if length(CRatioCell)>1
        Ratio=[str2double(CRatioCell{1}),str2double(CRatioCell{2})];
        Ratio=Ratio(CInd);
        N=size(CMap,1);
        CList1=CMap(1:floor(N/2),:);
        CList2=CMap((floor(N/2)+1):end,:);
        if mod(N,2)~=0
            CList3=CList2(1,:);CList2(1,:)=[];
            CInd1=kron((1:size(CList1,1))',ones(Ratio(1)*2,1));
            CInd2=kron((1:size(CList2,1))',ones(Ratio(2)*2,1));

```

```

CMap=[CList1(CInd1,:);repmat(CList3,[Ratio(1)+Ratio(2),1]);CList2(CInd2,:)];
    else
        CInd1=kron((1:size(CList1,1))',ones(Ratio(1),1));
        CInd2=kron((1:size(CList2,1))',ones(Ratio(2),1));
        CMap=[CList1(CInd1,:);CList2(CInd2,:)];
    end
    colormap(ax,CMap);
end
end
end

function [bdsurf,bdn,bds,bde,bdw]=bodycontact(Geobd)
    nx=size(Geobd,2); ny=size(Geobd,1);
    bdn=[Geobd(2:end,:);zeros(1,nx)]-Geobd;
    bds=[zeros(1,nx);Geobd(1:end-1,:)]-Geobd;
    bde=[Geobd(:,2:end),zeros(ny,1)]-Geobd;
    bdw=[zeros(ny,1),Geobd(:,1:end-1)]-Geobd;
    bdn(bdn<0)=0;bds(bds<0)=0;
    bde(bde<0)=0;bdw(bdw<0)=0;
    bdsurf=bdn+bds+bde+bdw; bdsurf(bdsurf>0)=1;
end

```